

408 四科考点精讲

以考研助手 AI-Passport 设备为贯穿案例 • 数据结构 / 计组 / 操作系统 / 计网

目录（点击可跳转）

408 四科考点精讲（以考研助手 AI-Passport 设备为贯穿案例）	6
○、总纲：先建立"知识地图"，再逐点填肉	6
复习节奏建议（三轮制）	6
第一大部分 数据结构（先把"怎么组织"想清楚）	7
D1 绪论：什么是数据结构 / 抽象数据类型 ADT / 算法复杂度	7
1.1 数据结构的三要素（背，这个直接决定后面怎么记）	7
1.2 抽象数据类型 ADT（概念题/简答题）	7
1.3 算法与复杂度（必考、送分）	8
D1 练习题	8
D2 线性表（顺序表 + 链表）	9
2.1 顺序表（Sequential List）	9
2.2 单链表（Singly Linked List）	9
2.3 双链表 / 循环链表	10
2.4 顺序表 vs 链表（必背对照表）	10
2.5 易错点	10
D2 练习题	10
D3 栈、队列（★循环队列大题，本章是重点）	11
3.1 栈（Stack）	11
3.2 队列（Queue）	11
★3.3 循环队列（重点，计算题必考）	11
3.4 双端队列（Deque）、优先队列	12
3.5 易错点	13
D3 练习题	13
D4 串（模式匹配：BF 与 KMP）	13
4.1 串的基本概念	13
4.2 串的存储	13
4.3 ★模式匹配	13
4.4 本项目落点	14
D4 练习题	14
D5 树与二叉树（第二大重点，记忆量最大）	14
5.1 树的基本概念	14
5.2 ★二叉树的重要性质（必背五条 + 每题都要能推）	15
5.3 二叉树的存储	15
5.4 二叉树的遍历（必背 + 会写递归/掌握非递归用栈/队列）	15
5.5 线索二叉树（线索化）	16

5.6 二叉排序树 BST (二叉查找树)	16
5.7 平衡二叉树 AVL	16
5.8 哈夫曼树 / 哈夫曼编码 (★ WPL 计算 + 构造)	16
5.9 堆 (Heap, 优先队列的实现)	17
5.10 树的应用 (本项目直接对应)	17
D5 练习题	17
D6 图	18
6.1 图的基本概念	18
6.2 图的存储 (★ 必会)	18
6.3 图的遍历 (必背)	18
6.4 最小生成树 MST (★ 计算题)	18
6.5 最短路径 (★ 必考计算)	18
6.6 拓扑排序与关键路径 (● 结合项目"任务依赖")	19
6.7 易错点	19
D6 练习题	19
D7 查找 (★ 哈希是重灾区)	19
7.1 顺序查找	19
7.2 二分查找 (★ 必须会手写与算 ASL)	19
7.3 二叉排序树 / 平衡树 / B 树 / 散列	20
7.4 ★散列 (哈希) —— 分值最高, 务必多练	20
D7 练习题	21
D8 排序 (★ 复杂度与稳定性必背 + 手推堆排/快排)	21
8.1 八大排序总表 (必须背熟)	21
8.2 各算法灵魂 (理解才有底气)	22
8.3 排序是否需要"比较" (概念):	23
8.4 项目落点	23
D8 练习题	23
【数据结构部分小结】 (背这一页)	23
第二部分 计算机组成原理 (★考得最细、分值最重, 务必花最多时间)	24
C1 计算机系统概述: 层次结构、性能指标 (袁 § 1)	24
1.1 计算机系统的层次 (由顶到底, 抽象逐层下降)	24
1.2 计算机的性能指标 (必背公式)	24
1.3 从 C 语言到 bin 的完整旅程 (★本项目最应深讲的一段, 考纲多次以它为背景)	25
1.4 段的细节与"代码在 Flash、变量在 RAM" (必懂)	26
1.5 链接优化: 函数级裁剪 (结合本项目 16k 内存紧张)	26
C1 练习题	26
C2 数据的表示与运算 (对应袁 § 2/ § 3, 选择里算分之王)	26
2.1 数制与真值/机器数	27
2.2 原码 / 反码 / 补码 / 移码 (n 位有效数值, 1 位符号, 总 n+1 位)	27
2.3 ★补码运算与溢出判断 (选择/大题高频)	27
2.4 定点数与 Q 格式 (本项目核心, 一定要懂)	29
2.5 IEEE 754 浮点 (★背诵 + 手算)	29

2.6 校验码 (选做, 但常考)	29
C2 练习题	30
C3 存储系统 (对应表 § 6, ★计算重地)	30
3.1 存储系统的层次与局部性 (背概念)	30
3.2 SRAM / DRAM / Flash (记忆 + 特性)	31
3.3 主存与"按字节寻址"、"地址空间" "分区 (本项目直接实践)"	31
3.4 ★Cache 与地址映射 (计算大头, 务必会算)	32
3.5 替换算法与写策略	33
3.6 虚拟存储器 (★页表 / TLB / 缺页, 408 大题)	34
C3 练习题	35
C4 指令系统 (对应表 § 4, 指令编码是亮点)	35
4.1 指令格式 (RISC-V RV32 有 6 种基本格式, 重点 R 与 I)	35
4.2 寻址方式 (背 8 种 + 各举一例)	35
4.3 分支与跳转的范围	36
C4 练习题 (指令编码, 408 风格)	36
C5 中央处理器 (对应表 § 5, ★流水线与中断是大题)	36
5.1 数据通路: 五阶段取指执行模型	36
5.2 硬布线 vs 微程序 (选择高频)	37
5.3 ★指令流水线 (计算 + 冒险)	37
5.4 异常与中断 (★必背流程, 本机整流/闪退都在讲)	38
5.5 中断向量表、上下文切换	38
C5 练习题	38
C6 总线与 I/O (对应表 § 7, ★DMA 重点)	39
6.1 总线基础	39
6.2 I/O 的三种控制方式 (★对照表必背)	39
6.3 缓冲与通道	39
C6 练习题	40
[第二部分小结: 计组的"一根主线 + 七个计算" (背)]	40
第三大部分 操作系统 (管"资源", 计算与 PV 是大头)	41
O1 OS 概述与运行环境	41
1.1 OS 的功能与特征 (背)	41
1.2 内核态 / 用户态 (特权级)	41
1.3 系统调用 (system call)	41
1.4 中断在 OS 中的地位 (与计组衔接)	41
O1 练习题	42
O2 进程与线程 (★调度、同步、死锁是全年重点)	42
2.1 进程、线程、PCB	42
2.2 进程的状态与转换 (三态/五态)	42
2.3 上下文切换 (context switch)	42
2.4 ★处理机调度 (调度算法 + 会算平均周转/等待)	43
2.5 同步与互斥 (★PV 操作, 大题)	43
2.6 死锁 (★判断与避免可计算)	44

O2 练习题	45
O3 内存管理 (★分页、页面置换计算是大题)	46
3.1 连续分配	46
3.2 页式存储 (★必会换算)	46
3.3 分段 / 段页式	46
3.4 ★页面置换算法 (★计算大题)	47
3.5 页面分配、抖动、TLB (快表)	47
O3 练习题	48
O4 文件管理 (★索引文件、磁盘调度是大题)	48
4.1 文件与逻辑结构	48
4.2 文件的物理结构 (★重点)	48
4.3 目录与 FCB、路径	48
4.4 空闲空间管理与磁盘调度 (★计算)	49
4.5 文件一致性 / 原子性 (本项目活案例)	49
O4 练习题	49
O5 I/O 管理 (与计组 C7 呼应)	49
5.1 I/O 系统的软件层次	49
5.2 缓冲技术	50
5.3 SPOOLing (假脱机技术)	50
5.4 设备分配	50
O5 练习题	50
[第三部分小结: OS 的"六张计算表" (背)]	50
第四大部分 计算机网络 (把"跨机器传数据"讲透)	51
N1 概述与分层	51
1.1 为什么要分层 (背)	51
1.2 OSI 七层 vs TCP/IP 五层 (本项目 LwIP 用五层)	51
1.3 性能指标 (背 + 会算)	51
N1 练习题 (时延计算)	51
N2 物理层	52
2.1 编码与调制	52
2.2 复用技术 (★考计算/概念)	52
2.3 传输介质与极限速率	52
N2 练习题	52
N3 数据链路层 (★CRC、窗口、CSMA 是大题)	53
3.1 封装成帧与链路标识	53
3.2 差错检测 (★CRC 会算)	53
3.3 可靠传输: 停止等待、GBN、SR (★窗口计算)	53
3.4 ★介质访问控制 (随机访问 MAC)	54
3.5 以太网、VLAN	54
N3 练习题	54

N4 网络层 (★IP、子网划分、路由是大题)	54
4.1 IPv4 与地址分类 (背)	54
4.2 ★子网划分与 CIDR (送分计算, 务必刷熟)	55
4.3 ARP / DHCP / ICMP / 路由 (背流程)	55
4.4 NAT (网络地址转换)	55
N4 练习题	55
N5 传输层 (★TCP 可靠/流量/拥塞控制是全年大题必守)	56
5.1 UDP vs TCP 对比 (背)	56
5.2 TCP 报文段 (背关键字段)	56
5.3 可靠传输与流量控制 (★计算)	56
5.4 ★TCP 拥塞控制 (★状态机, 背死)	56
N5 练习题	57
N6 应用层 (★HTTP 讲透, 本项目配网/下载全靠它)	57
6.1 DNS (域名系统)	57
6.2 ★HTTP (本项目配网 + 下载的核心协议)	57
6.3 其他应用层协议 (背)	58
N6 练习题	58
[第四部分小结: 计网"一张分层图 + 四类必算" (背)]	59
第五大部分 专题回顾 + 综合演练 (把四科串起来)	60
◆ 专题一: 按一次按键, 发生了什么? (跨计组 + OS + 数据结构)	60
◆ 专题二: 录音 16kHz 崩溃 (ESP_RST_PANIC) 背后是哪个考点? (跨计组 + OS)	60
◆ 专题三: 配网 + 下载录音 = 整个计网章节的"活体演练"	61
第六大部分 全书复盘 & 考前冲刺清单	62
6.1 三科主线的"一口闷"记忆卡	62
6.2 最容易记混 & 高频陷阱 (按科列出, 考前必扫)	62
6.3 考点优先级 (按 408 历年出现频率)	62
6.4 最后: 复习节奏建议 (回到总纲)	63

408 四科考点精讲（以考研助手 AI-Passport 设备为贯穿案例）

用途：考研 408 统考系统复习资料。紧扣《408 考试大纲》，把「数据结构 / 计算机组成原理 / 操作系统 / 计算机网络」四科每个考点的 定义 → 存储结构 → 原理推导 → 公式/规则 → 易错点 → 本项目硬件落点 → 典型例题+手写解析 一次讲透。

案例载体：一台使用 ESP32-C3(RISC-V 内核) 的录音学习机（真实代码可对照仓库）。它把四科抽象概念几乎全部具象化，你学到每个考点，都能在这台设备里"摸到"它。因此我尽量把每一条理论都落到这台设备的具体行为上，让"抽象"变成"看得见"。

〇、总纲：先建立"知识地图"，再逐点填肉

408 的四科不是四门孤立学科，而是一条"从问题到机器"的主线。用这台设备串起来就是：

你按下的物理按键（硬件 I/O）
| 中断 → CPU 取指 → 译码 → 执行（计组）
到了哪个 C 函数、哪个任务（数据结构/OS）
| 录成 FRC 文件存进分区（文件系统 OS）
| 通过 WiFi / TCP / HTTP 发给手机（计网）

四科各自负责管一块：- **数据结构**：解决"数据怎么组织、算法怎么快"——你在写任何 for / 链表 / 队列时都在用。- **计组**：解决"机器到底怎么算"——从 int16_t 的补码到 CPU 的每条指令，再到 Cache、加法器、中断。- **OS**：解决"多个程序怎么共享资源"——任务切换、内存分配、文件系统、中断管理。- **计网**：解决"数据怎么跨机器传"——IP、TCP、HTTP、配网、下载。

四科都从"这台设备"出发的好处：任何一道抽象真题，你都能先想"如果是我的设备会怎样"，再套理论。这就是把知识"长"在身上，而不是"背"进脑。

复习节奏建议（三轮制）

- **第一轮（打地基）**：逐科过一遍本文档"定义+原理"，每个 ★ 例题手推一遍。
- **第二轮（刷计算）**：只做标记为"送分计算"的题（Cache 容量、子网划分、页面置换、TCP cwnd），刷熟公式。
- **第三轮（串考点）**：用文末"综合演练"把多科考点串起来，检验"知识不再是一块一块，而是一张网"。

第一大部分 数据结构（先把"怎么组织"想清楚）

数据结构考的是"你怎么选择和组织数据，让关键操作最快"。408 里数据结构的题分值不高但必考，且选择题考得细（复杂度、性质、公式），大题考代码/手推过程。下面每章按"定义→存储→操作→复杂度→易错→例题"讲。

D1 绪论：什么是数据结构 / 抽象数据类型 ADT / 算法复杂度

1.1 数据结构的三要素（背，这个直接决定后面怎么记）

一个数据结构 = 三要素同时成立：

要素	含义	项目实例
逻辑结构	元素之间的抽象逻辑关系（与存储无关）	录音任务列表"顺序"关系；科目→子类→任务"树"关系
存储结构	逻辑结构在机器里怎么实现（内存怎么摆）	任务用顺序表（数组）存；目录用链表/树存
数据的运算	在逻辑结构上定义的操作（增删改查、排序等）	list_add、find_next、rename

- ① 逻辑结构（四种，必考选择题）：- 集合：元素间无关系（如一袋子无序的丢包记录）。- 线性结构：一对一（线性表、栈、队、串）。- 树形结构：一对多（树、二叉树）。- 图形/网状结构：多对多（图）。
- ② 存储结构（四种，也必考）：- 顺序存储：物理连续，逻辑相邻即物理相邻 —— 本项目任务列表 `study_task_t[256]`、录音 PCM 的 `int16_t buf[]`。- 链式存储：用指针把物理上分散的节点串起来 —— 本项目目录树、FreeRTOS 任务链表。- 索引存储：建一张"关键字→位置"的索引表，先查索引再取数据 —— 本项目 `find_task(title)` 先查名字。- 散列存储：按关键字直接算出存储位置， $O(1)$ 定位 —— 本项目 NVS 的"键值"查找（`find->get`）。

记忆法：逻辑结构是"关系"，存储结构是"实现"，运算定义在逻辑结构上但复杂度取决于存储结构。例如"找第 k 个元素"：逻辑上是线性关系；用顺序存储则 $O(1)$ ，用链表则 $O(k)$ 。

1.2 抽象数据类型 ADT（概念题/简答题）

- 定义：一个数学模型 + 定义在该模型上的一组操作。
- 三要素：数据对象、数据关系、基本操作。
- ADT 与实现分离：使用方只看操作（`push/pop`、`insert/delete`），不关心内部是用数组还是链表。
- C 语言两个实现手段：1. 结构体 + 函数指针表（`vtable` / 虚函数表）：一个结构体里放一组函数指针，调用时通过指针调用 —— 本项目任务存储抽象 `study_task_store_t` 就是：`add/remove/save/load` 都定义成可替换的接口，底层可以是 SPIFFS 也可以是 NVS。2. `typedef struct` + 操作函数：如 `typedef struct{...} Queue; Queue_new(); Queue_push();`。

一句话：ADT = "你说我要什么"（接口），实现 = "我具体怎么做"（细节）。考研常考"ADT 的优点"：封装、信息隐藏、可复用、便于维护。

1.3 算法与复杂度（必考、送分）

算法的五个特性：有穷性、确定性、可行性、有输入、有输出。（背顺口溜"穷定可行，有输入出"）

时间复杂度 $T(n)$ ：算法执行基本操作次数的渐进上界。通常用大 O 表示，忽略系数和低阶项。- 注意常数项： $T(n)=3n+2 \rightarrow O(n)$ ， $T(n)=2n^2+n \rightarrow O(n^2)$ 。- 双重循环不一定 $O(n^2)$ ：for(i..n) for(j..n) 才是 n^2 ；若 for(i..n) for(j=1;j<=n;j*=2) 是 $O(n\log n)$ 。

空间复杂度 $S(n)$ ：算法运行所需额外辅助空间（不含输入本身占用）。- 原地算法（in-place） $S(n)=O(1)$ 。如：冒泡、插入、简单选择、堆排。- 归并需要额外 $O(n)$ 数组；快排递归栈最坏 $O(n)$ 、平均 $O(\log n)$ 。

常用阶比较（必须会排）：

$O(1) < O(\log n) < O(n) < O(n\log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

目录里"循环里套循环看变化量"套路（高频）：- 步长 +1 $\rightarrow O(n)$ ；步长 $\times 2 \rightarrow O(\log n)$ ；内层又套一层外层 $\rightarrow O(n\log n)$ 或 $O(n^2)$ 。- 递归的复杂度：若每次递归把规模减半且每层 $O(n) \rightarrow O(n\log n)$ （如快排、归并）。

本设备落点：- 录音超时检查 if (frames*20 >= REC_MAX_SEC*1000) break; 每条循环都执行、不随数据规模变化 $\rightarrow O(1)$ 。- poll_evt 从环形队列取一个事件，固定几步 $\rightarrow O(1)$ 。- 对 PCM int16_t buf[320] 逐样本编码 $\rightarrow O(320)=O(n)$ （ n 为帧内样本数）。- 查找"下一次到点提醒"：顺序扫任务列表 $\rightarrow O(n)$ （所以列表大了会慢，这是线性查找）。

D1 练习题

Q1：求下列代码复杂度。

```
int count = 0;
for (i = 1; i <= n; i = i * 2)
    for (j = 1; j <= n; j = j * 2)
        count++;
```

解析：外层 i 以 2 倍增长， $i=1,2,4,\dots,2^k \leq n \rightarrow$ 运行次数 $k=\log n$ 层（约 $\log_2 n$ 次）；内层同理 $\log n$ 次。故总数 = $\log n \cdot \log n = O((\log n)^2)$ 。（注意：两个都 $\times 2$ 是 $(\log n)^2$ ，不是 $n\log n$ 。之前那个"外层 n 次、内层 $\log n$ 次"才是 $n\log n$ 。别混淆！）

Q2：for(i=0;i<n;i++) for(j=i;j<n;j++) for(k=0;k<n;k++) s++; 的复杂度约？**解析**：三层，每层都到 n 里变化，最内层总次数 = $n \cdot (n^2/2 \text{ 级别}) \rightarrow O(n^3)$ 。（两层平方即 n^2 ，三层立方。）

Q3（简答/选择）：一个算法能用递归实现，为何还要栈结构？**解析**（本书重要考点）：递归调用时，系统要用调用栈保存每层递归的返回地址、实参、局部变量、现场寄存器，逐层压栈、返回时弹栈。因此"递归程序 =

系统用栈实现", 栈是递归的**必要支撑结构**（对应计组章节的"调用栈/压栈现场", 见 C 部分）。若你把递归改成循环（如本项目把需要反复编码的 PCM 用循环而非递归跑），本质就是**手动模拟栈**。

D2 线性表（顺序表 + 链表）

线性表逻辑上是一对一的线性关系，元素类型相同。它有两种存储：**顺序表**（数组）和**链表**。

2.1 顺序表（Sequential List）

存储结构：元素存于一块连续的地址空间。

下标	0	1	2	3	...	n-1
地址	base	base+L	base+2L	...		

核心公式（必背）：设元素长度 L ，首地址 $base$ ，第 i 个元素（从 0 开始）地址：

$Loc(a_i) = base + i \times L$ (★推导它：每挪一格多 L 个字节)

- 由下标 $O(1)$ 算出地址 → **随机存取**（Random Access）。
- 插入/删除要**搬移元素**：平均移动 $n/2$ 个 → **$O(n)$** 。

插入操作（讲透平移）：在 i 位置插入一个元素，需把 $i \sim n-1$ 全部后移一位：- 最坏在头部插：移 n 个；最好在尾部插：移 0 个；平均移 $n/2$ → 时间 **$O(n)$** 。- **删除**同理：把 $i+1 \sim n-1$ 前移，平均移 $(n-1)/2$ → **$O(n)$** 。

顺序表优缺点（背）：优点——随机存取 $O(1)$ 、存储密度高（无指针开销）；缺点——插删慢、需预分配连续大空间、扩容代价大。

2.2 单链表（Singly Linked List）

存储结构：分散的内存节点，靠指针串联。

结点 = [data | next]
^—数据域 ^—指针域(指向下一结点首地址)

带头结点 VS 不带头结点：头结点是"哨兵"，不算数据元素，方便在表头统一插入/删除（**考点：头结点作用**）。

单链表关键操作（背复杂度）：- 按位置/按值查找：从 head 逐个走 → **$O(n)$** （无随机存取）。- 已知 p 指针，在 **p 后面**插入新结点： $c \text{ newNode} \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} = \text{newNode};$ → 只需改两个指针， **$O(1)$** 。- 已知 p 指针，删除 **p 后面**的结点： $q = p \rightarrow \text{next}; p \rightarrow \text{next} = q \rightarrow \text{next}; \text{free}(q);$ → **$O(1)$** 。- 但删除操作通常要先 $O(n)$ 找到 p ，所以"整体删除"仍是 $O(n)$ 。

头插法 / 尾插法（经常问）：- 头插法建表：每次把新结点插到最前面，结果会**逆序**。- 尾插法建表：用 tail 指针尾插，结果保持原顺序。本题设备里"按 R%07u 递增录新文件"若用链表，就是尾插保持递增序。

2.3 双链表 / 循环链表

- **双链表**：结点 [prior | data | next]，可双向遍历，删除给定结点可达 $O(1)$ （不用找前驱）。代价：多一个指针域（存储密度下降）。
- **循环链表**：尾结点 next 指回头结点（head）。判断表空：head->next==head。
- 好处：从任一结点出发都能遍历整个表。
- 循环双链表让"在表尾插入 $O(1)$ "且"前移 $O(1)$ "。
- 考研常考"从尾到头打印"：用**栈或递归**（等价于先压栈再出栈），也可先反转链表。

2.4 顺序表 vs 链表（必背对照表）

操作	顺序表（数组）	链表
随机访问第 k 个	$O(1)$ （地址公式）	$O(n)$ （逐个走）
按值查找	$O(n)$ （可配合二分若有序）	$O(n)$
表头插/删	$O(n)$ （搬移）	$O(1)$
表尾插/删(指向已知)	$O(1)$	单链 $O(n)$ （要找前驱）/双链 $O(1)$
存储密度	高(1)	低(多指针，通常 $\leq 1/2$)
空间	需预分配连续块	分散、按需分配
适用	查找多、增删少	增删多、查找少

选型结论（本项目直接对应）：录音任务列表保存后要"按序号读取"，用**顺序表**最合适（study_task_t[256] 数组即可随机访问第 i 个任务）；而**任务/目录间的父子关系**是树（用链式）；FreeRTOS 的**就绪任务队列**用链表实现（频繁插入/删除）——这就是"根据操作频率选存储"的活例子。

2.5 易错点

1. **头结点 vs 首元结点**：头结点不是数据元素，尾指针指向头结点才判"空循环链表"。
2. 单链表删除"给定结点"必须知道其前驱；真正 $O(1)$ 删除一个已知结点的 trick：把后继数据复制到当前结点，再删后继——经典面试题，考研偶尔考。
3. 顺序表扩容（realloc）：可能整体搬移数据 $\rightarrow O(n)$ ，这就是"预分配要合理"的原因。

D2 练习题

Q1：有 n 个元素的顺序表，在第 i 个位置（ $1 \leq i \leq n+1$ ）插入元素平均需移动几个元素？复杂度？**解析**： $i=1$ （表头）移 n 个， $i=n+1$ （表尾）移 0 个。平均移动数 $= (n + n-1 + \dots + 0)/(n+1) \approx n/2 \rightarrow \mathbf{O(n)}$ 。（要点：别把 i 记反——表头插最贵。）

Q2：设计判断单链表是否有环（快慢指针）的空间复杂度。**解析**：用快慢两指针，快指针每次走两步、慢指针走一步，若有环则必然追上。只用两个指针 $\rightarrow$ **空间 $O(1)$** 。（若用哈希表记录访问过的结点也行，但空间 $O(n)$ 。**选 $O(1)$ 是得分点。**）

Q3 (综合)：要把一系列数据"尾部拼新数据 + 从头删旧数据"高频执行，顺序表和链表各适合吗？**解析：**该模式=队列。顺序表（循环）时空 $O(1)$ ；链表头删尾加也 $O(1)$ 。但若还要"随机访问第 k 个"，则顺序表胜出。→ 结论：**没有唯一答案，要看操作组合**——这正是 408 最爱考的"根据需求选结构"。

D3 栈、队列（★ 循环队列大题，本章是重点）

3.1 栈 (Stack)

定义：只允许在**一端（栈顶 top）**进行插入/删除的线性表。- 运算：**push**（入栈=栈顶↑）、**pop**（出栈=栈顶↓）、**peek**（读栈顶）、判空、判满。- 特性：**后进先出 LIFO**。

顺序栈（数组实现）：`data[MaxSize] + top`（`top` 指向栈顶元素）。判空 `top == -1`；判满 `top == MaxSize - 1`。- 进栈：`data[++top] = x`；出栈：`x = data[top--]`。

链式栈（链表实现）：表头作为栈顶（头插/头删），进出都 $O(1)$ ，不会满。

栈的典型应用（必背）：1. **递归调用栈**（保存每层返回地址/实参/局部变量/现场）。→ 本项目 C 函数递归、FreeRTOS 任务栈都是这个概念（见计组"调用栈"）。2. **表达式求值**：- **后缀表达式（逆波兰）**求值：遇到操作数压栈，遇到运算符弹出两个操作数计算再压回。- **中缀 → 后缀**：用运算符栈，规则是"优先级高者先输出、括号处理括号"，需手动模拟（大题会考，多练）。3. **括号匹配**：遇左括号压栈，遇右括号弹栈配对，最后栈空则匹配。4. **迷宫/回溯/深度优先遍历**：靠栈记住"退路"。5. **进制转换**（除基取余，余数先进后出）：用栈正好。

本项目落点：FreeRTOS 每个任务对应一块**任务栈**（`uxTaskGetStackHighWaterMark` 看的栈水位就是看栈还剩下多少）——栈满（HWM 极低）= 递归/函数调用过深会溢出，正是之前录音 16k 崩溃的嫌疑点之一。

3.2 队列 (Queue)

定义：只允许**队尾入、队头出**的线性表。特性 **FIFO 先进先出**。运算：**enqueue**（入队）、**dequeue**（出队）、判空、判满。

为什么必须用"环形（循环）队列"：若线性地用数组 `front` 在前、`rear` 在后，出队后 `front` 前移造成前段空位无法复用，空间浪费严重（"假溢出"）。解决办法是用**取模回绕**构成循环队列。

★3.3 循环队列（重点，计算题必考）

设在数组 `data[MaxSize]` 中，用 `front`（队头）、`rear`（队尾）两个指针，按**牺牲一个存储单元**方式判满（这是最常考的实现）：

- 初始化空：`front = rear = 0`。
- 入队（在队尾）：`data[rear] = x; rear = (rear + 1) % MaxSize;`
- 出队（在队头）：`x = data[front]; front = (front + 1) % MaxSize;`
- 判空：`front == rear`。
- 判满：`(rear + 1) % MaxSize == front`。
- 元素个数：`(rear - front + MaxSize) % MaxSize`。（务必理解：尾减头，取模保证非负。）
- 队列最大容量：`MaxSize - 1`（因为要空一格来区分空和满）。

为什么空一个格：若不用空一格，则空态和满态都是 $\text{front} == \text{rear}$ ，无法区分 → 于是"牺牲一格"或者"加一个 size 计数器/flag"。考试用"牺牲一格"最普遍。

三个"推导"要亲自动手一次（别只背公式）： 1. 循环的本质：数组是线性的，把 $\text{data}[\text{MaxSize}-1]$ 的下一个想象成 $\text{data}[0]$ ，即下标 $i = (i+1) \% \text{MaxSize}$ 循环回绕。**取模 % 是把线性数组"卷成环"的那一步**——本项目代码里每个 $\% \text{EVT_N}$ 都是这个意思。 2. 判满为什么 $(\text{R}+1)\% \text{Max} == \text{F}$ ：满的时候尾指针"隔一格紧贴"头指针（front 指向首个元素，rear 指向最后一个空位后的占位格）。因为空了一格，Rear 下一步 +1 就等于 Front。队满时实际存了 $\text{MaxSize}-1$ 个元素。 3. 元素个数公式的直觉：若 $\text{F} \leq \text{R}$ （没绕环），个数 $= \text{R}-\text{F}$ ；若 $\text{R} < \text{F}$ （绕环一次），个数 $= \text{R}-\text{F} + \text{MaxSize}$ 。两者合并就是 $(\text{R}-\text{F} + \text{MaxSize}) \% \text{MaxSize}$ 。

变体（408 很爱出）——用"单独计数器 size"判空满： - 判别不靠牺牲格子，而是维护 size（当前元素个数）。 - 判空： $\text{size} == 0$ ；判满： $\text{size} == \text{MaxSize}$ ；个数直接 = size。→ 这种情况下容量就是 **MaxSize（不再 -1）**，元素个数也不用绕环公式。**做题先看题干是"牺牲一格"还是"size 标志"**，别套错公式。

△ **经典易错辨析：** - 队头 front 指向的是**第一个元素**还是**第一个元素的前一个位置**，不同教材定义不同。408 通常默认**front 指向队头元素、rear 指向队尾元素的下一个空位**。题目若给 $\text{front}=2, \text{rear}=5, \text{MaxSize}=8$ 则元素个数 $= (5-2) \% 8 = 3$ 。**看清题干对 front/rear 语义的定义再代公式**，比死背公式更重要。

本项目真实代码（study_recorder.c 的事件队列）：

```
#define EVT_N 8
// 入队：覆盖最旧（若满则最旧的被顶掉）
s_evt[s_evt_w] = e;
s_evt_w = (s_evt_w + 1) % EVT_N;
if (s_evt_w == s_evt_r) s_evt_r = (s_evt_r + 1) % EVT_N; // 满了丢掉最旧，生产者不阻塞
// 出队：判空
if (s_evt_r == s_evt_w) return (无事件);
e = s_evt[s_evt_r]; s_evt_r = (s_evt_r + 1) % EVT_N;
```

这就是一个**"满则覆盖最旧（丢弃）"**的循环队列变体：判空仍是 $\text{front} == \text{rear}$ ；不同的是它不是"满则阻塞/报错"，而是**丢最旧一条**，保证生产者永不阻塞（适合事件通知场景，丢旧事件比丢新事件可接受）。（对比：标准循环队列"满则阻塞"，适合有界缓冲生产者-消费者——那是 OS 章节的信号量队列。这里只是数据结构层面。）

一道推敲题（循环队列取模）： $\text{MaxSize}=5$ ，已知 $\text{front}=3, \text{rear}=1$ ，问元素个数、是否满、还能入队几个？

解析： 个数 $= (1-3+5) \% 5 = 3$ 。判满： $(1+1) \% 5 = 2 \neq 3 \rightarrow$ 未滿。还能入队： $\text{MaxSize}-1 - \text{个数} = 4-3 = 1$ 个（再入 1 个到 $\text{rear}=2$ ，此时 $(2+1) \% 5 = 3 = \text{front}$ 满）。→ 训练"绕环后也能算"。

3.4 双端队列（Deque）、优先队列

- **双端队列：**两端都能插入/删除的线性表（两侧都开放）。
- 输入受限双端队列：只允许一端入队。
- 输出受限双端队列：只允许一端出队。
- **优先队列：**出队优先取"优先级最大/最小"的元素，通常用**堆**实现（见 D5 堆）。本项目 FreeRTOS 的"定时器/就绪"就带优先级；"哪条录音到点先提醒"可用优先队列。

3.5 易错点

1. 循环队列元素个数公式： $(R-F+Max)\%Max$ ，不是 $R-F$ 。
2. 判空与判满不要搞混：判空 $F==R$ ；判满 $(R+1)\%Max==F$ 。
3. 栈与队列都不能随机存取中间元素，只能靠弹/出。
4. 看清"牺牲一格"还是"加 size 标志"，二者容量/公式不同（前者容量 $Max-1$ ，后者就是 Max ）。
5. 若题目用"front 指向队头前一个位置"的定义，判满/个数公式的写法要相应调整——先定义指针语义再套公式。

D3 练习题

Q1（循环队列经典）：队列容量 $m=5$ （数组5格，牺牲一格判满）。当前 $front=3, rear=4$ 。（i）队内元素数？（ii）还能入队几个？**解析**：- 元素数 $= (R-F+m)\%m = (4-3+5)\%5 = 1$ 。- 判满条件 $(R+1)\%m==F$ ，即 $(4+1)\%5=0 \neq 3 \rightarrow$ 不满，可继续入。- 还能入队的最多个数：从满的条件反推。队列最多容 $m-1=4$ 个元素（牺牲一格），现有1个，还能入 $4-1=3$ 个。验证：入3个后 $R=(4+3)\%5=2$ ，元素数 $=(2-3+5)\%5=4=$ 满；再入 $(R+1)\%5=3==F$ 判满，正好。**要点**：先算个数公式，再用"容量= $m-1$ "算还能装几个。

Q2（栈混洗/structure）：若入栈序列为 1,2,3，下面哪些是可能的出栈序列？- (a) 3,2,1 ✓ 压1压2压3，出3出2出1。- (b) 1,3,2 ✓ 压1出1，压2压3出3出2。- (c) 3,1,2 ✗ 要出3必须先把1,2,3全压入，此时栈顶是2，先出的应是2，出不了1。**解析**：模拟"能出括号里的当前栈顶就不出"。判定的通用法：**任意时刻栈顶元素必须是下一个要出的元素，否则无法出**。真题常考"给定入栈序和出栈序判断是否合法"。

Q3：用栈实现中缀 $a+b*c$ 转后缀并求值步骤简述。**解析**：- 扫到 $a \rightarrow$ 输出； $+$ 入运算符栈； $b \rightarrow$ 输出； $*$ ：和栈顶 $+$ 比， $*$ 优先级高于 $+$ ，入栈； $c \rightarrow$ 输出；结束把栈里 $*, +$ 依次弹出 $\rightarrow$ 后缀 $abc*+$ 。- 求值： $abc*+ \rightarrow a$ 压栈， b 压栈， c 压栈，遇 $*$ 弹 b, c 算 bc ，把结果压回，遇 $+$ 弹 a 和 (bc) 算 $a+(bc)$ 。 $\rightarrow$ 和后缀等价。**要点**：中缀转后缀的优先级比较是要点：栈顶运算符优先级 $\geq$ 当前运算符时，弹出栈顶 $*$ ，当前运算符才入栈；左括号特例。

D4 串（模式匹配：BF 与 KMP）

4.1 串的基本概念

- 串是字符组成的线性表（受到限制的线性表）。逻辑上同线性表，但基本操作常以"子串"为单位。
- 概念：子串（连续的一段）、前缀/后缀、主串、模式串。
- 概念区分：子串必须是连续的；"子序列"可以不连续（考研爱出这个坑）。

4.2 串的存储

- 顺序存储（定长数组 + length）、链式存储。
- 注意 C 风格以 $\backslash 0$ 结尾，length 指有效字符数（不含 $\backslash 0$ ）。

4.3 ★模式匹配

设主串长 n ，模式串长 m 。

BF 算法（朴素/暴力）： - 从主串每个位置开始与模式串逐一比较，一旦失配，主串指针回退到"起始+1"，模式串回到 0，重新比。 - 平均 $O(n+m)$ ，最坏 $O(n \cdot m)$ （如主串全 aaaa...a，模式 aa...aab，每个起点都几乎比到底）。 - 缺点：主串指针**反复回退**，很多已比较过的字符被重复比较。

KMP 算法（重点，背 next 计算）： - 思想：利用模式串自身的"最长相等前后缀"，失配时主串指针**不回退**，只让模式串右移。 - 设 $next[j]$ ：当 $P[j]$ 失配时，模式串跳到 $P[next[j]]$ 继续比。 - next 数组定义（两种版本，考纲常用 " $next[1]=0$ " 这种 1-based 写法）： - $next[j] = P[0..j-1]$ 的**最长相等前缀后缀的长度**（即前缀长度，符号上 $next[j]=k$ 表示前面有 k 个字符是相等的真的前后缀）。 - 约定 $next[1]=0$ （第 1 个字符失配则整体后移 1 格）， $next[j]$ 一般从 0 开始就是首字符。 - 另有 nextval 改进版：消除"失配后跳到的字符和原字符相同"的无效跳转。 - **KMP 时间复杂度： $O(m)$ 求 next + $O(n)$ 匹配 = $O(n+m)$** 。比 BF 好的在于主串不回溯。

（注：next 数组的求出算法需手推模拟，408 结合"给出模式串求 next"出题，务必练 3-5 个实例。）

4.4 本项目落点

- 配网表单解析 parse_form：按 & 拆分 ssid=...&pass=...、按 = 再拆——就是**按分隔符找子串**的串处理。
- 文件名按 R%07u 解析还原录音序号——正则/字符串到数字的转换。

D4 练习题

Q1（KMP 求 next）：主串 $S="abaabc"$ ，模式串 $P="abaabc"$ 。求 next 数组（1-based， $next[1]=0$ ）。**解析：** - $next[1]=0$ 。 - $next[2]$ ：看 $P[1..1]="a"$ ，前缀后缀相等的最长长度=0 $\rightarrow next[2]=1$ 。（2号前"a"没有相等真前后缀，按约定 $next=1$ ） - $next[3]$ ： $P[1..2]="ab"$ ，最长相等前后缀=0 $\rightarrow next[3]=1$ 。 - $next[4]$ ： $P[1..3]="aba"$ ，最长相等前后缀="a"（长度1） $\rightarrow next[4]=2$ 。 - $next[5]$ ： $P[1..4]="abaa"$ ，最长相等前后缀="a"（长度1） $\rightarrow next[5]=2$ 。 - $next[6]$ ： $P[1..5]="abaab"$ ，最长相等前后缀="ab"（长度2） $\rightarrow next[6]=3$ 。所以 $next = [0,1,1,2,2,3]$ 。**要点：**问你"失配在第 j 位时模式串右移到哪再比" $\rightarrow$ 看 $P[0..j-1]$ 最长相等前后缀长度。**Q2：**KMP 相比 BF 最大的改进点是？**解析：**主串指针**不回退**（BF 每次失配都回退）。带来的收益：匹配过程稳定在 $O(n)$ ，不会因"重复比较"退化到 $O(n \cdot m)$ 。**这题是概念送分。**

（KMP 的 next 推导在考试里常要求"算到第几个失配后主串跳到哪"，对照定义一步步推即可，重点是"前后缀要相等且最长"。）

D5 树与二叉树（第二大重点，记忆量最大）

5.1 树的基本概念

- 树 = 有且只有一个根结点的**层次结构**（一对多）。
- **度：**结点拥有的子树个数。树的度 = 所有结点度的最大值。
- **分支结点/内部结点**（度>0）、**叶子/终端结点**（度=0）。
- 二叉树是**度 ≤ 2** 且"每个结点的子树有左右之分"（有序）的树。
- **路径长度：**两结点之间边的条数；**树的路径长度/带权路径长度**见哈夫曼。

5.2 ★二叉树的重要性质（必背五条 + 每题都要能推）

设二叉树含 n 个结点，度为 0/1/2 的结点数分别为 $n_0/n_1/n_2$ ：1. **$n_0 = n_2 + 1$** （叶子数 = 度为 2 结点数 + 1）。- **推导**：边数 = $n - 1$ （ n 个结点 $n - 1$ 条边）；又边数 = $0 \cdot n_0 + 1 \cdot n_1 + 2 \cdot n_2 = n_1 + 2n_2$ 。故 $n - 1 = n_1 + 2n_2$ ，而 $n = n_0 + n_1 + n_2$ ，消去得 $n_0 = n_2 + 1$ 。**必须会推，选择题常考**。2. 第 k 层上最多 **2^{k-1}** 个结点（ $k \geq 1$ ）。3. 深度为 k 的**满二叉树**最多结点数： **$2^k - 1$** 。4. **完全二叉树**（只有最下两层可以不满，且最下层从左到右连续）： n 个结点的完全二叉树深度 = $\lfloor \log_2 n \rfloor + 1$ 。5. 完全二叉树顺序编号（从 1 开始）：编号 i 的结点，**左孩子 $2i$ 、右孩子 $2i+1$ 、双亲 $\lfloor i/2 \rfloor$** 。- 这是“数组存堆、数组存完全二叉树”的数学基础（本项目优先级队列若用数组实现就靠它）。

满二叉树：每一层都满（第 k 层有 2^{k-1} 个）的二叉树。

每一条的“深层理解 + 进制来源”（背性质才能推）：- **性质 1 是最常考选择题**。记住结论“叶子比二度结点多 1”，同时会反推：给你 n 和 n_2 ，立刻能写 $n_0 = n_2 + 1$ 、 $n_1 = n - n_0 - n_2$ 。**完全二叉树里 n_1 只可能是 0 或 1**（因为完全二叉树至多一个一度结点）。- **性质 2/3 一起记**：每层最多 2^{k-1} ，满二叉 k 层共 $1 + 2 + \dots + 2^{k-1} = 2^k - 1$ （等比求和）。- **性质 4 用于“已知结点数求树高”**：深度 = $\lfloor \log_2 n \rfloor + 1$ 。例： $n=15 \rightarrow \lfloor \log_2 15 \rfloor = 3 \rightarrow$ 深 4； $n=16 \rightarrow \lfloor \log_2 16 \rfloor = 4 \rightarrow$ 深 5。（判断“完全二叉但非满”时要 +1。）- **性质 5 是顺序存储的根基**：堆的下沉/上浮全靠 $2i / 2i+1 / \lfloor i/2 \rfloor$ 在数组下标间跳转。**注意编号从 1 开始**；若数组下标从 0 开始，则孩子是 $2i+1 / 2i+2$ 、父亲 $\lfloor (i-1)/2 \rfloor$ ——408 两种下标都可能给，**先看题干编号从 0 还是 1**。

层次/深度计算题（必练）：- 例 A：一棵**完全二叉树**共 767 个结点，问叶子 n_0 、度 1 结点数、树高。**解**：树高 = $\lfloor \log_2 767 \rfloor + 1 = 9 + 1 = 10$ 。 $n = n_0 + n_1 + n_2 = 767$ ，又 $n_0 = n_2 + 1 \rightarrow n_1 = 767 - 2n_2 - 1$ 。完全二叉树 $n_1 \in \{0, 1\}$ ：768 是偶数 $\rightarrow n_1$ 只能为 1（若 $n_1=0$ 则 $767=2n_2+1 \rightarrow n_2=383$ ，但这样 $2n_2+1=767$ ， $n_1=0$ ，须检查叶层…合并推最稳）。直接记结论：**结点数 n 为奇数时 $n_1=0$ ，偶数时 $n_1=1$** （因为 $n = 2n_2 + n_1 + 1$ ， $n-1$ 奇偶决定 n_1 ）。767 为奇数 $\rightarrow n_1=0$ ， $n_2=(767-1)/2=383$ ， $n_0=384$ 。- 例 B：某二叉树的先序与中序或中序与后序能唯一确定，但**先序+后序不能唯一确定**（除非不含单子树结点）。 $\rightarrow$ 常考“由遍历序列建树”的确定性问题。

5.3 二叉树的存储

- **顺序存储**（数组）：适合完全二叉树（可用公式找孩子/父亲）；对一般二叉树浪费空间（很多空位）。
- **链式存储**（常用）：`LNode { data; LChild; RChild }`（二叉链表）；再加 `parent` 即三叉链表（便于向上找父）。

5.4 二叉树的遍历（必背 + 会写递归/掌握非递归用栈/队列）

四种顺序：- **先序（前序）**：根 $\rightarrow$ 左 $\rightarrow$ 右 - **中序**：左 $\rightarrow$ 根 $\rightarrow$ 右 - **后序**：左 $\rightarrow$ 右 $\rightarrow$ 根 - **层序**：按层从上到下、每层从左到右（用队列实现 BFS）。

由遍历序列确定树（高频考点）：- 已知**先序 + 中序** $\rightarrow$ 唯一确定二叉树：先序第一个是根，用根把中序分成左/右子树，递归。- 已知**后序 + 中序** $\rightarrow$ 同理（后序最后一个为根）。- 已知**先序 + 后序** $\rightarrow$ **不能唯一确定**（只能确定根，无法区分左右子树）——这是常考陷阱。

“已知先序+中序恢复树”是必须能手推的。树的特点就是递归结构，用递归恢复。

5.5 线索二叉树（线索化）

- 用结点里空的左/右指针存中序（或先/后序）的前驱/后继，用标志位 LTag/RTag 区分"是指孩子还是线索"。
- 意义：让中序遍历不用栈/递归，直接从线索找后继， $O(1)$ 找前驱/后继。
- 考点：区分线索的 LTag（0 左孩子/null? 1 前驱）、在中序线索树上找前驱后继。

5.6 二叉排序树 BST（二叉查找树）

- 性质：左子树所有 $<$ 根 $<$ 右子树所有。中序遍历 BST = 递增有序序列。
- 查找：从根往下走，平均 $O(\log n)$ ；最坏退化为斜树（链表） $\rightarrow O(n)$ 。
- 插入：总是插入为叶子（不改变树其余部分）。
- 删除（三种情况，必背）：1) 删叶子：直接删。2) 只有一个孩子：把孩子顶上来。3) 两个孩子：用其中序前驱（左子树最大）或中序后继（右子树最小）替换被删结点，再删那个前驱/后继（它最多一个孩子）。
- ASL（平均查找长度）：成功和不成功都考。

5.7 平衡二叉树 AVL

- 定义：任一结点左右子树高度差的绝对值 ≤ 1 ，且都满足（递归）。
- 平衡因子 BF = 左子树高度 - 右子树高度，取值 -1/0/1。
- 插入导致失衡时旋转调整（四个情况）：
- LL（左左，在左孩子的左子树插入） $\rightarrow$ 右单旋。
- RR（右右） $\rightarrow$ 左单旋。
- LR（左右） $\rightarrow$ 先左旋成 LL 再右单旋。
- RL（右左） $\rightarrow$ 先右旋成 RR 再左单旋。
- AVL 的高度：含 n 个结点的 AVL 树最大深度约为 $1.44 \cdot \log_2(n+2)$ （比 n 小得多） $\rightarrow$ 保证查找 $O(\log n)$ 最坏也成立。
- 考点：插入后从低往上找第一个失衡结点调整——调整的对象是"离插入点最近的失衡结点"这个子树，旋转后整体重新平衡。

5.8 哈夫曼树 / 哈夫曼编码（★ WPL 计算 + 构造）

- 哈夫曼树（最优二叉树）：给定一组叶子权值，构造成带权路径长度 WPL 最小的二叉树。
- WPL（带权路径长度）= $\sum$ （叶子的权值 $\times$ 到根的路径长度 = 深度）。
- 构造算法（背步骤）：将 n 个权值看成 n 棵只有根的二叉树；选权值最小的两棵合并成一棵（新根权值和），反复直到剩一棵。
- 每次合并选最小两个 $\rightarrow$ 这是"贪心"思想。
- 哈夫曼编码（前缀码）：左 0 右 1，只有叶子上有对应字符（没有字符在另一字符的祖先位置，所以是无歧义的前缀编码）。频率高的字符编码短、频率低的编码长 $\rightarrow$ 总长度最短。
- 性质（考点）： n 个叶子的哈夫曼树共有 $2n-1$ 个结点（因为哈夫曼树无度为 1 的结点）。
- 本项目联系：FRC 变长编码帧的"自描述"思想 = "高频给短码、低频给长码"（和信息论的前缀码一致）。

5.9 堆 (Heap, 优先队列的实现)

- **堆** = 完全二叉树 + 每个结点 $\geq$ (大根堆) 或 $\leq$ (小根堆) 其孩子。
- 存储: 用**顺序存储 (数组)**, 用完全二叉树编号公式 孩子 $=2i, 2i+1$, 父 $=\lfloor i/2 \rfloor$ 。
- 关键操作:
- **建堆** (从最后一个非叶结点 $\lfloor n/2 \rfloor$ 开始往上做"向下调整"): **$O(n)$** (一次向下 $O(\log n)$, 从 $n/2$ 个位置做, 总量 $O(n)$)。
- **插入**: 先在末尾放新元素然后"向上调整" $\rightarrow O(\log n)$ 。
- **删除堆顶**: 用最后一个元素顶上, 再"向下调整" $\rightarrow O(\log n)$ 。
- 用途: **优先队列、堆排序** (见 D8)。
- 大根堆 vs 小根堆的**向下调整 (sift down)** 要会手写 (排排序大题)。

5.10 树的应用 (本项目直接对应)

- 本项目"科目 $\rightarrow$ 子类 $\rightarrow$ 任务" = **一棵多级树** (根是"全部任务", 中间是分类, 叶子是具体任务)。
- "找下一个到点提醒" = 在这棵树上做遍历, 本质是**先序/层序遍历 + 按时间排序** (可用优先队列 = 堆)。

D5 练习题

Q1: 某二叉树有 $n_0=10$ 个叶子, $n_1=5$ 个度为1的结点, 求结点总数。 **解析**: 由 $n_0=n_2+1 \rightarrow n_2=n_0-1=9$ 。总数 $= n_0+n_1+n_2 = 10+5+9 = 24$ 。 (直接套性质①, 送分。但注意不是所有题都给 n_1 , 可能给你"总边数"让你推。)

Q2 (先序+中序恢复): 先序 ABDECF, 中序 DBE AFC, 恢复二叉树并写后序。 **解析**: - 先序首位 A = 根。中序里 A 把左右分开: 左子树中序 DBE、右子树 FC。 - 左子树: 先序 (A后面的) BDECF 中根 B, 中序 DBE 里 B 的左边 D、右边 E $\rightarrow$ B 左孩子 D、右孩子 E。 - 右子树: CF: 先序 CF, 根 C, 中序 FC 里 C 右边有 F $\rightarrow$ C 右孩子 F。 - 后序 = 左-右-根 = DEB FCA。

Q3 (堆验证 + 建堆): 给序列 {9,5,8,2,3} 建成大根堆。 **解析**: 数组下标 1~5, 初始 9,5,8,2,3。最后一个非叶 $= \lfloor 5/2 \rfloor = 2 \rightarrow$ 对下标2元素5做向下调整: 5的孩子 2,3(对应2和3号) 中较大的3号=3; $5>3$ 需交换 $\rightarrow$ 5和3换? 但5的左孩子是2 (下标4), 右孩子3 (下标5)。取孩子中较大者: 左孩2、右孩3, 较大是3 (值3) 但5与3比不用换 ($5>3$)。再看下标1的9, $9>8>$ 所有, 无需换。结果仍是 9,5,8,2,3。 **要点**: 向下调整是"沿较大孩子的路径下潜", 建堆从 $\lfloor n/2 \rfloor$ 往前做。多练一次"5,2,9,8,3"这类非堆序列的建堆过程。

Q4 (WPL): 字符 A,B,C,D 频次 7,5,2,4, 构造哈夫曼树并求 WPL 与每个编码。 **解析**: 权值 {7,5,2,4}。 - 取最小的 2,4 $\rightarrow$ 合并为6 $\rightarrow$ 剩 {7,5,6}。 - 取最小的 5,6 $\rightarrow$ 合并为11 $\rightarrow$ 剩 {7,11}。 - 合并 7,11 $\rightarrow$ 18 (根)。 - WPL = 7×1 (A 深度1) + $(B\ 5) \times 2$ (若B挂11下深度2) + $(2\ 和\ 4\ 深度3) = 7 \cdot 1 + 5 \cdot 2 + 2 \cdot 3 + 4 \cdot 3 = 7+10+6+12 = 35$ 。 (注意树形一不同 WPL 可能不同, 但所有哈夫曼树 WPL 都最小且相等。) **Q5** (哈夫曼结点数): 用 $n=4$ 片叶子构造哈夫曼树, 共多少个结点? **解析**: 无度为1, 结点总数 $= 2n-1 = 2 \times 4-1 = 7$ 。记住"2n-1"这个送分公式。

D6 图

6.1 图的基本概念

- 图 = 顶点集 V + 边集 E 。
- 无向图 / 有向图；度（无向图中顶点的度=关联边数；有向图分入度/出度）。
- 完全图：无向 n 个顶点 完全图边数 $n(n-1)/2$ ；有向完全图 $n(n-1)$ 。
- 连通/强连通、生成树、连通分量。
- 带权图（网）、稀疏图 ($|E| \ll |V|^2$) / 稠密图。

6.2 图的存储（★ 必会）

- 邻接矩阵： $A[i][j] = 1$ （或权值）表示存在边 (i,j) 。空间 $O(n^2)$ ，适合稠密图；判断"是否存在边 $i \rightarrow j$ "是 $O(1)$ 。无向图的邻接矩阵对称（可存一半省空间）。
- 邻接表：每个顶点一个链表（或 vector）存其邻接点。空间 $O(n+e)$ ，适合稀疏图；求某顶点的出边快。
- 关系：Dijkstra 用邻接矩阵实现较直观；稀疏图上用邻接表省空间。

6.3 图的遍历（必背）

- 深度优先 DFS：用栈（显式或递归），从一个顶点出发尽可能深地走，无路可走时回溯。
- 广度优先 BFS：用队列，一层层扩展（先访问所有邻接点，再访问邻接点的邻接点）。
- 两者都需要 visited 数组防止重复访问。
- 连通分量：一次 DFS/BFS 能到达的顶点集合。非连通图需要多次遍历，次数=连通分量数。
- 本项目落点：WiFi 配网的边界/可达设备发现、任务图的拓扑关系都可用 BFS/DFS；录音文件依赖关系可用有向无环图(DAG)建模。

6.4 最小生成树 MST（★ 计算题）

- 生成树： n 个顶点的连通图，取其 $n-1$ 条边构成的不含环的连通子图。生成树有 $n-1$ 条边。
- Prim 算法（初始选一个顶点逐步"并入最小边"）：适合稠密图， $O(n^2)$ 。
- Kruskal 算法（按权从小到大地选不会成环的边，连同并查集判环）：适合稀疏图， $O(eg \cdot \log e)$ 。
- 两种算法得到的权值和都最小（Prim 从点出发、Kruskal 从边出发），但具体树形可能不同。
- 考点：会手算给定图/Prim/Kruskal 的选边顺序。

6.5 最短路径（★ 必考计算）

- Dijkstra（单源、非负权）：从源点出发，每次选"当前未定中距离最小"的顶点加入，并松弛其邻接边。
- 每次选最小 $\rightarrow$ 可用优先队列（堆）优化。
- 时间：朴素 $O(n^2)$ ；堆优化 $O((n+e)\log n)$ 。
- 注意：Dijkstra 只适用于非负权。
- Floyd（多源、可带负权，不含负环）：动态规划，三重循环， $O(n^3)$ ； $dp[k][i][j]$ 表示经过 $\{0..k\}$ 中点为中转的 $i \rightarrow j$ 最短路。
- 试卷常让你手推 Dijkstra 表（一次一格地填"到各顶点的当前最短路 + 前驱"）。

6.6 拓扑排序与关键路径 (● 结合项目"任务依赖")

- **拓扑排序 (AOV 网)**：有向无环图，输出一种"每顶点都在其所有前驱之后"的次序。
- 方法：找**入度0**的顶点**输出并删除其所有出边**（入度减1），反复。
- 若无法完全输出（还剩顶点）说明**有环**。
- 用**队列/栈**维护当前入度为0的顶点。时间 $O(n+e)$ 。
- **关键路径 (AOE 网)**：带权有向无环图，顶点 $\leftrightarrow$ 事件、边 $\leftrightarrow$ 活动。
- 求**最早发生时间 ve**（从源点正向，取 max）、**最迟发生时间 vl**（从汇点反向，取 min）。
- **关键活动** = $vl - ve = 0$ 的活动；关键路径上的活动之和最长，决定工程总工期。
- **项目对应**：录音任务流水线（采集 $\rightarrow$ 编码 $\rightarrow$ 写盘 $\rightarrow$ 可回放）的依赖可用 AOE；哪步最慢（关键），优化它就缩短总耗时——正是"录音编码慢则录长会溢出"的关键路径思想。

6.7 易错点

- Dijkstra 不能处理负权边；Floyd 不能处理负权回路。
- 拓扑排序结果**不唯一**（有多个入度0可选时选哪个不同，结果不同）。
- 生成树一定有 **$n-1$** 条边，这是判别的核心。

D6 练习题

Q1：6 个顶点的无向图，至少几条边可能不连通？**解析**：最节约但不连通：5 个顶点成完全图 $C(5,2)=10$ 条边 + 1 个孤立顶点 0 条边总和 10 条仍不连通。再加 1 条把孤立连起来 $\rightarrow$ 11 条必连通。所以**最多 10 条边仍不连通**；不连通的边数最大值 = $C(n-1,2)=10$ 。**要点**：背公式 $C(n-1,2)$ 是一个 n 顶图仍可**不连通**的最大边数。**Q2**（最小生成树手推）：给图求 Prim 选边顺序（略图，方法如下）。按"从某顶点开始，每次并入与已选集合相连的**最小权边**"填空，Kruskal 则按全图边权从小到大大选且用并查集判环。**要点**：真题给具体图蹲你手推，配合表格每轮更新"当前 min 距离"即可。手敲 2-3 遍最稳妥。

Q3（Dijkstra 性质）：为什么 Dijkstra 只能用于非负权？**解析**：Dijkstra 采用"贪心"：每次认为"当前到某顶点距离已永久确定（不再更新）"。若存在**负权边**，某顶点的距离可能在之后通过负权边**再变小**，破坏了"已是最终值"的假设，结果错误。 $\rightarrow$ 负权必须用 Bellman-Ford / Floyd。

D7 查找 (★ 哈希是重灾区)

7.1 顺序查找

- 逐个比较，从头到尾。成功平均查找长度 $ASL = (n+1)/2$ ，失败 = $n+1$ 。
- 时间 $O(n)$ 。适合顺序表（可加哨兵减少分支判断）。

7.2 二分查找 (★ 必须会手写与算 ASL)

- 前提：**顺序存储且有序**。
- 每次取 $mid = \lfloor (low+high)/2 \rfloor$ ； $key == A[mid]$ 命中； $key < A[mid]$ 高 = $mid-1$ ；否则低 = $mid+1$ 。
- **ASL 成功** $\approx \log_2(n+1)-1$ （即折半查找的判定树是一棵平衡树）。判定树的**树高** = $\lfloor \log_2 n \rfloor + 1$ ，这就是最多比较次数。
- 复杂度 $O(\log n)$ 。**不适用于链表**（无法 $O(1)$ 随机访问中间）。

- 考点：画折半查找判定树，数某次查找的比较次数、ASL。

7.3 二叉排序树 / 平衡树 / B 树 / 散列

- BST：平均 $O(\log n)$ ，最坏 $O(n)$ （退化为链）；AVL 保证 $O(\log n)$ 。
- B 树 / B+ 树（大索引常考）：
- m 阶 B 树：每个结点最多 m 个孩子、最多 m-1 个关键字；所有叶子（失败的查找结点）在同一层，即所有路径等长。
- B+ 树：数据只存在叶子，且叶子之间有链指针相连（适合范围查找）；非叶结点只存索引。
- 考点：B 树的高度、每个结点关键字数与孩子的对应关系（叶子层所有失败结点同层是 B 树的本质）。
- 本项目：SPIFFS 的块位示图、NVS 键值查找本质可看作“索引/散列”，理解理论即可。

7.4 ★散列（哈希）——分值最高，务必多练

散列函数 $H(\text{key})$ ：常用 - 除留余数法： $H(\text{key}) = \text{key} \% p$ ，p 常取质数（且接近表长）以减少冲突。- 数字分析法、平方取中法、折叠法等（了解）。

冲突处理：- 开放定址法（冲突后在表内找空位）：- 线性探测： $H_i = (H(\text{key}) + i) \% m$ ，依次探测 +1, +2, ...。缺点：堆积（连成一片导致二次冲突）。- 平方探测： $H_i = (H(\text{key}) + i^2) \% m$ （二次探测），减少堆积，但不一定探测完所有格子（表长须取特殊形态）。- 再散列/伪随机等。（开放定址法删除时不能真删，只能“标记删除”，否则会断掉探测链——这是常考的小细节。）- 链地址法（拉链法）：同义词（散列到同一格）挂在同一个链表里。不产生堆积，删除容易。本项目 NVS 的键-值可类比为链地址散列表。

装填因子 $\alpha = \text{表中元素数} / \text{表长}$ 。- 冲突越多越难找；ASL 随 α 增大而增大。线性探测 ASL 成功 $\approx \frac{1}{2}(1+1/(1-\alpha))$ ，失败 $\approx \frac{1}{2}(1+1/(1-\alpha)^2)$ （选填偶尔直接用公式，了解原理即可）。

ASL 计算（必考，务必手算到底）：- 查找成功 ASL：每个关键字“定位次数”求和 / 关键字个数。（定位次数：第一次就命中=1，探测第几次命中就计几次——冲突越多定位次数越大）- 查找失败 ASL：对每个散列地址 $(0..m-1)$ ，从该地址出发到第一个空位(含该空位本身)+1 需要探测的次数总和 / m。（失败要看“空位在哪停”。注意：失败 ASL 平均除以的是表长 m，而不是关键字个数 n——408 常挖这个坑。）- 做题模板：先画一张 $0..m-1$ 的格子表，把关键字经各探测放到对应格子；再把“每个关键字各自命中时的探测次数”列出来求和得成功 ASL；再对每个散列地址算“到空位探测几步”求和除以 m 得失败 ASL。

两个最容易错的点：1. 成功平均除以 n（关键字个数），失败平均除以 m（表长）。2. 构造成功 ASL 时，“探测次数”从 1 开始记（第一个空位即命中算 1 次）；构造失败 ASL 时，走到第一个空位的那次也要算一次探测，且这个空位之后的格子不再探测。

书本经典题（自己完整推一遍，把上表填满）：

表长 13， $H(\text{key}) = \text{key} \% 13$ ，线性探测，依次插 {12, 25, 38, 51, 25, 77, 90, 5} ...（练习时改数据反复做）。最后验证：- 成功 ASL = 所有关键字命中探测次数之和 ÷ 关键字个数；- 失败 ASL = 每个散列地址走到首个空位的探测数之和 ÷ 13。一定要落笔画格子 + 数次数，这是哈希题拿满分的唯一办法。 **

排序项目中"哈希就是按名字找录音"：录音文件名 R0000001.FRC 若做成目录，查找就是"精确匹配键→值"；把文件名 % 表长即可 O(1) 近似定位——正是散列思想。理解它对理解 NVS 命名空间键值也有帮助。

D7 练习题

Q1（哈希 ASL 成功+失败）：表长 13，散列函数 $H(key)=key\%13$ ，依次插入 {19,14,23,1,68,20,84,27,55,11,10,79}（冲突用线性探测）。求查找成功与失败的 ASL。解析（先铺表）：19%13=6；14%13=1；23%13=10；1%13=1 → 冲突，探测 2，放2；68%13=3；20%13=7；84%13=6 → 探测7(20)占、8放8；27%13=1 → 探测2(1)占、3(68)占、4放4；55%13=3 → 探测4占、5放5；11%13=11；10%13=10 → 探测11占、12放12；79%13=1 → 探测2,3,4,5,6,7,8? 依次 2,3,4,5,6(19),7(20),8(84),9放9。表填充结果：下标0空,1:14,2:1,3:68,4:27,5:55,6:19,7:20,8:84,9:79,10:23,11:11,12:10。- **成功 ASL**：每个元素的探测次数：14(1)、1(2)、23(1)、68(1)、20(1)、84(2)、27(4)、55(2)、11(1)、10(3)、79(9)。求和 $=1+2+1+1+1+2+4+2+1+3+9=27 \rightarrow ASL_{成功}=27/12=2.25$ 。- **失败 ASL**：对每个散列地址0~12，从该地址探测到第一个空位。下标0空 → 0号失败查1次；1号(14)→1,2(1),3(68),4(27),5(55),6(19),7(20),8(84),9(79),10(23),11(11),12(10),再13%13=0空 → 探测 14 次。依次算……（本题较绕，锤炼"失败看空位"的算法即可）。**要点**：成功看"每个元素定位几次"；失败看"从某地址到空位探测几次"。这两种 ASL 是高频，务必会分。

Q2（二分）、：在有序数组 1..100 中查找 60 至少比较几次能找到？解析：mid=(1..100)→50<60 → low=51; mid=(51..100)=75>60 → high=74; mid=(51..74)=62>60 → 51..61; mid=(51..61)=56<60 → 57..61; mid=59<60 → 60; 可见约 6 次左右。公式：判定树高度 $\lceil \log_2 100 \rceil + 1 = 7 \rightarrow$ 最多7次。背" $\lceil \log_2 n \rceil + 1$ "为最大比较次数。

Q3（散列缺点）： $\alpha=1$ 表示什么？会怎样？解析： $\alpha=1$ 表已满（元素数=表长）。线性探测充满时查找几乎遍历整表；链地址法离 0.7 以上性能也变差。**散列表应留空位、 α 别接近 1**，这是为什么常设"负载因子警戒线"且扩容。

D8 排序（★ 复杂度与稳定性必背 + 手推堆排/快排）

8.1 八大排序总表（必须背熟）

排序	平均	最好	最坏	空间	稳定
直接插入	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	✓稳定
希尔(Shell)	$O(n^{1.3\sim 2})$	—	$O(n^2)$	$O(1)$	✗
冒泡	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	✓稳定
快速	$O(n\log n)$	$O(n\log n)$	$O(n^2)$	$O(\log n)\sim n$	✗
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗
堆排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(1)$	✗
归并	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(n)$	✓稳定
基数	$O(d(n+r))$	同	同	$O(r)$	✓稳定

稳定性记忆口诀：稳定的：冒泡、直接插入、归并、基数（"冒插归基"稳）；不稳定的：快排、简单选择、堆、希尔（"快选堆希尔"不稳）。

底层原因（理解记忆，别死背）：- 直接插入、冒泡、归并：比较时“= 不交换”即可保持相对次序 → 稳定。- 快排、希尔、堆、简单选择：会**跨越式交换/移动**，容易把相等的两个调换顺序 → 不稳定。- 简单选择：选择最小的跟前面交换，可能跳过与它等值的元素 → 不稳定。

每个格子怎么记（这一段背下来就拿满“复杂度”小题）：- 平均 $O(n \log n)$ 且比较的分治三兄弟：快排、归并、堆。三个中只有归并稳定 + 要 $O(n)$ 空间；快排最坏退 $O(n^2)$ + 空间 $O(\log n) \sim n$ ；堆永远最坏 $O(n \log n)$ + $O(1)$ 空间（最稳的时间复杂度，但常数大、不稳定）。- $O(n^2)$ 的四个：直接插入、冒泡、简单选择、希尔（希尔平均略好于 $O(n^2)$ ）。- “好排” $O(n)$ 的只有两个：直接插入、冒泡（已基本有序时）。为什么：插入/冒泡一趟发现没交换即可停，做原地 $O(n)$ ，适合“近乎有序”数据。简单选择对有序也无能为力（永远固定比较次数 $O(n^2)$ ，但交换少）。- 空间 $O(1)$ 的原地排序：除归并（ $O(n)$ 辅组）、基数（ $O(r)$ ）、快排（递归栈 $O(\log n) \sim n$ ）外全 $O(1)$ 。

拿来即用的“最××”问答（选择题常考精确说法）：- 最坏也是最好（时间恒定 $O(n \log n)$ ）：堆排序、归并（快排最坏会塌到 $O(n^2)$ ，故不算）。- 比较次数不受初始顺序影响：简单选择（永远 $n(n-1)/2$ ）、堆、归并大致也是。- 交换次数最少：简单选择（最多 $n-1$ ）；交换(移动)次数最多：直接插入/冒泡/快排某情况。- 最适合“总体有序，仅少量乱序”：直接插入（ $O(n)$ ）。适合顺序存储也适合链表：归并（链表归并空间可 $O(1)$ ）；快排/堆排依赖随机访问，不太适链表。- n 很大、随机，要求最优市场：堆排或归并（保证 $O(n \log n)$ ）；快排常数小，工程常选（快速加随机/中值基准规避最坏）。

一趟也常考（明白“一趟”=一次划分或一趟冒泡，别混淆）：- 快排“一趟”=一次划分（选 pivot 放好位置，左右分完）。- 冒泡/简单选择/堆“一趟”=一轮比较交换。- 归并“一趟”=把相邻子表两两合一轮。

空间复杂度：- 快排递归栈：平均/最好 $O(\log n)$ ，最坏退化成 $O(n)$ （每次都 1 vs $n-1$ ）。- 归并：需要额外 $O(n)$ 辅助数组（也是它唯一缺点）。- 其余多数 $O(1)$ 。

8.2 各算法灵魂（理解才有底气）

- **直接插入**：把每个新元素插入前面已有序的子序列的正确位置。最好（已有序） $O(n)$ ，平均/最坏 $O(n^2)$ 。
- **冒泡**：相邻比较交换，一趟把最大（或最小）冒到最后。最好（已有序） $O(n)$ （一趟发现问题即可停）。
- **希尔**：直接插入的改进，先按增量分组做插入，增量递减到 1。不稳定。
- **简单选择**：每趟选最小放到最前。比较次数固定 $O(n^2)$ ，但**交换次数少**（最多 $n-1$ 次交换）——适合“交换代价高”场景（如大对象）。
- **快速排序**：选 pivot（基准），一趟把序列分成“小于基准”和“大于基准”两边（partition），递归。平均 $O(n \log n)$ 最快，但最坏（每次基准是最大/最小） $O(n^2)$ ；空间 $O(\log n)$ 。不稳定。
- **划分类(Position)的核心手写**：双指针 low/high，基准先用 `arr[low]` 占位，交替从右找小、从左找大填洞。
- **堆排序**：建大根堆 → 把堆顶（最大）与末尾交换，堆规模减 1，重新向下调整。原地、 $O(n \log n)$ 、 $O(1)$ 空间、不稳定。恰好结合 D5 的堆知识。
- **归并**：合并两个有序表（二路归并），分治递归；需 $O(n)$ 辅助数组；稳定。
- **基数排序**：按位（个/十/百）用“计数/桶”进行**分配-收集**，适合位数少的整数/字符串；时间 $O(d(n+r))$ ， d 为位数， r 为基数（如 10 进制 $r=10$ ）；稳定。

8.3 排序是否需要"比较" (概念) :

- 基于比较的排序 (除基数外) **下界是 $\Omega(n \log n)$** ——快排/归并/堆的最优也就 $O(n \log n)$, 不可能低于它 (这是"比较排序下界", 408 选择/简答考点)。
- 基数排序不基于关键字比较 (用分配-收集), 可突破 $O(n \log n)$ (但依赖 r 和 d)。

8.4 项目落点

- 录音文件名单: R0000001.FRC...R0000123.FRC, 因为**7 位补零**, 所以**字典序 = 数值序**, 可直接用插入/归并对 dirent 排序。
- 这正是工程技巧: **命名规范直接变成排序键**。如果不补零, "10" 的字典序在 "9" 前面 → 排序就错了——这就是"字典序 vs 数值序"的坑, 也是"如何设计排序键"的考点。

D8 练习题

Q1 (稳定性): 序列含重复值 3,2a,3,1 (2a 表示第一个2), 直接插入排序后 2a 和第二个 2 相对位置如何?

解析: 直接插入稳定, 相等的元素 3 在插入时放到已出现的同值后面, 因此 **2a 保持在第二个 2 前面**, 相对次序不变。→ 稳定。

Q2 (快排复杂度): 对逆序数组 $n, n-1, \dots, 1$ 用"选第一个作基准"的快排, 共约多少次比较? 空间? **解析**: 快排对已逆序 (且基准取首) 的序列, 每次划分极不平衡 (1 个 vs $n-1$ 个), 递归深度 n , 比较总数 $\approx n(n-1)/2 = O(n^2)$, 递归栈空间 $O(n)$ 。→ 务必背"快排最坏在基准取到极值时发生, $O(n^2)$ "。

Q3 (堆排手推): 用大根堆对 {5,2,9,8,3} 排序, 第 1 趟交换并调整后数组变成? **解析**: 先建大根堆 (自己练), 堆顶 9 与末尾交换 → {3,8,5,2|9} (已放好9), 对 {3,8,5,2} 重新向下调整 → 8 是最大的上浮 → {8,3,5,2,9} 第一趟完成。"堆顶换到最后 + 缩小堆 + 向下调整"三步是堆排的每一趟。

要点: 凡考堆排, 必须能把建堆、交换、向下调整写全。这是最容易手滑的排序。

【数据结构部分小结】 (背这一页)

- **复杂度**: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$ 。循环 $\times 2$ 是 $\log$, 双层循环看结构。
 - **线性表**: 顺序表随机 $O(1)$ 插删 $O(n)$; 链表相反。
 - **循环队列**: 判空 $F == R$ 、判满 $(R+1) \% M == F$ 、个数 $(R-F+M) \% M$, 容量 $M-1$ 。
 - **树的必背**: $n_0 = n_2 + 1$; 深度 k 满 $2^k - 1$; 完全树编号 $2i/2i+1/\lfloor i/2 \rfloor$; 哈夫曼 $2n-1$ 结点; AVL 旋转 LL/RR/LR/RL。
 - **图**: DFS 用栈、BFS 用队列; Prim 点、Kruskal 边; Dijkstra 非负权; 完全图边数 $C(n, 2)$ 。
 - **哈希**: α = 元素/表长; 成功 ASL 看定位次数、失败 ASL 看空位; 冲突 线性/平方/链地址。
 - **排序**: 稳定 = 冒插归基; 堆排/快排细节手推; 比较排序下界 $\Omega(n \log n)$ 。
-

第二部分 计算机组成原理（★考得最细、分值最重，务必花最多时间）

计组是 408 中"知识最碎、最能算"的一科，也是最依赖"把每个细节讲清楚"的一科。袁春风教材强调一条主线：**程序怎么从 C 语言变成 CPU 能执行的机器码 / 指令，再被 CPU"取指→译码→执行"**。只要这条主线你每一步都知道"发生了什么、数据在哪、为什么"，这一科就赢了大半。下面按顺序深讲，并把每个抽象概念落到这台 ESP32-C3(RISC-V 内核) 录音设备上。

C1 计算机系统概述：层次结构、性能指标（袁 § 1）

1.1 计算机系统的层次（由顶到底，抽象逐层下降）

用户的意图（要"录一段 10 秒的 16kHz 音频"）
|
应用软件层：录音 App / UI (study_rec_ui.c)
|
操作系统层：FreeRTOS（任务调度、文件系统 SPIFFS、驱动）
|
汇编/机器指令层：RISC-V RV32I 指令集（ESP32-C3 内核）
|
微体系结构层：数据通路 + 控制器（取指→译码→执行）
|
逻辑/物理层：晶体管、存储器、I/O 电路

要点：上层通过"接口/命令"调用下层，下层向下提供底层能力，"机器字长、地址空间"等由最底层决定。计组主要研究从"机器指令"往下的部分，而上限到"编译/汇编把高级语言变成机器指令"。

1.2 计算机的性能指标（必背公式）

- **机器字长：**一次运算能处理的二进制数据位数（本项目 CPU=32 位，即 RISC-V RV32）。
- **主频：**CPU 时钟频率 f (Hz)。主频越高不一定越快（吞吐还看每指令周期数）。
- **CPI（每条指令时钟周期数，Cycles Per Instruction）：**执行一条指令平均需要的时钟周期数。
- **CPU 时间 = 指令条数 $\times$ CPI $\times$ 时钟周期**（时钟周期 = $1/\text{主频}$ ）。CPU 执行时间 = 指令数 $\times$ CPI $\times 1/f$ （这是"性能计算"的总公式，任何性能题都从它展开。）
- **IPS / 吞吐量：**每秒执行指令数 = 主频/CPI。
- 运算速度常标 MIPS（每秒百万条指令）= 主频/(CPI $\times 10^6$)。
- **MFLOPS / FLOPS：**每秒百万浮点操作——衡量浮点性能，本项目无 FPU，浮点靠软模拟，很慢，所以用 Q 格式定点（见 C2）。

本项目落点：ESP32-C3 主频最高 160MHz，32 位 RISC-V。Speex 编码是**整数/定点**运算，改为浮点版则每条浮点指令要软件模拟 → CPI 暴增、性能下降，这正是"16k 用定点 WB 才能扛住"的原因之一。

1.3 从 C 语言到 bin 的完整旅程（★本项目最应深讲的一段，考纲多次以它为背景）

我们要回答："int16_t pcm[320]; 这样的 C 代码，最终怎么变成烧进 Flash 的一串 0/1？CPU 又怎么把它取出来执行？"

完整 ⑦ 步：

- ① 预处理 (.i) #include #define #if 等文本层面的展开与过滤
- ② 编译 (.s) 把 C 变成汇编，做语法分析、中间表示、寄存器分配、指令选择
- ③ 汇编 (.o) 汇编器把汇编变成机器码，同时生成符号表 + 重定位表
- ④ 链接 (.elf) 多个 .o 合并段、符号解析、重定位 → 可执行 ELF
- ⑤ 格式转换 去掉调试信息/ELF 头 → app.bin (裸机器码镜像)
- ⑥ 烧录 用 ESP 烧录工具按分区地址写进 SPI Flash (factory@0x10000)
- ⑦ 装载执行 bootloader 跳转到入口 → CPU 取指 → 译码 → 执行

每步细讲"产物是什么、干了什么"：

① **预处理**：把 #include <speex/speex.h> 的头文件**逐字粘贴**进源文件；#define FIXED_POINT 1 在全文做**宏替换**；#if / #ifdef 按条件**保留/删除**代码段。产物是 .i 文件（还是纯文本，仍是人可读的高级语言）。**不做任何语法/类型检查**。

② **编译**：前端做**词法→语法→语义分析**形成抽象语法树（AST），中端会生成中间表示（IR）并做优化（公共子表达式删除、循环展开、寄存器分配、指令选择、**调度减少冒险**）。后端把 IR 翻译成**汇编指令**。产物 .s。这里是"高级语言 → 机器语义"的关键分水岭：- arr[i] → 用**基址寄存器 + 偏移寻址**（见 C4 寻址方式）；- for 循环 → 用**条件分支 beq + 跳转 jal**；- 函数调用 → 用 jal / jr + **调用栈（栈指针 sp 下降压入返回地址、实参、局部变量）**。

③ **汇编**：汇编器逐条把汇编指令翻译成**机器码**（一条指令一张 32 位位的"编码表"），输出 .o（目标文件，即 ELF 的一类）。同时生成两块关键元数据：- **符号表（symbol table）**：记录本目标文件里定义/引用的每个全局符号（函数名、全局变量名）及其在当前 .o 内的"偏移/未定地址"。- **重定位表（relocation table）**：记录"哪些位置引用了外部符号，但地址还没定，等链接时回填"。**为什么必需**：比如 study_recorder.c 里调用了 printf，但要到把所有 .o 拼起来之后才知道 printf 在最终内存的哪个地址。所以编译阶段先在该函数的调用位置填 0，并登记一条重定位记录"此处 + 引用 printf"。

④ **链接**：链接器把**多个 .o 的各个段按段合并**（所有 .text 合并成一个大 .text，.data 合并成 .data），做**符号解析**（每个引用的符号在最终 ELF 中的地址确定下来），再**重定位**（把每处引用回填成最终绝对地址）。产物 .elf（Executable and Linkable Format，含程序头、节头表、符号表）。**地址在这一步真正确定**。- 静态链接 vs 动态链接：静态 = 链接时就把用到的库代码拷进 ELF（嵌入式全都这样，因为内存小、无热更新需求）；动态 = 运行时才加载 .so（PC 上常见）。

⑤ **格式转换**：嵌入式没有操作系统"从磁盘加载 ELF"的过程，而是把 ELF 里**程序段的二进制内容**抽出来，去掉 ELF 文件头/调试信息，做成一段"**从固定地址开始、可直接执行的裸镜像**" app.bin。用 esptool / objcopy --binary 完成。

⑥ **烧录**：把 app.bin 按**分区表**写到 SPI Flash 的指定偏移（本项目 factory 分区在 0x10000，即从 64KB 处开始，留出 bootloader 和分区表）。烧录就是"把 bin 的每个字节写到对应 Flash 地址"。

⑦ **装载与执行**：上电后 **bootloader**（ROM 里的一段引导代码）先运行，初始化最小系统，跳转到 0x10000 处的应用入口。CPU 从此开始循环取指（IF）→译码（ID）→执行（EX）→访存（MEM）→写回（WB），一条条执行。**装载=把 ELF/镜像映射到 CPU 可访问的地址空间并跳进去。**

1.4 段的细节与"代码在 Flash、变量在 RAM"（必懂）

- **.text**：可执行机器码，出厂只读（本项目放 Flash 只读窗，可经 MMU 只读映射到 0x42000000 区域执行）。
- **.rodata**：只读常量（如 Speex 量化码本 `gc_quant_bound[16]` 等表格），也放 Flash，CPU 直接读，不占宝贵的 SRAM。
- **.data**：**已初始化**的全局/静态变量，初值存在 Flash 里的拷贝，启动时由 `crt0` 拷到 RAM 的对应地址。
- **.bss**：**未初始化/零初始化的全局变量**（C 里未赋值或 `=0`），标准是"程序运行时才在 RAM 里清零"，**不占 bin 文件空间**（因为全是 0，没必要存初值）。→大量 `static int ... = 0` 数组放在 `.bss`，`bss` 大了 `bin` 不一定大，但会占运行时 RAM。

项目落点：之前聊天里的"空闲堆约 33KB、最大连续块约 16KB"、任务栈等，全是**运行时 RAM（SRAM 里的堆/栈）**；而**录音 16k 崩溃**是因为 Speex WB 编码器初始化要分配较大堆、又要 16KB 任务栈——这两块在 SRAM 里**不能连续命中**或直接不足，导致分配失败/堆损坏 → 触发"异常/PANIC"。这正是下面 C3 存储、C5 中断、OS 内存三章的活案例。

1.5 链接优化：函数级裁剪（结合本项目 16k 内存紧张）

用编译选项 `-ffunction-sections -fdata-sections` 让**每个函数/常量单独放进一个独立的节**，链接时 `--gc-sections` 把**从未被引用的节直接丢弃**。本设备就是这么做的：把 Speex 里没有被用到的码本/模式裁掉，压缩 Flash 与 RAM 占用——这就是"内存不够时靠裁剪未用代码腾空间"的工程手段，和 OS 的"内存换出"是两回事（这是编译期裁剪）。

C1 练习题

Q1（性能公式）：某 CPU 指令总数为 5×10^8 ，平均 CPI=2，主频 500MHz，求 CPU 执行时间。**解析**：CPU 时间 = 指令数 $\times$ CPI / 主频 = $5 \times 10^8 \times 2 / (500 \times 10^6) = 10^9 / 5 \times 10^8 = 2 \text{ s}$ 。（套"指令数 $\times$ CPI $\times$ 时钟周期"公式即可。）

Q2：同一程序，CPU1 主频 3000MHz、CPI=3；CPU2 2000MHz、CPI=2，谁快？**解析**：CPU 时间 = 指令数 $\times$ CPI/f。设指令数 N：CPU1 时间 = $N \cdot 3/3G = N/G$ ；CPU2 = $N \cdot 2/2G = N/G$ → **一样快**。（提示：主频高不一定快，要综合 CPI。这是 408 常设陷阱。）

Q3（概念）为什么嵌入式用静态链接？**解析**：嵌入式无独立操作系统动态加载/共享库机制、内存小、免去运行时符号解析开销与安全风险；程序加载=把固定地址的镜像直接执行。→ 静态链接。

C2 数据的表示与运算（对应表 § 2/ § 3，选择里算分之王）

这章你必须把**每个数制转换、每个机器数定义、每次溢出判断、每道 IEEE754 都算到手酸**。

2.1 数制与真值/机器数

- **真值**：带正负号的真实数值（人类视角），如 -101（二进制真值）、+7。
- **机器数**：把符号也数字化，一个数的符号用 1 位（0 正 1 负）连在数值前。真值在机器里用原码/反码/补码/移码四种"机器数"来表示。
- **进制转换（必须滚瓜烂熟）**：二进制 $\leftrightarrow$ 十六进制（每 4 位二进制对 1 位十六进制）；十进制 $\rightarrow$ 二进制"除 2 取余"；二进制 $\rightarrow$ 十进制"按权展开"。

2.2 原码 / 反码 / 补码 / 移码（n 位有效数值，1 位符号，总 n+1 位）

设共 N 位（含符号，假设 N=8）：

码制	正数表示	负数表示	数值范围	0 的表示
原码	0 符号+真值	1 符号+真值	$-(2^7-1) \sim +(2^7-1)$ ，即 -127~127	+0=00000000，-0=10000000（两种）
反码	同原码	符号不变，数值各位取反	-127~127	+0=00000000，-0=11111111（两种）
补码	同原码	反码+1 （符号不变）	-128~127 （多负数一方）	00000000（ 唯一 ）
移码	补码符号位取反	补码符号位取反	同补码（用于阶码）	10000000

必会的三件事：1. $[-x]_{\text{补}} = [x]_{\text{补}}$ 按位取反 + 1（包括符号位），即"求补"。2. **补码范围不对称**：负数比正数多一个，最小负数是 $-2^7 = -128$ （10000000），最大的正数 127。**为什么**：0 只占一个编码，省出来的那个编码留给了 -128。3. **符号扩展**：从 8 位扩展到 16 位，正数高位补 0、负数（补码）高位补 1（保持数值不变）。

为什么计算机用补码（必考/常考）：- ① **减法变加法**： $A - B = A + (-B)_{\text{补}}$ ，CPU 只需加法器（节省一堆减法电路）。- ② **0 唯一**（原码/反码有两个 0，补码只有 0000...0），消除歧义。- ③ 补码加法的**进位规则与"无符号数"一致**，省控制逻辑。- ④ 补码表示范围比原码/反码多表示一个负数（ -2^n ），充分利用编码空间。

2.3 ★补码运算与溢出判断（选择/大题高频）

① 补码加法的本质（先把"为什么"想通）

补码减法变加法： $A - B = A + [-B]_{\text{补}}$ 。所以 CPU 只有加法器也能做减法，核心操作就是 $X + Y$ （其中 Y 可能是减数的补码）。做加法时**最高位（符号位）与数值位同样参与进位**，符号位的进位（进位到机器字之外的那一位）被**丢弃**，结果仍按补码解释。

② 溢出（Overflow）定义（必须先分清概念）

真值溢出 $\neq$ 进位丢弃。补码加法中，符号位产生的"进位 1"丢在机器字外是**正常现象**（比如 $-1 + 1 = 0$ ， $[-1]_{\text{补}} + [1]_{\text{补}} = 1111..1 + 0000..1 = 1\ 0000..0$ ，那个进位 1 被丢弃，结果 0 正确）。**溢出指的是：运算结果超出该码制能表示的数值范围，真值已经非正常**（例如两个正数加出负数、两个负数加出正数）。这两者不能混，是 408 最爱埋的坑。

一句话：**进位丢弃是常态，溢出是结果坏了**。只有当"结果的符号"已经不再是正确真值应有的符号时，才是溢出。

③ 什么时候才可能溢出（先判这步，省一半时间）

- **同号相加** 可能溢出（正+正可能超过最大值，负+负可能低于最小值）。
- **异号相加** 一定不溢出（结果夹在两者之间）。
- 减法看成加法： $A - B$ 溢出等价于 $A + [-B]$ 补 溢出，即**同号相减不溢出**（正-正当 $A > B$...实际判"符号是否异常改变"最稳）。

④ 三种等价判据（任选，考熟一种即可）

1. **符号位判据（最直观）**：两操作数同号，且结果的符号与操作数相反 $\rightarrow$ 溢出。
2. **进位判据**：符号位进位 C_s 与最高数值位进位 C_n 是否相同。不同 $\rightarrow$ 溢出；相同 $\rightarrow$ 不溢出。（用异或： $\text{溢出} = C_s \oplus C_n$ ，这是硬件加法的实际判据，加法器里就一个异或门。多练几道建立直觉。）
3. **双符号位（变形补码）判据**：将操作数扩展成两位符号位 S_2S_1 参与运算，结果： $-00 \rightarrow$ 正，无溢出； $11 \rightarrow$ 负，无溢出； $-01 \rightarrow$ **正溢出**（上溢）； $10 \rightarrow$ **负溢出**（下溢）。即"两位符号位不同 $\Rightarrow$ 溢出"。写入寄存器时只留一位。

⑤ **三个口诀（背死）** - 同号相加、结果反号 $\Leftrightarrow$ 溢出。 - 符号位进位 $\oplus$ 数值最高位进位 $= 1 \Leftrightarrow$ 溢出。 - 双符号位 $01/10 \Leftrightarrow$ 溢出， $00/11$ 无溢出。

本项目落点：PCM 采样是 `int16_t` 补码，范围 $[-32768, 32767]$ 。静音=0（`0x0000`），最大正幅=32767（`0x7FFF`），-1=`0xFFFF`。加减法（如直流偏移消除 $x - \text{mean}$ ）都是补码运算——溢出会夹出爆音/削波，所以音频处理里**必须先做饱和（clamp）再写入缓冲**（`if (v > 32767) v = 32767; if (v < -32768) v = -32768;`），这正是"防止补码溢出"的工程实现。Speex 定点滤波器（乘累加 MAC）尤其容易溢出，常用右移/限幅来防。

⑥ 课堂式例题（补码+溢出，务必每道手算一遍）

- 例 A：8 位补码求 $-13 - 15$ 。 $[-13]_{\text{补}} = -13 = 11110011$ ； $[-15]_{\text{补}} = 11110001$ 。做 $A + [-B]$... 这里实际是 $(-13) + (-15)$ ： $11110011 + 11110001 = 1\ 11100100$ ，进位丢 1，结果 $11100100 = -28$ 。两负数相加得负数，符号位进位 1、数值最高位进位 1（ $C_s=1, C_n=1$ 相同） $\rightarrow$ 不溢出。正确。
- 例 B：8 位补码 $100 + 100$ ，判断是否溢出。 $[100]_{\text{补}} = 01100100$ ； $01100100 + 01100100 = 11001000$ 。两个正数相加，结果符号位是 1（负数） $\rightarrow$ 违反"正+正得正" $\rightarrow$ **溢出**。用进位判据： $C_s=0, C_n=1$ 不同 $\Rightarrow$ 溢出。真值应为 200，超过 8 位补码最大值 127。 $\sqrt{\quad}$
- 例 C：8 位补码 $120 + 8$ 。 $01111000 + 00001000 = 10000000 =$ 补码 **-128**。正+正得负 $\Rightarrow$ **溢出**。（注意结果 `10000000` 正好是 -128——这里容易误判，符号位由 0 变 1 就是溢出信号。）
- 例 D：8 位补码 $(-1) + (-3)$ 。 $11111111 + 11111101 = 1\ 11111100$ 丢进位 $\rightarrow 11111100 = -4$ 。两负相加得负，不溢出（ $C_s=1, C_n=1$ ）。
- 例 E：判断 `0x40 + 0x40`（都是 +64）是否溢出——见上类，正+正出负 $\Rightarrow$ 溢出。

⑦ 考研常见命题角度（心里有数，见到不慌） - 给两个八位十六进制补码，问相加是否溢出、结果真值多少（注意可能丢弃进位）。 - 用"进位判据"或"双符号位"判什么硬件信号反映溢出（电路题）。 - 把饱和/夹取与溢出结合（如本项目 DSP 场景）。 - 符号扩展后相加是否仍溢出（扩展不影响数值，溢出判定方法不变）。

2.4 定点数与 Q 格式（本项目核心，一定要懂）

- **定点数**：小数点的位置在数中是**固定的**。若无浮点单元（FPU），就把一个小数 x 放大 2^Q 倍存成一个**整数** $X = \text{round}(x \cdot 2^Q)$ ，这里的 Q 叫"小数位数"，称为 **Q 格式**。
- 运算时按整数做，最后右移 Q 位还原成小数。
- 例：Q15 表示 $x=0.5 \rightarrow X=0.5 \times 2^{15} = 16384$ ； $x=-0.25 \rightarrow -0.25 \times 32768 = -8192$ 。
- **Q 运算规则（必记）**：
 - **加/减**：同 Q 直接加减（都在整数域），无 Q 变化。
 - **乘**：Q15 $\times$ Q15 $\rightarrow$ 结果是 Q30，需要再**右移 15** 得到 Q15，并注意**可能溢出**（两个最大数相乘超范围），用饱和防止。
 - **除**：要先左移再除，避免精度损失。
- **为什么 Speex 用 FIXED_POINT**：ESP32-C3 无 FPU，浮点运算是软件模拟、非常慢、还很占栈/栈；改用整数 Q 格式后，LPC/CELP 的预测、量化、滤波全变成整数乘法与移位，快且省。量化码本 `gc_quant_bound[16]` 里的"边界值"就是用定点编码的表格（QD 表 / 量化表）。

2.5 IEEE 754 浮点（★背诵 + 手算）

单精度 float（32 位）布局：

| 1 位符号 S | 8 位阶码 E (偏移 127) | 23 位尾数 $\text{fraction}(f)$ |

规格化数的值：

值 = $(-1)^S \times (1.f)_2 \times 2^{(E-127)}$ ，其中 $(1.f)_2$ 的隐含 1 不存储

关键规则： - **阶码用"移码"**：真阶码 = 存储阶码 $E - 127$ （偏置量是 127）。 - **隐含位 1**：规格化数尾数总是 $1.f$ ，这个"1"不存，白赚 1 位精度。 - **特殊值**（务必背）： - 阶码全 1（ $E=255$ ）、尾数全 0 $\rightarrow \pm\infty$ （用于除以 0）； - 阶码全 1、尾数非 0 $\rightarrow \text{NaN}$ （非法结果： $0 \div 0$ 、 $\infty - \infty$ 、负开根）； - 阶码全 0（ $E=0$ ） $\rightarrow$ **非规格化数** $(0.f)_2 \times 2^{(-126)}$ ，或 ± 0 （尾数全 0）。 - 最大值/最小规格化：最大规格化尾数 $1.111\dots 1$ ，阶码 $254 - 127 = 127 \rightarrow \approx 3.4 \times 10^{38}$ ；最小规格化正数 $1.0 \times 2^{(-126)}$ 。 - 双精度 double（64 位）：1 符号 + 11 阶码（偏移 1023） + 52 尾数。

本项目落点：若把 Speex 换用浮点版，那么录音的每次滤波/量化都会走到 IEEE754，本来要命中这套公式；因无硬件浮点，改成 Q 格式定点后，反而"绕开了"浮点，用整数完成——两种都建立在"数的二进制表示"之上。理解 IEEE754 会让你看懂为什么定点更省。

2.6 校验码（选做，但常考）

- **奇偶校验**：在原数据后补 1 位，使整个码字中 '1' 的个数为奇数（奇校验）或偶数（偶校验）。只能**检错**（检测奇数位错），不能纠错，且偶数位错检测不到。最简单。

- **海明码 (Hamming)**：能纠正 1 位错（并发现 2 位错）。冗余校验位个数 r 满足 $2^r \geq n+k+1$ (k 是数据位, $n=k+r$ 总位数)；校验位放在 2 的幂位置 (1,2,4,8...)，每个校验位负责一组“其二进制编号中该位为 1”的信息位。纠错靠各校验位算出 **海明位置 (出错位编号)**。工作量大，考上要会手排。
- **CRC (循环冗余校验)**：把数据看作多项式，除以一个生成多项式，把余数作为 CRC 校验位附在末尾。接收端同样除，余数为 0 则无错。能检测多位错、可按生成多项式纠错。是计网里以太网 FCS 用的校验法 (见 N3)。要会多项式除法 (模 2 除)。
- **本项目落点**：SPIFFS 页 CRC、NVS 的校验、WiFi 帧的 FCS 都是“用校验码保证 FLASH/网络数据不出错”的工程落地；文件“损坏/FLASH 位翻转”靠它发现并重读/报错。

C2 练习题

Q1 (求补码)：求 8 位补码表示 -45。解析：45(十进制) = 00101101 (8 位原码)。取反 = 11010010，+1 = 11010011。所以 [-45]补 = 11010011。反码+1 是唯一要背的求补法。 **Q2 (溢出判断)**：用 8 位补码算 10011011 + 01100101，判断是否溢出？解析：10011011 = -101 (负)，01100101 = +101 (正)。异号相加不可能溢出 (结果 -101~+101 之间)。真值 = -101+101 = 0，正确，不溢出。(若用“进位法”：加出来符号位进位 1、次高位进位 0 → 不同 → 误判？其实这种“1+0”进位的组合对异号相加是正常。所以大原则先判“同号才可能溢出”。) 要点：先看符号，异号相加永不溢出；只有同号相加且结果符号改变才是溢出。 **Q3 (IEEE754)**：把 6.75 转成单精度 float 的十六进制。解析：6.75 = 110.11₂ = 规格化 1.1011 × 2²。符号 S=0。真阶码=2 → 存储阶码 = 2+127 = 129 = 10000001。尾数 f = 1011 (隐含头 1 不存)，后补 0 到 23 位：101100...0。拼位：0 | 10000001 | 1011000 0000 0000 0000 0000 = 0x40D80000。易错：符号、阶码偏置 127、隐含 1——三个都要对。 **Q4 (Q 格式)**：Q15 表示，计算 0.5 × (-0.25)。解析：X1=0.5×32768=16384；X2=-0.25×32768=-8192。相乘 = 16384×(-8192) = -134217728 (Q30)。右移 15 → -4096，还原 -4096/32768 = -0.125。(中间是 Q30，记得右移回 Q15。) **Q5 (CRC, 计组/计网都考)**：生成多项式 G(x)=x³+x+1 (即 1011)，数据 1100，求 CRC 校验码。解析：校验位数 = 生成多项式位数-1 = 3。数据左移 3 位补 0 → 1100000。模 2 除法：1100000 ÷ 1011：1100 xor 1011 = 0111；下移一位 01110，首 1 → xor 1011 = 0101；下移 01010，首 0 → 01010；下移 10100，xor 1011 = 0001...重复到末。余数 R=具体算得某 3 位 (此例为 010)。最终发送码字 = 数据 + 余数 = 1100010。套路：数据片 → 左移 r 位 → 模 2 除生成多项式 → 取余数 → 拼在尾部。模 2 除法 = 异或且不借位，多练几次。

C3 存储系统 (对应袁 §6, ★计算重地)

3.1 存储系统的层次与局部性 (背概念)

层次：寄存器 → Cache (SRAM) → 主存 (DRAM) → 外存 (Flash/磁盘)。- 越上越快、越贵、越小；越下越慢、越便宜、越大。- 为什么分这么多层：兼顾“速度接近最快 + 容量接近最大 + 单价比最低”——用每层各取所长。

局部性原理 (必须背, Cache 存在的前提)：- **时间局部性**：刚访问过的数据/指令在很快还会被访问 (循环、栈顶变量)。- **空间局部性**：刚访问过的地址附近的区域很快会被访问 (顺序取指令、顺序读 PCM 缓冲)。-- > 程序顺序执行指令、循环重复执行同段 → Cache 能命中；本项目“循环读同一码本、顺序读取每个 PCM 样本”都是局部性的活例。

3.2 SRAM / DRAM / Flash（记忆 + 特性）

特性	SRAM（静态）	DRAM（动态）	Flash（闪存）
存储原理	6 晶体管锁存	1 晶体管 + 电容电荷	浮栅晶体管存电荷
速度	最快	中	慢（读尚可，写更慢）
易失性	易失（掉电丢失）	易失（还要周期性刷新）	非易失
用途	CPU Cache、小容量高速内存	大容量主存	本项目外存（SPI Flash）
写入特点	可改写	可改写	写前先擦、按块擦、按页写，有擦写寿命

要点： - **DRAM 靠电容存电荷**，会漏电，要定时**刷新（refresh）**，这就是"动态"和"比 SRAM 慢/复杂"的原因。 - **Flash 非易失**适合存固件/录音文件；但**写前必须先擦、擦以块为单位、擦写有限寿命** → 文件系统要做**磨损均衡**（本项目 SPIFFS 的分层/均衡）。 - **本项目存这些**：`.text/常量` 放 Flash（非易失，只读执行）；全局变量/堆/栈放 SRAM（Δ 主存易失，掉电清零）。

3.3 主存与"按字节寻址"、「地址空间」「分区（本项目直接实践）」

- **按字节寻址**：每一个字节一个唯一地址。（用字寻址则每个 4 字节单元一个地址，地址少 2 位。）
- **地址位数 ↔ 可寻址容量**：n 位地址可寻址 2^n 个单元（如 32 位地址空间 4GB）。
- **本项目 SPI Flash 分区就是固定地址分区**（也是一种"主存分区"的映射）。
- **整片容量**：按官方仓库基线，本设备 SPI Flash 为 **8MB**（`sdkconfig` 里 `CONFIG_ESPTOOLPY_FLASHSIZE_8MB=y`）。分区表由 `partitions.csv` 描述，每个分区 = 一块**起始偏移 + 大小**的连续区域，各分区**首尾相接**，最后一个分区 `recovery` 结束于 `0x800000`，正好是 8MB——即"分区拼起来刚好占满整片 Flash"。

分区名	起始地址	大小	类型	用途
nvs	0x9000	24KB	data/nvs	非易失存储：WiFi 凭证、时间、任务完成度、设置项
phy_init	0xf000	4KB	data/phy	射频校准数据
factory	0x10000	3MB	app/factory	应用固件（考研助手 + 菜单 demo），开机默认启动
cardid	0x356000	16KB	data/nvs	BLE Recovery 安装器固定地址（官方，不可改）
voicepack	0x35a000	480KB	data/spiffs	语音包/声音资源（mp3→opus.frc）
recordings	0x3d2000	≈3.17MB	data/spiffs	用户录音（ADPCM@8kHz → 约 12 分钟容量）
recovery	0x700000	1MB	app/test	

分区名	起始地址	大小	类型	用途
				官方的 BLE Recovery 恢复/升级固件

换算：容量 = 结束地址 - 起始地址；换算 16↔10 进制 要到张口就来（如 $0x10000=64KB$ ， $0x003D2000=3D2000_{16}=4006144\approx 3.05MB$ ）。这段就是要你对"地址是用字节量化、地址区间=长度"有直觉。

几个关键点（也贴合 408 的"存储分配"考点）：

- 分区是大块连续分配，类似固定分区/静态分配：一次定好大小、地址固定，不存在页式那种动态换页——所以 **factory** 预留 3MB、**recordings** 预留约 3MB，都是"按上限多给余量"的静态划分。
- **cardid / recovery** 是官方 BLE Recovery 安装器写死的地址，二创不能动——这就是"系统/厂商保留分区"，挪了会导致官方安装器无法定位。
- **nvs** 是键值非易失存储：深睡/断电后要保留的 WiFi、时间、任务进度都在这里，开机读回（对应"外存非易失、掉电不丢"）。
- **voicepack / recordings** 用的是 **SPIFFS 文件系统分区**：里面是一个个文件，靠 SPIFFS 做块位图管理 + 磨损均衡（对应 OS 文件管理那章的位示图 / 磨损均衡知识点）。

3.4 ★Cache 与地址映射（计算大头，务必会算）

主存地址划分（Cache 结构下的视图）：

高—————低
Tag(标记) | Index(组/行索引) | BlockOffset(块内偏移)

设：块大小 = 2^b 字节（ b =块内偏移位数），Cache 共 C 行，每组 N 行（组相联的"路数"），组数 $S = C/N$ 。

三种映射（必背 + 会算位数）：

1. **直接映射**（每组 1 行， $N=1$ ）：- 行号 = 主存块号 mod C （就一行能放）。- Index 位数 = $\log_2 C$ 。- **缺点**：多个需要映到同一行的主存块互相"踢"，命中率低、抖动明显。
2. **全相联**（只有 1 组， $S=1$ ）：- 任意主存块可放任意一行，只需 **Tag** 全比较；**Index=0**。- 好处命中率高；坏处**比较电路昂贵、Tag 很大**。
3. **组相联**（折中， N 路每组）：- 组号 = 主存块号 mod S ，组内 N 行任放。- Index 位数 = $\log_2 S$ 。- 组相联合 N =直接全 S =组数 $\rightarrow N=C$ 即全相联。所以**直接映射 = 1 路组相联、全相联 = 全路（ $N=C$ ）组相联**，三者是同一家族。

位数计算（必背公式）：

主存地址 A 位
块内偏移位数 $b = \log_2(\text{块大小字节数})$
行/组索引位数 $s = \log_2(\text{行数 } C \text{ 或 组数 } S)$
Tag 位数 = $A - b - s$

Cache 总容量（含标志位，常考）：

Cache总存储位数 = 有效位(1) + Tag 位数 + 数据位(块大小×8 位) × 行数 C

（每一行都要额外存"有效位"和"Tag"，这些不计入"数据容量"但是要花的硬件。）

例题(cache 容量/位数)：主存 64MB（地址 26 位），字块大小 4B，Cache 8KB 直接映射。- $b = \log_2 4 = 2$ 位。- 行数 $C = 8KB/4B = 2048$ 行 $\rightarrow s = \log_2 2048 = 11$ 位。- $\text{Tag} = 26 - 2 - 11 = 13$ 位。- 主存块数 $= 64MB/4B = 2^{26}/2^2 = 2^{24}$ ；每 2^{11} 块共享一行。- Cache 总容量（不含 Tag/有效位的数据容量）= 8KB；含 Tag/有效位 $= 2048 \times (1+13+32)$ 位 $= 2048 \times 46$ 位 $= 94,208$ 位 $\approx 11.76KB$ 。

命中率 / AMAT（平均访存时间，必背）：

$$\begin{aligned}\text{AMAT} &= \text{命中率} \times \text{命中时间} + \text{未命中率} \times (\text{命中时间} + \text{未命中代价}) \\ &= \text{命中时间} + \text{未命中率} \times \text{未命中代价}\end{aligned}$$

（后一个写法把"命中时间"提出去更常用： $\text{AMAT} = t_{\text{hit}} + (1-h) \cdot t_{\text{miss_penalty}}$ 。考"命中/失效访问次数"也常出现——把访问串一条条判命中数就得到命中率 h 。）

AMAT 应用题（手算一遍建立直觉）：已知 Cache 命中时间 1 个时钟周期，未命中代价（从主存取一块）需 50 个时钟周期，命中率 $h=95\%$ ，求 AMAT。

$$\text{AMAT} = t_{\text{hit}} + (1-h) \times \text{miss_penalty} = 1 + 0.05 \times 50 = 1 + 2.5 = 3.5 \text{ 周期}$$

对比（必考的"成也 Cache 败也 Cache"）：如果命中率掉到 90%： $\text{AMAT} = 1 + 0.10 \times 50 = 6$ 周期。命中率只降 5%，AMAT 快翻倍 $\rightarrow$ 说明未命中代价巨大，所以设计核心是**提高命中率**（加大 Cache、选好替换策略）。

主存带宽/缺失率计算角度：题目常给"CPU 每 1000 条指令产生 X 次访存、Cache 缺失率 $Y\%$ "，问"因 Cache 引起的平均额外延迟"或"缺失导致 CPI 增加多少"。统一套路：

$$\begin{aligned}\text{CPI}_{\text{new}} &= \text{CPI}_{\text{base}} + \text{缺失次数/指令} \times \text{缺失惩罚} \\ &= \text{CPI}_{\text{base}} + (\text{访存次数/指令} \times \text{缺失率}) \times \text{miss_penalty}\end{aligned}$$

例： $\text{CPI}_{\text{base}}=1.5$ ，每指令 1.5 次访存，缺失率 2%，缺失惩罚 100 周期 $\rightarrow \text{CPI} = 1.5 + (1.5 \times 0.02) \times 100 = 1.5 + 3 = 4.5$ 。这个"引申 CPI"题型 408 反复考，务必背公式。

3.5 替换算法与写策略

- **替换：**Cache 满需要腾一行给新块时，换出哪个？
- **LRU（最近最久未用）：**换出"最久没有用过的"。（命中率高，需记录访问最近度）
- **FIFO：**先入先出（实现简单、可能抖动，即某块刚被换出又立刻要用）。
- 随机替换。
- **OPT/LFU（最优）：**理论下界，实际不可实现，做对照。
- 直接映射不用替换（映射唯一）；组相联在组内选。
- **名真题型：**给访问串 + 2 路组相联 + LRU 替换，问命中率。一张草稿画"列=访问序列，行=每组两个位置，被命中打钩"即可不遗漏。若没蓄命中会硬伤。
- **写策略：**
- **写直达（Write Through）：**Cache 命中时同时写主存（可加写缓冲区），实现简单、主存与 Cache 总一致，但每次写都要（最终）等主存慢速。
- **写回（Write Back）：**只写 Cache，置脏位（dirty），直到该行被换出时才把整块写回主存。快，但要"脏位 + 换出时写回"，且主存可能暂不一致。
- **写不命中：**

- **写分配 (Write Allocate)**：先把主存块取进 Cache，再在 Cache 里写（配合写回用）。
- **非写分配 (Non-Write Allocate, 写绕过)**：不取块，直接写主存（配合写直达用）。
- 口诀："写回配写分配，写直达配非写分配" 是常见搭配。
- **多核缓存一致性 (进阶概念, 408 选择偶尔出现)**：写直达天然易维护一致；写回需要窥探 (snooping) / 监听协议 (如 MESI) 跟踪各核 Cache 的行状态 (Modified/Exclusive/Shared/Invalid)。本项目单核，无此问题，但题目会考你"为什么写回需要一致性协议"。
- **本项目落点**：CPU 访问 Flash 里的代码/常量走只读 Cache (通常 read-only 指令 Cache)；数据写 (变量、堆栈) 放 SRAM 不经过 Flash Cache——理解"哪些读写走 Cache"即可。整个 C3 的换算在 408 是 **大题必考**，务必多算。

Cache 一题全通 (综合练, 覆盖映射/位数/命中率/AMI 全部)：

某机字长 32 位，主存按字节编址，共 **16MB**，Cache **8KB**，**4 路组相联**，块大小 **16B**。

- ① 地址总位数 A? $A = \log_2 16MB = \log_2 2^{24} = 24$ 位。
- ② 块内偏移 b? $b = \log_2 16 = 4$ 位。
- ③ 组数 S? 索引位数 s? 行数 $C = 8KB/16B = 512$ 行；4 路 → 组数 $S = 512/4 = 128 \rightarrow s = \log_2 128 = 7$ 位。
- ④ Tag 位数 t? $t = A - s - b = 24 - 7 - 4 = 13$ 位。
- ⑤ 每组的"行复用关系"：主存共有多少块？每几块映射到同一组？主存块数 $= 16MB/16B = 2^{24}/2^4 = 2^{20}$ ；映射到同一组的主存块 $= \text{主存块数} / \text{组数} = 2^{20}/2^7 = 2^{13}$ (正好等于 Tag 组合数 2^{13} ，自洽)。这就是"同一组里放 Tag 不同的块"，靠 Tag 区分。
- ⑥ 含标志位时 Cache 总容量？每行 = 有效位 1 + Tag 13 + 数据 $16 \times 8 = 128$ (位) = 142 bit； $\times 512$ 行 = **72,704 bit $\approx$ 9.08 KB**。

3.6 虚拟存储器 (★页表 / TLB / 缺页, 408 大题)

- **虚拟地址 → 物理地址**：虚拟地址 = 虚页号 VPN | 页内偏移 delta 物理地址 = 物理页框号 PPN | delta (页内偏移不变)
- **页表**：一张表，页表项索引 = 虚页号，页表项内容含**有效位**、**物理页框号**、**修改位(脏)**、**访问位**等。CPU 用 VPN 查页表得到 PPN。
- **快表 TLB**：因为页表在主存里，每次访问都要"查页表"多一次访存——用 **Cache 缓存最近用过的页表项 (TLB, Translation-Lookaside Buffer)**。TLB 命中免查主存页表 → 一次地址转换 $O(1)$ 。
- **流程**：虚拟地址 → TLB 命中? → 是：直接得 PPN；否：查主存页表 → 页表项有效? → 是：得 PPN 并 (可以的话) 回填 TLB；否：**缺页中断** → OS 从磁盘/Flash 调页装入 → 更新页表/TLB → 重新执行。
- 注意几个点：
 - **只有部分程序真正在内存**，其余在磁盘 (或本项目这种嵌入式就是**固定在 Flash**)。
 - **页表项格式**：有效位(P)、修改位(D)、访问位(A)是关键。
 - **TLB 未命中 $\neq$ 缺页**，是两回事：TLB 未命中可以查页表 (慢但可能命中)；页表项无效才是缺页 (要去外存读页)。

本项目落点：ESP32-C3 没有完整 MMU（无虚拟内存/分页换入），但它用"Flash 的固定地址可映射/只读窗（如 0x42000000）"这种**固定地址转换**做过对照物。理解页表/TLB 后你就明白了"地址转换是硬件 + 页表协作"。

C3 练习题

Q1（Cache 位数）：主存 4GB（32 位地址），块大小 64B，Cache 大小 32KB **2 路组相联**，求 Tag、组号（Index）、块内偏移位数，及总共多少组。**解析：** $b = \log_2 64 = 6$ 。Cache 行数 $C = 32\text{KB}/64\text{B} = 512$ 行；2 路 → 组数 $S = 512/2 = 256 \rightarrow s = \log_2 256 = 8$ 位。Tag = 32 - 6 - 8 = **18 位**。组数 = 256。（这是 408 最典型：先算行数、再按路数求组数、再减。**Tag=A-b-s** 是唯一公式。）**Q2**（写回/脏位）：写回 Cache 中的数据，什么时候才写回主存？**解析：**命中时只写 Cache 并置脏位；当该行被换出（或其他原因淘汰）且**脏位=1**时，才把整块写回主存。若脏位=0（没改过）则直接丢弃。→ **写回 = 换出才写、靠脏位**（并配合修改位）。**Q3**（AMAT）：命中 95%，命中时间 1 周期，未命中须到主存多花 100 周期（代价 100），AMAT？**解析：**AMAT = $0.95 \times 1 + 0.05 \times (1+100) = 0.95 + 5.05 = 6$ 周期。（或写成 $1 + 0.05 \times 100 = 6$ 。）**Q4**（页表计算）：虚拟地址 32 位，页大小 4KB，页表项 4B，一级页表需要多大？**解析：**页内偏移 = $\log_2 4\text{K} = 12$ 位 → VPN = 32 - 12 = 20 位 → 页面数 = 2^{20} 。页表大小 = $2^{20} \times 4\text{B} = 4\text{MB}$ 。**一级页表太大 → 要二级/多级页表**（这是多级页表存在的动机）。（多级页表只让"用得着的"页表页在内存。例：两级页表可把常驻部分裁到很小。）

C4 指令系统（对应表 S 4，指令编码是亮点）

4.1 指令格式（RISC-V RV32 有 6 种基本格式，重点 R 与 I）

每条 RV32 指令固定 **32 位**。常用：- **R 型**（寄存器-寄存器运算，add/sub/and/or 等）：
31 25 24 20 19 15
14 12 11 7 6 0 funct7 | rs2 | rs1 | funct3 | rd | opcode
7 位 | 5 位 | 5 位 | 3 位 | 5 位 | 7 位 - **I 型**（立即数运算/加载 addi/lw 等）：
imm[11:0] | rs1 | funct3 | rd | opcode
12 位 | 5 位 | 3 位 | 5 位 | 7 位 - RISC-V 的**低 7 位 opcode**决定"这是哪一类指令"；funct3/funct7 进一步区分同类型具体运算（如 add = funct7 0000000+funct3 000, sub = funct7 0100000+funct3 000）。

为什么研考拼指令编码：因为考题常给你 addi x9,x9,-2 让你写出 32 位机器码，考你对**字段划分 + 补码立即数 + 十六进制拼合**的熟练度。

4.2 寻址方式（背 8 种 + 各举一例）

1. **立即寻址：**操作数就在指令里（addi s1, s1, 1）。
2. **寄存器直接寻址：**操作数在寄存器里（add a0, a1, a2）。
3. **隐含寻址：**操作数隐含在特定寄存器（如累加器 ACC），指令不显式写。
4. **直接寻址：**指令给出内存操作数的**主存地址**（跳转/jal 目标）。
5. **间接寻址/寄存器间接：**指令给的是"地址的地址"（lw a0, (a1) 从 a1 指向的内存取）。RISC-V 的 load/store 用"寄存器 + 偏移"。
6. **基址寻址 / 基址+偏移：**有效地址 = 基址寄存器 + 指令位移。**本项目 buf[i] 访问 = 数组基址 s0 + i × 2 的偏移**就是实打实的基址寻址。
7. **相对寻址：**PC + 指令中的偏移（beq / blt 都属于 PC 相对）。循环、if 全靠它。
8. **堆栈寻址：**操作数从**栈顶**取得，隐含用 sp / 栈指针（push/pop 对应 OS 调用栈）。

记忆：实体是"立即 / 寄存器 / 内存 / 栈"，配合"直接/间接/基址/相对"定位。

4.3 分支与跳转的范围

- beq (Branch if Equal, SB 型)：偏移量 13 位（有符号，实际 imm[12:1]），故可跳 $\pm 2^{12}$ 半字 = $\pm 4\text{KB}$ 左右（相对 PC）。**短跳。**
- **长跳转**：auipc + jalr 用高 20 位 + 低 12 位拼接出完整的 32 位目标地址，可跳到任意地址。本项目大函数分散在 Flash 里时，编译器会自动生成这种"构造全 32 位目标 + 间接跳"的序列。
- jal (Jump And Link, J 型)：无条件跳转并保存返回地址到 rd（函数调用连返回）。jalr 用寄存器计算目标（函数返回 jr ra 就是 jalr x0, 0(ra)）。

C4 练习题（指令编码，408 风格）

Q1：把 addi x9, x9, -2 编码为 32 位 RISC-V 机器码 (RV32I)。已知 addi: opcode=0010011, funct3=000, I 型。**解析（分步）：**- I 型字段：[imm[11:0]] [rs1] [funct3] [rd] [opcode] - rd = rs1 = x9 = 01001₂ - imm = -2 的 12 位补码 = 111111111110₂ - 拼接（从高位到低位）：111111111110 01001 000 01001 0010011 - = 十六进制 0xFFF48493。验证：(0xFFF<<20)|(9<<15)|(0<<12)|(9<<7)|0x13 = 0xFFF00000|0x48000|0x480|0x13 = 0xFFF48493 ✓ - **小端存储：**内存字节序为 93 84 F4 FF。**要点：**即时位负数必须写成**12 位补码**，字段顺序按格式表，最终逐字段拼位移。这个题是"计组指令编码"的标准题型。

Q2：beq x1, x2, label (SB 型) 跳跃范围为什么有限？**解析：**SB 型只给 13 位有符号立即数（乘以 2 → $\pm 4\text{KB}$ 左右半字），意味着**分支只能在当前指令附近（PC 相对小范围）跳**；超出就需 auipc+jalr。→ 回答"有限范围"是因为指令里偏移位数少"。

Q3：为什么 CPU 上电从固定地址（本项目 bootloader → 0x10000）开始执行？**解析：**CPU 复位时 PC 被硬件置为**固定入口地址**，该地址处放的就是 bootloader，再跳转到应用入口。这叫**复位向量/中断向量**的最简单形式（见 C5）。

C5 中央处理器（对应袁 § 5，★流水线与中断是大题）

5.1 数据通路：五阶段取指执行模型

一条指令完整走：

IF 取指 → 从 PC 指向的（指令）存储器取 32 位指令， $PC \leftarrow PC+4$
ID 译码 → 拆出 opcode/funct/寄存器号，读寄存器，产生控制信号，算立即数
EX 执行 → ALU 做运算 / 算访存地址（load/store 用“基址+偏移”）
MEM 访存 → load 从存储器读、store 写存储器（本项目访问 SRAM 数据）
WB 写回 → 把 ALU 结果或 load 到的数据写回目的寄存器

- 时钟周期：一条指令通常跨越多个周期（非流水线时 CPI>1）。
- **控制器**根据 opcode+funct 输出**控制信号**：选哪两个寄存器、ALU 做什么、是否访存、是否写回使能——这就是"译码驱动执行"。

5.2 硬布线 vs 微程序（选择高频）

- **硬布线控制**：控制信号由**组合逻辑电路**直接产生。特点：快、但改设计要改电路。RISC-V/RISC 多用。
- **微程序控制**：把一条指令对应的控制信号序列**先存进一块"控制存储器(ROM)"**，执行时逐条读出微指令执行。特点：灵活、易改（改微程序即可），但慢一档，CISC 多用。
- 记忆：**硬布线=硬件真电路、微程序=查表执行微指令**。
- ESP32-C3 是 RISC-V（RISC），控制相对硬布线。

5.3 ★指令流水线（计算 + 冒险）

流水线思想：五段彼此独立，像工厂流水线，**每周期都有一条新指令进入**，理想吞吐 = 1 条/周期，理想 CPI=1。

为什么流水线能提速而不用改时钟周期：把一条指令的耗时拆成 5 段，各段用**独立的硬件**（寄存器堆、ALU、存储器）同时开工。流水线**不缩短单条指令的延迟**（反而稍增——要加流水段寄存器），它提升的是**吞吐（throughput）**。这是常考辨析：**流水线提升吞吐率，不降低单条延迟**；且流水线深度受限于各段中"最慢段"的延迟（时钟周期 = 最慢段 + 寄存器开销）。

n 条指令总周期（理想、无冒险）：

$$T = 5 + (n-1) \quad (\text{第一条 5 周期灌满流水，之后每周期完成一条})$$

吞吐率 = n/T （条/周期）；吞吐趋向 1 条/周期。**加速比**（相对非流水单周期执行）：非流水 n 条需 5n 周期，流水需 $5+(n-1) \rightarrow$ 加速比 ≈ 5 （n 越大越接近段数）。

三种冒险（必背三类 + 各举解法 + 影响）：1. **结构冒险（资源冲突）**：同一硬件同一周期被两条指令同时用。例：单端口指令存储器，取指和取数据同周期就撞。 $\rightarrow$ 解决：**插气泡（stall）**，或**分离指令存储器与数据存储器**（本项目指令取 Read-Only 区、数据在 SRAM，天然分离）。2. **数据冒险（RAW，读写依赖）**：后指令要用上一条**还没写回**的结果。- 例：`add x5,x1,x2` 下一条 `sub x6,x5,x3` ——x5 还没写回。- 分类：**RAW（真依赖）**、WAR（写后读）、WAW（写后写）；五段流水无乱序时主要处理 RAW。- 解决：**转发/旁路（forwarding/ bypass）**——把 ALU 的输出直接接回到 ALU 输入，不必等 WB；有些 RAW（如 **load 后立刻用**，因为数据要等 MEM 段才到）**插 stall（nop，气泡）**。- 名真题：**load-use 数据冒险**。`lw x1,0(x2)` 后紧跟 `add x3,x1,x4`：即使有转发，load 的数据要到 MEM 段末才可用，必须**插入 1 个气泡**。会考"需要插几条 nop / 用转发可消除几条"。- 本项目 Speex 是密集算术，编译器会**重排指令 + 利用转发减少气泡**（这是优化代码性能的关键）。3. **控制冒险（分支冒险）**：分支指令还没算出跳哪，（也许）已经取错了下一条指令。- 影响：每遇到分支若猜错，要**清空已进入流水线的错误指令**（冲掉） $\rightarrow$ 浪费若干周期（称为分支惩罚 branch penalty）。- 解决：**分支预测**（预测跳/不跳，错了再冲掉；本项目 RISC-V 有简单静态/动态预测）、**延迟槽（delay slot）**、以及取指段就启动分支判断。- 注：无条件跳转（jal）不依赖运算结果，控制冒险简单；难的是**条件分支**（依赖 ALU 结果）。

例题（流水线周期）：5 段流水，1000 条指令无冒险，总周期与吞吐。**解析**： $T = 5 + (1000 - 1) = 1004$ 周期；吞吐 = $1000 / 1004 \approx 0.996$ 条/周期。

进阶例题（计入冒险的 CPI）：5 段流水，其中 20% 是分支指令，每次分支猜错清空 2 条指令（即平均每分支多花 2 周期），其余无冒险。求含冒险时的近似 CPI 与吞吐。**解析**：理想 CPI=1；因分支每条额外平均 0.2×2

= 0.4 周期 → 平均 CPI ≈ 1.4；吞吐 ≈ 1/1.4 ≈ 0.71 条/周期。→ 记住套路：

$CPI = 1 + \sum(\text{各类冒险频率} \times \text{该类冒险惩罚})$ 。

5.4 异常与中断（★必背流程，本机整流/闪退都在讲）

中断与异常的区别（选择常考）：

- **中断 (interrupt)：**由外部 I/O 设备或定时器异步触发（本项目：按键 ADC、定时器、DMA 完成、WiFi）。
- **异常 (exception)：**由 CPU 内部、指令执行引起，又分：
 - **故障 (fault)：**可恢复，如缺页（页表项无效，处理完重新执行当前指令）。
 - **自陷 (trap)：**主动触发，如系统调用 `ecall`、调试断点。处理完接着执行下一条。
 - **终止 (abort)：**不可恢复的致命错误，如除零、非法地址 → 本机表现为 `ESP_RST_PANIC` 复位重启。

中断处理流程（大题必背 8 步）：

- 0) 当前指令执行完（不损失状态）
- 1) 关中断（防止嵌套/再被打断）
- 2) 保存现场：把 PC、PSW(程序状态字)、相关寄存器压栈
- 3) 识别中断/查中断向量表 → 得到 ISR 入口（RISC-V 用 `mepc` 记被打断的 PC）
- 4) 进入中断服务例程 ISR 执行（要尽量短）
- 5) 恢复现场：弹栈恢复 PC、PSW、寄存器
- 6) 开中断
- 7) 返回被打断程序继续执行（RISC-V 用 `mret`）

- **多级中断/优先级：**高优先级中断可**抢占/嵌套**低优先级；判断优先级可用硬件链式查询或向量表。
- **本项目按键：**定时器**周期中断**读 ADC（ISR 极短）→ 再放到任务作判定 → 更新 UI。这就是最基层的**中断驱动 I/O**。
- **16k 闪退的异常本质：**非法内存访问/堆损坏触发 CPU 异常（abort）→ OC 恢复 → `ESP_RST_PANIC`，这就是“异常/终止 → 系统恢复”的注册表对应关系。

5.5 中断向量表、上下文切换

- **中断向量表 (IVT)：**一张“中断源 → ISR 首地址”的表，按中断号查。
- RISC-V 用 `mtvec` 指向异常入口；`mepc` 保存被中断的 PC；`mcause` 保存原因。这些是实现“保存现场/恢复现场”的硬件基础。

C5 练习题

Q1（流水线计算）：5 段流水执行 100 条指令（无冒险），需要多少周期？**解析：** $T = 5 + (100 - 1) = 104$ 。（背公式 $T = 5 + (n - 1)$ 。）**Q2**（冒险判断）：下面两条指令是否会有数据冒险？如何解决？

```
add x5, x1, x2
sub x6, x5, x3
```

解析：sub 需要 x5，而 x5 是上一条刚算的、尚未写回 → **RAW 数据冒险**。用**数据转发 (forwarding)**：把 add 在 EX 阶段算出的 x5 直接旁路接给 sub 的输入，不用等 WB。→ 典型转发场景。**Q3**（异常区分）：缺页中断 and 系统调用分别属于哪种异常？处理后在哪继续？**解析：**缺页 = **故障(fault)**，处理完**重新执行该指令**（OS 调好页后重做）；系统调用 `ecall` = **自陷(trap)**，处理完执行**下一条指令**。（这个“故障重执行、自陷执行下一条”是高频区分点。）**Q4**（为什么 ISR 要短）：中断服务应尽量短，为什么？**解析：**因为长 ISR 会长时

间中断/一次占用 CPU，影响其他中断与实时任务响应；通常把耗时处理放到"下半部/任务线程"（本项目把按键判决放任务线程就是这个思想）。→ 兼顾实时性与响应速度。

C6 总线与 I/O（对应表 § 7，★DMA 重点）

6.1 总线基础

- **总线**：连接 CPU、主存、I/O 的一组共享传输线（地址线、数据线、控制线）。
- **性能**：**总线带宽 = 总线位宽 × 时钟频率**（换算成字节要 ÷ 8）。
- 例：SPI 50MHz × 8bit = 400Mbps = 50MB/s；I2S 音频 16 位 × 2 声道，用 BCLK/LRCK 同步。
- **总线层次**：片内总线(CPU内部)、系统总线(CPU↔主存/北桥)、外设总线(I2C/SPI/UART/GPIO)。
- **总线事务**：请求 → 仲裁 → 传输 → 结束/释放。多外设分时共用 SPI，靠**片选(CS)**仲裁谁占用。
- **仲裁**：
- **集中式**：由总线控制器统一判优（链式查询：设备串成一串、靠优先级；计数器定时；独立请求：每个设备有自己的请求线）。
- **分布式**：各设备自己协商（如以太网的 CSMA/CD、I²C 的仲裁）。
- **串行 vs 并行**：I2C/SPI/UART 串行、一次一位；并口并行多根线。本项目 I2S 是串行但带字时钟同步。

6.2 I/O 的三种控制方式（★对照表必背）

控制方式	CPU 参与度	特点	本项目对应
程序查询(轮询)	忙等，CPU 全程查状态	简单、最慢、占 CPU	单次 ADC 读取（低频可）
中断驱动	只有就绪才响应	提高利用率、不忙等	定时器中断采样 ADC
DMA	只在开头给配置、结束 CPU 只受"完成中断"	数据由硬件 DMA 控制器 直接在外设↔主存间搬	I2S 音频收发 DMA (dma_desc_num=6, dma_frame_num=240)

DMA 工作流程（必背）：1. CPU 为该传输在 DMA 控制器里配置：**源地址、目标地址、传输长度、方向**。2. CPU 发"启动"命令后**不再管**。3. DMA 控制器每准备好一批数据，就**申请总线（仲裁拿到总线）**，在外设与主存间**成块搬运**（不经过 CPU 寄存器）。4. 整块传完后**发一个"完成中断"**通知 CPU。→ 因此录音时 CPU 只在一头一尾参与，中间时间还能跑 UI、WiFi、编码——这是"能持续录音而系统不卡"的关键。

什么是"数据不经过 CPU"：DMA 传输是在**外设和主存之间**直接搬（CPU 只读首尾），数据流不经过 CPU 的寄存器/ALU → 解放 CPU 大量访存时间。

6.3 缓冲与通道

- **缓冲(Buffering)**：用一段中间存储缓解外设与 CPU/主存的速度差、减少每个字节的中断次数。本项目：I2S DMA 环缓冲、SPIFFS 页缓冲、HTTP chunk 缓冲。
- **通道(IOP)**：更进一步用一台**小型处理器（I/O 处理机）**来管理多台设备，CPU 更省。本项目无独立通道，用 DMA 即够。

C6 练习题

Q1（总线带宽）：32 位总线、50MHz，带宽多少 MB/s？**解析**：带宽 = $32\text{bit} \times 50\text{MHz} / 8 = 2000\text{Mbps} / 8 = 250\text{MB/s}$ 。

Q2（三种 I/O）：为什么听音乐时用 DMA 而不是程序查询？**解析**：程序查询要 CPU 每字节或每小段忙等、沿途检查状态，几乎占满 CPU；DMA 由硬件自动搬运，CPU 只在两端参与 → 听音乐同时能跑 UI/WiFi。→ DMA 专治"高速、大量、持续"的 I/O。

Q3（DMA 与 CPU 关系）：DMA 传输中 CPU 忙吗？**解析**：不直接搬数据、基本空闲（可执行别的任务）；只在**传输前给配置、传输结束被完成中断**打扰。但它仍要与 DMA 控制器**共享总线/仲裁**，传输会短暂占用总线。

[第二部分小结：计组的"一根主线 + 七个计算"（背）]

这根主线：C → 编译/汇编/链接 → bin → 装载 → CPU 取指 → 译码 → 执行，全部嵌在 C1/C2/C4/C5。 **七个必算**：1. 性能：CPU 时间 = 指令数 × CPI × 时钟周期。2. 补码：取反+1；符号扩展。3. 溢出：同号相加改变符号 → 溢出（进位法/双符号位）。4. 定点 Q：±按整数、乘后右移回原 Q。5. IEEE754： $(-1)^S \times 1.f \times 2^{(E-127)}$ ，阶码偏置 127。6. Cache：Tag=A-b-s、容量含 Tag、AMAT。7. 流水： $T=5+(n-1)$ 、三种冒险与解法（转发/分支预测）、中断 8 步。

第三大部分 操作系统（管"资源"，计算与 PV 是大头）

操作系统研究"多个程序怎么公平且高效地共享一台机器"。408 中重视**计算（调度、页面置换、死锁判定）与算法设计（PV 操作）**。本项目跑的是 FreeRTOS（一个真正的实时操作系统），所以 OS 的所有概念在这台设备里都有真实实现：任务=线程、信号量、事件队列、文件系统 SPIFFS、NVS。

01 OS 概述与运行环境

1.1 OS 的功能与特征（背）

- **五大功能**：处理器（进程/线程）管理、存储器管理、文件管理、设备管理、网络与联网。
- **四大特征**：**并发、共享、虚拟、异步**（必背）。
- 并发：多个进程同时存在、交替执行（本项目多任务：UI / 录音 / WiFi 并发）。
- 共享：资源可被多进程同时/互斥共享。
- 虚拟：把物理资源抽象成逻辑上的更多（虚拟内存、虚拟设备）。
- 异步：进程执行速度不定、推进无严格时序。

1.2 内核态 / 用户态（特权级）

- CPU 有**特权指令**（改内存映射、关中断、调 I/O、切换进程），只能**内核态**执行。
- **用户态**：普通程序只能执行用户指令，想用特权功能要**陷入内核**。
- RISC-V 有三种模式：**U 用户、S 监督、M 机器**。本项目 ESP32-C3 通常只跑裸机/FreeRTOS，应用多位于特权更高的模式，但**概念上仍分用户/内核**。
- 好处：隔离、保护操作系统不被应用程序破坏。

1.3 系统调用（system call）

- 应用程序不能直接访问硬件，必须通过**系统调用**接口请 OS 帮忙。
- 实现机制：用户程序执行 `ecall`（陷阱指令）→ CPU 切内核态 → OS 查系统调用号 → 执行服务 → 返回用户态。
- 本项目 `fwrite`（SPIFFS 文件写）、`socket / connect`（LwIP TCP）、`sleep` 最后都落到 OS 提供的服务 = 系统调用。
- **库函数 vs 系统调用**：`fprintf` 是库函数，底层可能调用 `write` 系统调用；库函数不一定要进内核。

1.4 中断在 OS 中的地位（与计组衔接）

- OS 的调度、I/O 完成、时钟节拍**全靠中断**驱动。
- 本项目 FreeRTOS：`esp_timer` 产生时钟节拍 → 中断/tick 触发调度器 → 每个任务按时间片运行。这就是"中断是 OS 的心脏"的体现。

O1 练习题

Q1: 下列哪项必须在内核态执行? (a) 读用户内存 (b) 关中断 (c) 取一个算术 (d) 读寄存器 **解析:** (b) 关中断是**特权指令**, 只能内核态; 其余用户态即可。→ 记"对硬件特权资源的操作用特权指令、必在内核态"。**Q2:** 为何系统调用比普通函数调用慢? **解析:** 系统调用要**切换用户态→内核态**、保存/恢复更多现场(用户上下文、内核栈), 还要查系统调用号并经过权限检查 → 开销比一次普通函数调用大得多。

O2 进程与线程 (★调度、同步、死锁是全年重点)

2.1 进程、线程、PCB

- **进程:** 资源分配的基本单位 + 调度的基本单位 (现代 OS 常把"调度"交线程)。
- **线程:** 进程内的一个**独立执行流**, 是**处理机(调度)的最小单位**; 同一进程的线程**共享**地址空间与资源, 切换开销比进程小。
- **本项目 FreeRTOS 任务 = 线程:** 每个任务有独立栈、优先级、状态; 任务间用信号量/队列通信。

进程控制块 PCB (Process Control Block): 进程的"户口本", OS 管理进程都用它, 含: - 进程标识 (ID、祖先); - **CPU 现场** (PC、寄存器镜像、栈指针) ——切换时保存/恢复; - 调度信息 (优先级、状态、已用时间); - 内存信息 (代码/数据/栈所在地址); - 资源和文件信息; - 与进程状态、优先级相关的排队指针。

本项目对应: FreeRTOS 的任务 = TCB(任务控制块) + 私有栈 + 优先级 + 状态。

`uxTaskGetStackHighWaterMark()` 看的就是这个任务的"现场移动够不够" (栈余量)。

2.2 进程的状态与转换 (三态/五态)

三态模型:

就绪(Ready) —— 调度 → 运行(Running) —— 时间片到/被抢占 → 就绪
运行 —— 等待事件 → 阻塞(Blocked) —— 事件到达 → 就绪

- **就绪:** 可被调度但没有 CPU。
- **运行:** 正占用 CPU。
- **阻塞/等待:** 等某个事件, 即使有 CPU 也不能执行。

五态: 加了**新建** (创建中) 与**终止** (退出但还没回收)。- 转换要点: "**运行→阻塞**"只能等事件; "阻塞→就绪"由事件唤醒; **不是任何两个状态都能互转** (如"就绪→阻塞"不可能, 因为就绪说明能运行不会等 CPU)。

本项目落点: 录音任务在"运行/就绪/阻塞 (等 USB/帧就绪、等文件写完成、等事件)"切换; UI 任务阻塞等 `poll_evt` 事件返回, 就是"运行→阻塞→就绪"的过程。**状态机是 408 常考"画状态转换图 + 谁触发"**。

2.3 上下文切换 (context switch)

- 从一个进程切到另一个: 保存当前进程的**现场 (PC、寄存器、栈指针、PSW)** 到其 PCB → 恢复目标进程的**现场**。
- 上下文切换有**开销** (纯账, 无计算产出) —— 这就是调度频率、时间片大小的权衡依据。任务太多、切换太频繁会浪费。

2.4 ★处理机调度（调度算法 + 会算平均周转/等待）

一批指标（背）：- **周转时间** = 完成时间 - 到达时间 = 等待 + 运行（含阻塞）。- **带权周转** = 周转 / 运行时间。
- **平均周转 / 平均等待**：求和取平均。- **响应时间**：从提交到首次得到 CPU 的时间（交互式重要）。

三种层面：- 高级（作业）、中级（进程换入换出）、低级（进程内任务调度——本项目 FreeRTOS 调度器）。

经典算法（必背 + 会算）：

1. **FCFS 先来先服务**：按到达顺序调度，**对大作业友好？**（不对）——大作业先到则后面的短作业全等，**平均等待较大**。非抢占、公平。
2. **SJF 短作业优先**（非抢占）：谁运行最短先调度。**平均等待最短**（理论上最优，用于对比）。- 抢占版 **SRTB/SJF 抢占**：新到达的更短任务可抢占当前。
3. **优先级调度**：优先级高的先。可抢占或非抢占。**低优先级可能饥饿**。
4. **RR 时间片轮转**：按时间片轮换，每个就绪进程一次时间片。时间片太大→退化成 FCFS；太小→切换开销太大。**响应性好**。
5. **多级反馈队列（MLFQ）**：多个队列，优先级从高到低、时间片从短到长；高队列未用完时间片仍没做完→降级到下一队列。兼顾响应与长作业不饿死。**综合派，408 重点**。

调度示例（SJF 计算）：- 到达 0，运行 8/4/1/9 → SJF 非抢占顺序 = 1(先) → 4 → 8 → 9（因为全 0 时刻到达，运行最短先）。- 完成时刻：1、1+4=5、5+8=13、13+9=22。- 等待时间：P3=0、P2=1、P1=5、P4=13 → 平均 = (0+1+5+13)/4 = 19/4 = 4.75。- 若 FCFS（按 8,4,1,9）：完成 8,12,13,22；等待 0,8,12,13=33/4=8.25。→ SJF 更优，验证“最短作业平均等待最小”。

本项目落点：FreeRTOS 用**优先级抢占 + 时间片轮转**：录音/编码任务设较高优先级，UI 较低；事件处理适配实时性。多级反馈的思想在“优先级高先于低 + 不饿死低优先级任务”里也能体现。

2.5 同步与互斥（★PV 操作，大题）

临界资源 / 临界区：一次只允许一个进程使用的资源（打印机、共享变量），访问它的代码段叫**临界区**。- 四个“进入临界区”满足的条件 / **三大原则**（背：**互斥、让权等待、有界等待（不饥饿）、空闲让进**）。

软件/硬件方法：- 关中断 / Test-and-Set (TAS) / Swap 等硬件指令实现互斥。- **信号量（Semaphore）**：整型变量 + 等待队列 + PV 操作。

信号量 PV（P=V 是荷兰文）：- **P（wait / 申请）**：value--；若 value<0 则阻塞入队。- **V（signal / 释放）**：value++；若 value≤0 则唤醒队首。- **二元信号量**（value 0/1）等价于**互斥锁**：P 拿到就进临界区，V 释放。- **计数信号量**：允许多个线程同时用（value 表示可用资源数），本项目典型：

本项目真实同步对象（背这种映射）：- bsp_lvgl_lock(timeout)：GUI 用的是**互斥量 MUTEX**，还带**优先级继承**（防优先级倒置——低优先级任务持锁时高优先级任务等锁，低优先级先持锁完成、避免系统卡死）。- 录音事件的环形队列：有界**生产者-消费者**队列，生产者（录音任务）入队、消费者（UI 任务）出队。- 文件写互斥：多个任务都不阻塞地写同一个 FRC 文件 → 用锁串行化。

经典同步问题（必背伪码，408 大题）：

(1) **生产者-消费者**（有 K 个槽缓冲）：

```
empty = K (空槽数) , full = 0 (满槽数) , mutex = 1 (保护内层互斥)
生产者: P(empty); P(mutex); 放入; V(mutex); V(full);
消费者: P(full); P(mutex); 取出; V(mutex); V(empty);
```

- 为什么里面还要 mutex: empty/full 只保证"有空间/有货", 但"放/取"本身是读改写, 两个进程同时改缓冲区会乱; mutex 把"放/取"串行化。这四行是必默写的黄金模板。

(2) 读者-写者: - 读者优先: 多个读者可同时读, 写者必须等没有读者; 读者用 reader_count 维护, 第一个读者 P(互斥写), 最后一个读者 V(互斥写)。- 写者优先 / 不饥饿: 要处理队列避免读者无限占住。

(3) 哲学家进餐: - 每位哲学家需要两把叉子才能吃。直接"拿左叉→拿右叉"会死锁 (5 人各拿一把左叉互等)。- 解决: ① 规定一次最多只能有 4 人同时取餐 (= 限制同时 4 人竞争); ② 奇偶编号, 奇数先左后右、偶数先右后左; ③ 谁拿不到就放下。左手拿不到右手叉的约束破坏"循环等待/持有等待"。- 关键对比 (很容易考): 哲学家进餐的最终目标其实是"让所有人都能吃完" (无死锁、无饥饿), 不追求"两个人同时吃", 所以数学上"同时有 5 人竞争"的进餐标准是死锁; 去掉一个席位或限制 4 人即破坏循环等待。

读者-写者 (RW, 读多写少场景的经典, 背伪码骨架):

```
写者互斥信号量 rw = 1, 读者计数信号量 mutex = 1, 读者计数 count = 0
读者: P(mutex); count++; if(count==1) P(rw); V(mutex); // 第一个读者锁写
读...
P(mutex); count--; if(count==0) V(rw); V(mutex); // 最后一个读者放写
写者: P(rw); 写; V(rw);
```

- 读写公平题 (408 常问): 上面是"读者优先"——写者可能被不断到达的读者无限饿死。加"信号量 w (写者排队)"或用"读写都排队"的队列可做到写者优先/公平。会考你"如何改成写者优先"。

生产者-消费者题型变体 (务必会推): - 单生产者单消费者 + 有界缓冲: 就是上面的 3 信号量模板。- 多生产者多消费者: P(empty) 在多生产者之间也要互斥 (其实 empty 本身自带互斥保证, 但"放"加 mutex)。- 每题先识别哪几个是"计数约束" (empty/full)、哪几个是"互斥" (mutex), 再按"先资源约束、后互斥"的顺序写 P, V 相反序。P 顺序错会死锁 (如先 mutex 再 empty 且缓冲区空时两个生产者各持 mutex 等 empty) ——这是 408 设陷阱点。

PV 为什么难但必考: 因为它不是"背模板算数", 而是"用信号量表达约束"。408 大题常让你补 PV、判死锁、分析是否可以如何改进。工程落点: 本项目每个"两个任务都要碰的东西" (GUI、文件、事件队列) 都是一个临界区的化生, 你没有显式写信号量也能看出来"为什么必须锁"。

一道课上推敲 (信号量初值): 若两个进程都要"打印后再发消息", 用二元信号量把 print() 串行化, 且"打印完成"前另一方不能发——按上面的"先给计数/资源、后互斥"写即可, 注意信号量初值决定"先到者是否被挡"。

2.6 死锁 (★判断与避免可计算)

死锁四必要条件 (都存在才可能死锁): 1. 互斥: 资源一次仅一任务能用。2. 占有并等待 (持有并请求): 已持有一个资源, 又申请别的。3. 不可剥夺: 已占资源不能强抢。4. 循环等待: 存在一个"互相等着对方资源"的环。

破坏方法（每个必要条件的反着来）： - 破坏"持有并等待"：一次性申请全部资源。 - 破坏"不可剥夺"：允许在申请失败时释放已持资源。 - 破坏"循环等待"：给资源编号，只能按序申请（单调）。 - 互斥在有些资源（如打印机）上不能破坏。

死锁避免：银行家算法（★必须会手算） - 每次分配前**试探**，看是否仍存在**安全序列**，存在才分配。 - 数据结构：Max（每进程最大需求）、Allocation（已分配）、Need=Max-Allocation、Available（当前可用）。 - 安全判定：找 $Need[j] \leq Available$ 的进程，模拟分配并回收其所有资源（ $Available += Allocation[j]$ ），循环，能全部推进则安全。 - **银行家算法手算演示（务必自己推一遍）：**

系统三类资源 A/B/C，Available = (3,3,2)；四个进程：P0: Allocation=(0,1,0), Max=(7,5,3) → Need=(7,4,3) P1: Allocation=(2,0,0), Max=(3,2,2) → Need=(1,2,2) P2: Allocation=(3,0,2), Max=(9,0,2) → Need=(6,0,0) P3: Allocation=(2,1,1), Max=(2,2,2) → Need=(0,1,1) P4: Allocation=(0,0,2), Max=(4,3,3) → Need=(4,3,1)

问当前是否安全、找安全序列。解：初始 Available=(3,3,2)。找 $Need \leq Available$ 的：P1(1,2,2)✓、P3(0,1,1)✓、P4(4,3,1)✗、P0✗、P2✗。挑 P3：回收 → Available=(3,3,2)+(2,1,1)=(5,4,3)。再 P1 → (7,4,3)。再 P4 → (7,4,5)。再 P0 → (7,5,5)。再 P2 → (10,5,7)。→ **安全,序列如 P3,P1,P4,P0,P2**（不唯一）。**变体陷阱**：如果在资源紧张时把 Available 换成分配请求后状态再判,答案会骤变——**先请求改变状态,再对该新状态运行安全算法。**

死锁检测与解除： - **资源分配图**：进程结点 + 资源结点，申请/分配作有向边。**图中存在环 = 死锁**（结合必要条件判断是否真是死锁）。 - 解除：剥夺资源、撤销进程、回滚。 - **判定细节**：资源分配图中，若每个资源类型只有一个实例,成环等价死锁；若一个资源类型有多个实例,还要用"**简化分配图**"（把能完成的进程边去掉）——能全部简化则不死锁。

本项目落点：嵌入式里极简资源（内存、外设、文件锁）。若"录音任务占文件锁不放、WiFi 任务又要同一文件锁"，就会死锁/阻塞 → 所以本项目用了**带超时的锁（timeout）**，等不到就报错而不是无限挂起——这就是"**死锁避免靠超时 + 不无限等待**"的工程化。

O2 练习题

Q1（SJF/FCFS）：四个进程到达均为 0，服务时间 8,4,1,9，分别算 SJF 与 FCFS 平均等待时间。**解析**：见上——SJF 顺序 1,4,8,9，完成 1,5,13,22，等待 0,1,5,13=**4.75**；FCFS 顺序 8,4,1,9，完成 8,12,13,22，等待 0,8,12,13=**8.25**。→ 最短作业平均等待最小；长作业在 FCFS+LBF 后会被拖累→为什么引入抢占/分级。**Q2（银行家）**：见上表。判断系统是否安全（需写安全序列）。**解析**：

P1: $Need[1,2,2] \leq Avail[3,3,2]$ → 可完成，回收 → Avail=[5,3,2]
P3: $Need[0,1,1] \leq [5,3,2]$ → 可完成 → Avail=[7,4,3]
P0: $Need[7,4,3] \leq [7,4,3]$ → 可完成 → Avail=[7,5,3] (P0 Alloc [0,1,0]→7? 0+0? 注意Allocation是列向量)
准确：完成 P1 回收其 Alloc[2,0,0] → 以后逐步累积。
P4 → P2：一路可推进。

→ 存在安全序列（如 P1,P3,P0,P4,P2）→ **安全**。**手算套路**：每次找 $Need \leq Available$ 的进程，把其已分配全回收，更新 Available。**Q3（死锁判据）**：哲学家 5 人拿叉子，每人先左叉，会死锁吗？**解析**：每人持一把

左叉、都等右叉→**循环等待**成立→死锁。已通过"限制同时 4 人取餐"破坏"循环等待/持有并等待"解决。→判死锁看**四个必要条件**。**Q4**（临界区原则）：进入临界区要满足四条件中的"有界等待"指什么？**解析**：一个进程提出进入请求后，**不能无限期等**（必须在有限时间内进入）——防止饥饿。有界等待 ⇔ 不让任一进程饿死。

O3 内存管理（★分页、页面置换计算是大题）

3.1 连续分配

- **固定分区**：内存预先切成固定大小的区，每区装一个进程。简单但**内部碎片**多、利用率差。
- **动态（可变）分区**：按需分配，有**外部碎片**（小块碎缝没法用）。
- **动态分区分配算法**：
 - 首次适配（first fit）：从头找**第一个够大的空块**。快。
 - 最佳适配（best fit）：选**最小够用空块**。碎片少但慢、产生小碎片。
 - 最坏适配（worst fit）：**最大空块**。减少小碎片。
 - 循环首次适配。
 - **碎片/紧凑（compaction）**：把分散的小块合并成大块，代价是搬移进程。

本项目落点：ESP 堆就是**动态堆分配 + 外部碎片**——它没有分页，用的就是"连续分配策略"。之前的"空闲堆约 33KB、最大连续块 16KB"正是**外部碎片**的活例子：总空闲还有 33KB，但最大连续块只有 16KB，大对象（如 16KB 任务栈）分配不出来，就"rec error/崩"。**OS 章节的"碎片"在这台设备上真实出现。**

3.2 页式存储（★必会换算）

思路：把逻辑地址空间和物理内存都**切成同样大小的页**。进程的"页"可放在**不连续的物理页框（块）**里——解决外部碎片。

- **逻辑地址 = 页号 P | 页内偏移 W**，其中页大小 = 2^w → 页内偏移占 w 位。
- **物理地址 = 页框号 F | 页内偏移 W**（页内偏移在逻辑/物理里相同）。
- **页表**：页号→页框号的映射（一个页表项：有效位 + 页框号 + 修改位/访问位）。
- **换算**：物理地址 = 块号 × 页大小 + 页内偏移。

例题：页大小 4KB（0x1000），逻辑地址 0x3E24，页表第 3 项映射块 9。- 页号 = $0x3E24 \gg 12 = 3$ （因为 0x3E24 高 12 位是页号），页内偏移 = 0xE24（低 12 位）。- 块 $9 \times 0x1000 = 0x9000$ ；物理 = $0x9000 + 0xE24 = 0x9E24$ 。**步骤套路**：页号找块、块号拼页内偏移。每次读题都先定 w（页大小取 $\log_2$ ），再分字段。

为什么分页能消除外部碎片但引入"页表占用 + 查表开销"：页表本身要存、内存访问两次（先查页表再取数据），所以有 TLB/快表加速（见 O3.5）。

3.3 分段 / 段页式

- **分段**：按**逻辑模块**（代码段/数据段/栈）划分，大小可变；**段表项 = 段号 →（段基址、段长）**。逻辑地址 = 段号 | 段内偏移。好处：好共享、好保护、好动态增长（段可独立扩展）；缺点：有内部（段内页对齐问题）+ 外部碎片、段长可变难管理。
- **段页式**：先分段、再分页（两段映射），兼顾"分享/保护"与"无外部碎片"。两套表：段表 + 页表。

解题要点：**分清"地址由谁拆"**：- 页式：逻辑 = 页号 | 偏移（拆物理也对齐）。- 段式：逻辑 = 段号 | 偏移，物理不一定连续（按段表基址+偏移）。- 段页式：段号 → 页表 → 页框。

3.4 ★页面置换算法（★计算大题）

当内存页框不足、要调入新页时，换出哪一页？

- **OPT 最优（理论下界）**：换出"未来最长时间不再访问"的页。**实际不可知**（未来），只做对照。手算用它做"下界"，看算法离最优差多少。
- **FIFO**：换出最早调入的页。实现简单，但有 **Belady 异常**（页框变多，缺页反而变多）。
- **LRU（最近最久未用）**：换出"最久没被访问"的页。命中率高，但需要硬件计数器/链表维护访问最近度（开销大）。**手算时盯"上次访问顺序"——谁最久没被碰就换谁。**
- **简单 CLOCK / 改进 CLOCK**：近似 LRU——用"访问位"循环扫描，可改为"干净/脏、使用/未用"四类双循环。**手算经常考 CLOCK（循环替换）。**

手算为保证：缺页次数 / 缺页率 - 命中率 = 命中访问次数 / 总访问次数；缺页率 = 缺页次数 / 总访问次数。-（首屏（刚开始从空内存调入的）都算缺页。）- **算 LRU 不丢步的写法**：维护一张 3 行的表，每行是一个页框位，访问某页若在 → 把它"挪到最后/标记最新"；缺页 → 把"最旧"那行换出并把新页标成最新。画列 = 访问序列、行 = 3 个页框、每格写当前内容即可。

例题（同样访问串，三种算法对比，背答案）：访问串 1,2,3,4,1,2,5,1,2,3,4,5，页框 3：- **FIFO**：逐步踢最早调入的（见下），缺页 **10**。1 → [1](缺) 2 → [1,2](缺) 3 → [1,2,3](缺) 4 → 踢 1 [2,3,4](缺) 1 → 踢 2 [3,4,1](缺) 2 → 踢 3 [4,1,2](缺) 5 → 踢 4 [1,2,5](缺) 1(在) 2(在) 3 → 踢 1 [2,5,3](缺) 4 → 踢 2 [5,3,4](缺) 5(在) - **LRU**：换"最久没访问"的，缺页 **9**（比 FIFO 少 1）。设计 LRU 时比 FIFO 多命中第 3 个 1、第 3 个 2。- **OPT**（已知完整访问串，看未来）：缺页 **7**，是最优下界。

为什么 LRU 用"最近"而 OPT 用"未来"：LRU 用**历史**近似"未来还会不会再用"——你刚用过的、八成还会再用（时间局部性）。OPT 要"神预言"未来，只作标准。

CLOCK 算法（手算步骤，常考） - 每个页有**访问位 R**（被访问置 1）。指针循环扫：- R=0 → 换出它；R=1 → 清 R 为 0 并让指针继续。- **改进 CLOCK**：每页有 (R, M)（访问位 M=修改位），按 (0,0) → (0,1) → (1,0) → (1,1) 偏好顺序先淘汰"干净未用"，其次"脏未用"，第三"干净用过"，最差"脏用过"。好处：优先淘汰**没改过的干净页**，省去写回外存。- **考点**：CLOCK 的第一次扫描会把 R 清零，所以刚被访问过的页也能在第二轮被淘汰。

3.5 页面分配、抖动、TLB（快表）

- **抖动（thrashing）**：同时运行的进程驻留集太小，频繁缺页换入换出，利用率极低。解决：设计合适的驻留集、降低多道度。
- **工作集**：一段时间内实际访问的页面集合；工作集小于内存驻留集则较稳。**驻留集 ≥ 工作集**才不易抖动。
- **TLB（快表/旁路转换缓冲）**：一级查 TLB（Cache），命中直接拿页框号；未命中查主存页表；页表项无效才缺页。**TLB 未命中 ≠ 缺页**。
- **两级页表**（页表太大时）：逻辑地址拆成 页目录号 P1 | 页表号 P2 | 页内偏移，多一次查表但省内存（页表可部分不在内存）。408 常考"逻辑地址位数 → 各级页表项数"。

O3 练习题

Q1 (分页换算)：页大小 512B，逻辑地址 0x0A3F，页表第 5 项→块 12，物理地址？**解析**：512B=2⁹→低9位页内偏移=0x3F，高(逻辑地址位数-9)位是页号。设逻辑地址 0x0A3F=二进制...010 1000 0111 1111，低9位=001 1111111 页内偏移；页号=10100₂=20? 需给定位数——**套路**：偏出去 9 位，页框 12×512=0x1800，物理=0x1800+0x3F=0x183F。**Q2** (置换)：访问串 1,2,3,4,1,2,5,1,2,3,4,5，页面 3，求 LRU 缺页数与 FIFO 缺页数，谁的缺页少？**解析** (略推)：二者都 10 左右；此题 LRU 与 FIFO 缺页数相同 (经典用例)，但 **FIFO 会出现 Belady 异常而 LRU 不会**——这是关键概念考点。**Q3** (Belady 异常)：只出现在哪种算法？通常增加多少页框就减少缺页？**解析**：Belady 异常**只发生在 FIFO** (有时也见于 CLOCK，但 LRU/OPT 不会)。一般页框增多，缺页应减少，Belady 例外。→记"FIFO 有 Belady、LRU/OPT 没有"。**Q4** (TLB vs 缺页)：虚地址命中 TLB、命中页表、缺页，三者的访存次数差异？**解析**：命中 TLB=1 次 (快表)；未命中查页表但页表项有效=慢 (访问页表再取数据，多次) 但没有缺页；缺页需先从磁盘/Flash 调页。**分层缓存思想** (见 C3)。

O4 文件管理 (★索引文件、磁盘调度是大题)

4.1 文件与逻辑结构

- 文件 = 一段有名字的逻辑相关信息的集合。
- **逻辑结构**：无结构 (字节流)；有结构——定长记录 / 变长记录。
- **逻辑按内容访问 vs 按位置**。

4.2 文件的物理结构 (★重点)

把逻辑文件映射到磁盘/Flash 块：1. **连续分配**：文件占**连续的若干块**。随机访问快，但难扩展、有外部碎片。2. **链式 (链接) 分配**：文件块用"下一块指针"串起来。无外部碎片、易扩展，但**只能顺序访问** (跳转靠指针)、指针多耗空间、坏链风险。3. **索引分配 (inode)**：文件有个**索引块**，记录它所有数据块的块号。随机访问快、无外部碎片。- 单级索引：一个索引块，可指文件的所有块 (受索引块能放多少指针限制)。- **多级/混合索引**：直接块 + 一级/二级/三级间接索引。**本题本项目"索引思想"**。

混合索引计算 (408 必考)：设块大小 B (字节)、每个块号指针 4 字节：- 每个索引块能容纳指针数 = B/4。- 直接块：12 个直接指针 → 直接数据 12×B。- 一级间接：1 个指针指向一个含 B/4 个指针的块 → 可覆盖 B/4 块。- 二级间接：1 个指针 → 指向的块里又有 B/4 个"一级索引" → 可覆盖 (B/4)² 块。- 三级同理 (B/4)³。

例题：块 4KB、指针 4B、inode=12 直接 + 1 一级 + 1 二级。- 一级可指 4096/4=1024 块 → 覆盖 1024×4KB=4MB。- 最大文件 = 直接 12×4KB + 一级 1024×4KB + 二级 1024²×4KB ≈ 4GB。- 访问偏移 4MB 处：先走直接块 (前 12 块=48KB 之后)，再走一级间接→1 次读索引块 + 1 次读数据 = **2 次磁盘**。

本项目 FRC 文件系统 (SPIFFS)：用索引/位图管理块，本质接近"索引分配 + 位示图"；FFS 把块分配、判断空余用位图+B+树美化。题目测你的"结构层次 + 算多少块/几次磁盘"。

4.3 目录与 FCB、路径

- **FCB (文件控制块) / inode**：文件名、类型、大小、物理位置、访问权限、时间戳等。是文件的"PCB"。本项目每个 FRC 文件的 inode/头部保存这些。
- **目录**：目录项存"文件名 + 指向 inode/FCB 的指针"。

- **目录结构**：单级（一个目录装所有文件——本项目 /rec/ 录音区就是**单级目录**）、两级、树形（最流行）、无环图（共享）。
- **路径解析**：/rec/R0000007.FRC 从根开始逐级查目录表，最后找到该文件 FCB → 数据块。
- **目录查找**：顺序/索引/哈希。

4.4 空闲空间管理与磁盘调度（★计算）

空闲空间管理：- **空闲链表**（链空闲块）。- **位示图（bitmap）**：一个 bit 表示一块是否空闲——本项目 SPIFFS 用位图管理 Flash 块。考"第 k 块对位示图第几字节第几位"。- **成组链接法**（大文件系统）。

磁盘调度算法（求平均寻道距离）：设磁盘磁道 0~199，当前在 100，请求队列 {55,58,39,18,90,160,150,38,184}：- **FCFS**：按到达顺序，寻道距离 = 累加相邻差的绝对值。- **SSTF（最短寻道优先）**：每次选离当前最近的道（近似最短）。可能"来回"导致**饥饿**。- **SCAN（电梯）**：先向一个方向走到头再回头，中途把经过请求都服务。- **C-SCAN（循环扫描）**：只向一个方向服务，回到 0 再从 0 扫（减少回头抖动，等待更均匀）。- 计算就"模拟走一遍，逐段累加距离"。

4.5 文件一致性 / 原子性（本项目活案例）

- 本项目录音"先写 ACTIVE.TMP，完成后 **rename** 成 R0000xxx.FRC"：这是一个**原子改名（提交点）**。崩溃时旧文件完整、新文件不出现，系统一致。
- `nvs_commit()` 是 NVS 的**事务提交**（把待写入的变更最后落盘、带 CRC）——类比数据库"提交即生效"。这就是 **OS/文件系统"原子性/崩溃安全"**的工程实现。

O4 练习题

Q1（索引）：块 1KB、指针 4B，inode 10 直接 + 1 一级 + 1 二级，求最大单文件大小与访问第 3000 块需几次磁盘。 **解析**：一级索引指针数 = $1024/4=256$ → 覆盖 $256 \times 1KB=256KB$ 。直接 10 块=10KB。最大 = 10KB + 1 级 256KB + 二级 $256^2 \times 1KB \approx 64MB$ 。第 3000 块：直接 10 块、一级 256 块 → 还没到 3000，走二级：先读二级索引块(1次)→读到对应的一级索引块(1次)→读数据块(1次) = **3 次**（访问索引要 2 次 + 数据 1 次）。 **Q2（位示图）**：某文件系统用位示图管理 4096 个块，每个字节 8 位，问需多少字节？ **解析**：4096 块 = 4096 bit = 512 字节。→ 位示图只用 512B，非常省。 **Q3（磁盘调度）**：当前磁道 100，请求 55,58,39,18,90,160,150,38,184，SSTF 的访问顺序？ **解析**：从 100 出发，最近的 90(距10)→55(35)? 90 后 55 距 35，58 距 37，选 55→58(3)→39(19)→38(1)→18(20)→150→160→184……算出顺序并求总寻道距离。（重点：**SSTF 每次就近**，会来回。） **Q4（原子性）**：为何用"临时文件 + rename"而不是直接把分块写进正式文件？ **解析**：rename 是**原子操作**。若直接写正式文件，中途崩溃会留下"半截文件"（损坏）。写临时文件、最后一次 rename 改成正式名——崩溃时要么旧文件完整、要么新文件完整，**绝无中间态**。→ 保证文件一致性（会见投标本）。

O5 I/O 管理（与计组 C7 呼应）

5.1 I/O 系统的软件层次

用户进程（读/写文件）
I/O 设备独立性层（通用接口：open/read/write，设备驱动差异被屏蔽）
设备驱动（os_sys、硬件操作）

- 本项目 bsp_* 封装就是把设备相关性藏在驱动层，上层用统一 API → **设备独立性层 + 驱动**。
- **DMA / 中断**在计组 C7 已详述，这里同样适用于"录音持续传数据、CPU 干别的"。

5.2 缓冲技术

- **单缓冲/双缓冲**：一块/两块缓冲交替装入，减少等待。
- **循环缓冲/缓冲池**：若干缓冲块循环使用。
- 本项目 I2S 双环缓冲 + DMA，SPIFFS 页缓冲，HTTP 分块缓冲——都是"让 CPU 和慢设备解耦"。

5.3 SPOOLing（假脱机技术）

- 用**磁盘作为中介缓冲**，把独占设备（如打印机）模拟成"可同时占用"的共享设备：作业先写到磁盘上的"输出井"，由 SPOOLing 后台程序一批批真实输出。
- 本项目类比："要回放的录音排成一个队列，后台依次读文件送入音频"就是 SPOOLing 的简化。

5.4 设备分配

- 独享设备（打印机）/共享设备（磁盘）/虚拟设备（SPOOLing 出来的）。
- 分配策略：安全与否、是否可剥夺；逻辑分配/物理分配。

O5 练习题

Q1（为何要缓冲）：程序每次写 1 字节，设备按块存取，为什么要缓冲？**解析**：缓冲把多次小写**聚合**，减少对外设（磁盘/Flash）的访问次数与中断次数，提高效率（写 1 字节聚合到一页再一次性写 Flash）→ 缓解 CPU 与慢设备速度差。**它是计组 C7"缓冲"在 OS 的落地。****Q2**（SPOOLing）：SPOOLing 用什么把独占设备变成共享？**解析**：磁盘+后台进程形成"虚拟设备"，多个作业的输出**同时排队**、互不阻塞，从使用者看设备仿佛可独占可共享。→ "凭空多出逻辑设备"就是虚拟化概念。

[第三部分小结：OS 的"六张计算表"（背）]

1. **调度**：SJF 平均等待最短；RR 时间片权衡；算平均周转。
 2. **PV**：生产者-消费者 4 行模板；读者写者；哲学家防死锁。
 3. **死锁**：四必要条件 + 银行家安全序列（手算）。
 4. **分页**：逻辑地址拆页号|偏移、物理=块号×页大小+偏移。
 5. **置换**：FIFO/LRU/OPT/CLOCK 手算缺页，Belady 只在 FIFO。
 6. **索引/磁盘**：inode 直接+间接的容量与访存次数；SSTF/SCAN 求寻道。
-

第四大部分 计算机网络（把"跨机器传数据"讲透）

计网考试既能考分层概念，又能考算（子网划分、复用、拥塞窗口、往返时延）。这台设备是智能配网 + HTTP 下载的真实网络节点，所以 TCP 握手、DHCP 配网、HTTP 请求/下载全都能"摸到"。下面自底向上逐层细讲，重点层（网络层、传输层）放最多笔墨。

N1 概述与分层

1.1 为什么要分层（背）

- 把复杂的网络通信拆成若干互相独立的层，每层只关心"本层的功能与对等通信"，层间用服务接口衔接。
- 好处：各层独立设计/替换、模块化、易于实现与标准化。

1.2 OSI 七层 vs TCP/IP 五层（本项目 LwIP 用五层）

OSI 7层	TCP/IP 5层（考点常用）	典型案例
应用	表示 会话 → 应用层(应用/表示/会话合并)	HTTP、DNS、FTP、SNTP、DHCP
传输	→ 传输层	TCP、UDP
网络	→ 网络层	IP、ARP、ICMP、路由
数据链路	→ 数据链路层	以太网、WiFi(802.11)、MAC
物理	→ 物理层	电信号/射频、编码调制

- 收发自底向上，对等层之间"逻辑"通信，实际数据每层加"首部"封装（见 N3 组帧）。

1.3 性能指标（背 + 会算）

- 带宽 (bandwidth)：信道最高数据率，单位 bps (bit/s)。
- 吞吐量：实际每秒传的数据量。
- 时延 = 发送时延 + 传播时延 + 处理 + 排队。
- 发送（传输）时延 = 数据长度 / 发送速率。
- 传播时延 = 信道长度 / 信号传播速率（约 2×10^8 m/s，光速量级）。
- （易错点：发送时延是"发完要多久"，传播时延是"传到对方要多久"，别混。）
- 往返时延 RTT：发送端从发出到收到确认/响应的时间（ $\approx 2 \times$ 单程传播时延 + 处理）。
- 利用率：发送方发送速率 / 带宽。

本项目落点：下载录音时首页 Content-Length / 下载进度就是你算"带宽×时间"的最真实场景。

N1 练习题（时延计算）

Q1：数据 1000B，带宽 10Mbps，信道长 2000km，传播速率 2×10^8 m/s，求发送时延与传播时延。解析：发送时延 = $(1000 \times 8) \text{ bit} / 10 \times 10^6 \text{ bps} = 8000 / 10^7 = 0.8 \text{ ms}$ 。传播时延 = $2000 \times 10^3 / 2 \times 10^8 = 10^{-2} \text{ s} = 10 \text{ ms}$ 。（注意传播时延通常远大于发送时延，这是"长链路传输"的基础事实。） Q2：什么情况下传播时延大于发送时

延？**解析**：数据很小 + 链路很长时（如 1bit / 千公里）→ 传播时延主导。它决定"存多少数据在途中"（带宽延迟积）。

N2 物理层

2.1 编码与调制

- **数字-数字编码**（数字信号传输）：**曼彻斯特编码**（每个码元中间跳变压/电平，自带时钟）、**差分曼彻斯特**（靠跳变有无表 0/1）、**不归零法 NRZ**、**归零 RZ**、**8B/10B / 4B/5B** 等。
- **数字-模拟调制**（用载波的幅度/频率/相位表示 0/1）：
- **ASK** 幅移键控、**FSK** 频移键控、**PSK** 相移键控、**QAM** 正交调幅（混合幅度+相位，如一码元多位）。
- WiFi 用的是 **OFDM（正交频分复用）**——把信道分成多个子载波并行传，属"分频 + 相/幅调制"。

2.2 复用技术（★考计算/概念）

- **频分复用 FDM**：不同频率互不干扰（收音机）。
- **时分复用 TDM**：同一频率分时隙（时隙轮着用）。
- **波分复用 WDM**：不同光波（光纤）。
- **码分复用 CDM（CDMA）**：用户用**正交码**叠加在同一时间同一频率，接收端用"相关"分离。**算题常考**：码片序列点积，如要 A 收 B，A 用 B 的码片序列做内积求和判 0/正/负。

CDMA 例题：码片 $a=(1,-1,1,-1)$ ， $b=(-1,1,-1,1)$ ，它们正交吗？**解析**： $a \cdot b = 1 \cdot (-1) + (-1)(1) + 1 \cdot (-1) + (-1)(1) = -1 - 1 - 1 - 1 = -4 \neq 0 \rightarrow$ 不正交。（正交码片要求内积=0 才能同时传；编码后的叠加用目标码片内积可还原自己的数据。）

2.3 传输介质与极限速率

- 双绞线（差分、抗干扰）、同轴、**光纤**（全反射、带宽大）、无线（电磁波）。
- **奈奎斯特准则**（无噪声，码元速率上限）： $C = 2W \log_2 V$ （ W =带宽， V =码元电平数/进制）。
- **香农定理**（有噪声，理论最大速率）： $C = W \log_2(1 + S/N)$ ，其中 $S/N = 10^{(\text{信噪比dB}/10)}$ 。
- **这两条必背计算**：题干给"带宽/信噪比/电平数"，求 bps 上限。

香农例题：带宽 3kHz，信噪比 30dB，理论最大速率？**解析**：30dB $\rightarrow S/N = 10^3 = 1000$ 。 $C = 3000 \times \log_2(1+1000) \approx 3000 \times 9.97 \approx 30\text{kbps}$ 。

N2 练习题

Q1（香农）：带宽 4kHz，信噪比 20dB，最大数据率？**解析**：20dB $\rightarrow S/N = 10^2 = 100$ 。 $C = 4000 \times \log_2(101) \approx 4000 \times 6.66 \approx 26.6\text{kbps}$ 。**Q2（Nyquist）**：带宽 3kHz，采用 8 种电平（ $V=8$ ），最大码元速率与数据率？**解析**：码元速率 $= 2W = 2 \times 3000 = 6000$ 码元/s（波特）。数据率 $= 6000 \times \log_2 8 = 6000 \times 3 = 18\text{kbps}$ 。（Nyquist 看"电平数"给多进制速率。）

N3 数据链路层（★CRC、窗口、CSMA 是大题）

3.1 封装成帧与链路标识

- 帧 = 首部 + 数据（负载）+ 尾部（FCS校验）。
- 帧定界：字符填充（字节填充）/ 位填充 / 物理编码违例。
- MAC 地址（48 位）/ 广、单、组播；本项目 WiFi MAC → 配网 SSID 后加 4 位。

3.2 差错检测（★CRC 会算）

- 奇偶校验：1 位，只能检奇数位错，不能纠错。
- 海明码：能纠 1 位错（见计组 C2）。
- CRC（循环冗余）：发送方用生成多项式除数据得余数附后；接收方同样除、余数=0 则正确。能检测多位错。以太网 FCS 用的即 CRC-32。要会手算多项式除法（模 2 除）（见 C2 Q5）。本项目 FRC 文件头/SPIFFS/网络帧都用 CRC 校验。

3.3 可靠传输：停止等待、GBN、SR（★窗口计算）

假设每帧长度为 L ，发送速率 v ，信道单程传播时延为 τ ($RTT=2\tau$)：

- 停止-等待（停等）：发一帧 → 等 ACK → 再发下一帧。利用率 $U = \text{发送时延} / (\text{发送时延} + 2\tau)$ （含帧全部传完到 ACK 回来）。
- 流水线/滑动窗口：允许同时多个帧在途，提高利用率。
- GBN（回退 N / Go-Back-N）：接收方按序收，失序帧丢弃；发送方窗口 > 1 ；一个 ACK 丢失时回退重传从丢的那个之后所有帧。需要窗口 $\leq 2^n - 1$ ($n = \text{序号位数}$) 防止序号重名/混乱。
- SR（选择重传 / Selective Repeat）：接收方缓存失序帧，只重传没 ACK 的那帧；窗口 $\leq 2^n - 1$ 。
- TCP 用的就是“滑动窗口 + 累计确认 +（快重传/快恢复）”（见 N5）。

例题（停止等待利用率）：传播时延 1ms，数据 1000B，带宽 10Mbps，利用率？解析：发送时延 $= 1000 \times 8 / 10M = 0.8ms$ ； $U = 0.8 / (0.8 + 2 \times 1) = 0.8 / 2.8 \approx 28.6\%$ 。（若启用 N 帧流水线： $U = N \times (\text{发送}) / (\text{发送} + 2\tau)$ 。这个提升是 TCP 窗口的意义。）

流水线窗口（GBN/SR）核心公式与上限（必背，过年题）：- 信道利用率（N 帧流水线）： $U = N \times T_{\text{发送}} / (T_{\text{发送}} + 2\tau)$ ，N 为发送窗口大小（在途帧数）。- 为使链路不空等，需要的最小窗口：让“发出 N 帧用时 $\geq$ 一个往返时间”→ 取 $N \geq (2\tau + T_{\text{发送}}) / T_{\text{发送}}$ ，常化为“ $N \geq \lceil 2\tau / T_{\text{发送}} + 1 \rceil$ ”。- 序号 n 位的最大窗口：GBN $\leq 2^n - 1$ ；SR $\leq 2^{n-1}$ 。（成因：接收方按序确认会因序号循环重名；GBN 未确认窗口越界、SR 收发双窗口需各占一半序号空间。）- 能识别出“窗口与序号够不够”：给 $n=3$ 位序号 + 窗口大小，判断会不会“序号混淆”——窗口太大就会把旧帧误当新帧（408 常考）。

综合例题（利用率+窗口，实打实算一遍）：帧长 1000B，速率 1Mbps，单向传播时延 25ms。① 停止-等待利用率？解析： $T_{\text{发送}} = (1000 \times 8) / 1 \times 10^6 = 8ms$ ； $RTT = 2\tau = 50ms$ 。 $U = 8 / (8 + 50) = 8 / 58 \approx 13.8\%$ 。② 用 GBN 且让利用率 $\geq 90\%$ ，至少开多大窗口（帧数为整数）？解析： $U = N \times 8 / (8 + 50) \geq 0.9 \rightarrow N \times 8 \geq 0.9 \times 58 = 52.2 \rightarrow N \geq 6.525 \rightarrow N = 7$ 。③ 若窗口=7，至少需要多少位序号（用 GBN）？解析：需 $2^n - 1 \geq 7 \rightarrow 2^n \geq 8 \rightarrow n = 3$ 位（此时可支持最大窗口 7，恰好够）。④ 若改 SR 且窗口=7，需几位序号？解析：需 $2^{n-1} \geq 7 \rightarrow 2^n \geq 14 \rightarrow n = 4$ 位（ $2^3 = 8 \geq 7$ ）。

为什么窗口上限不同（一步理解）：序号是模 2^n 循环的。GBN 发送方只需区分"未确认窗口内的帧"与"新帧"，保证窗口 $<$ 序号空间即可，故 $\leq 2^n - 1$ ；SR 收发都要缓存、接收方需能区分新旧，双窗口加起来不能超过整个序号空间，故仅用一半 $\rightarrow$ 窗口 $\leq 2^{n-1}$ 。

3.4 ★介质访问控制（随机访问 MAC）

- **信道共享方式分类：**随机访问（ALOHA/CSMA/CSMA-CD）、受控访问（轮询/令牌）、信道复用（FDM/TDM，见 N2）。
- **CSMA（载波监听）：**
 - 1 坚持 CSMA：监听忙就一直监听，空闲立即发。
 - 非坚持 CSMA：监听到忙就等随机时间再重试（减少冲突但增加延迟）。
 - p 坚持 CSMA。
- **CSMA/CD（有线以太网）：**先听后发、边发边听，检测到冲突立即停止并退避重发（指数退避）。
- **为什么要求最短帧长：**要"发送方在发完一个帧之前能发现冲突"，则要求 发送时延(最短帧长/速率) $\geq 2\tau$ （一个 RTT，保证冲突信号能传回来）。这是计算最短帧长的大题。
- **CSMA/CA（无线 802.11，本项目 WiFi）：**无线无法边发边听（自己发会盖过收到），于是：**先听 DIFS(帧间间隔) $\rightarrow$ 随机退避 $\rightarrow$ 发 $\rightarrow$ 等 ACK**；可配合 RTS/CTS 预留信道解决隐藏终端问题。
- **ALOHA：**不监听直接发，冲突概率大，利用率低。

CSMA/CD 最短帧长例题：网络最长距离 2500m，速率 100Mbps，传播速率 2×10^8 m/s，求最短帧长。**解析：**单程传播时延 $= 2500 / 2 \times 10^8 = 12.5 \mu s$ ， $2\tau = 25 \mu s$ 。最短帧长 $= \text{速率} \times 2\tau = 100 \times 10^6 \times 25 \times 10^{-6} = 2500 \text{ bit} \approx 312.5 \text{B}$ 。 $\rightarrow$ 大题的经典解。

3.5 以太网、VLAN

- **以太网（802.3）：**帧结构（前导码+DA+SA+类型/长度+数据+FCS）、MAC 48 位。
- **VLAN（虚拟局域网，802.1Q）：**给帧加 4 字节 Tag（含 VLAN ID），把一台交换机切成多个逻辑广播域。

N3 练习题

Q1（GBN 序号范围）：帧序号 3 位，GBN 发送窗口最大多少？SR 呢？**解析：**GBN：窗口 $\leq 2^3 - 1 = 7$ ；SR：窗口 $\leq 2^{3-1} = 4$ 。（这是**防序号重名**的经典限制，必背。）**Q2**（CSMA/CA 为何用 ACK）：无线网为什么发完要等 ACK？**解析：**无线**无法同时收发**（不能边发边听，故没有 CSMA/CD 的冲突检测）；收方收到后回 ACK 作为"真的送到"的证据；若不回就重发。 $\rightarrow$ 无线用 **CSMA/CA（碰撞避免）+ ACK**，有线用 CSMA/CD（碰撞检测）。**Q3**（最短帧长）：速率 10Mbps， $2\tau = 50 \mu s$ ，最短帧长？**解析：**最短帧长 $= 10 \times 10^6 \times 50 \times 10^{-6} = 500 \text{ bit} = 62.5 \text{B}$ 。

N4 网络层（★IP、子网划分、路由是大题）

4.1 IPv4 与地址分类（背）

- **IPv4：**32 位，点分十进制（4 组 0~255）。
- **分类编址：**

- A类：0 开头，网络号 8 位，0.0.0.0 ~ 127.255.255.255，主机号 24 位。
- B类：10 开头，16/16，128~191。
- C类：110 开头，24 网络 + 8 主机，192~223。
- D：组播 224~；E：保留 240~。
- 私有地址（RFC1918，本项目 192.168.4.x 就是）：
- 10.0.0.0/8、172.16.0.0/12、192.168.0.0/16。内网可用、不能直接上公网（需 NAT）。
- 主机号全 0 = 网络地址，全 1 = 广播地址，都不能分配给主机 → 可用主机数 = $2^h - 2$ 。

4.2 ★子网划分与 CIDR（送分计算，务必刷熟）

子网划分：借用若干主机位作子网号。给定 IP/掩码：- 块大小 = 256 - (非255的掩码字节)。- 网络地址 = IP 与掩码逐位与。- 广播地址 = 网络地址 + 块大小 - 1（或把主机位全置 1）。- 可用主机数 = $2^h(\text{主机位数}) - 2$ 。

CIDR（无类别域间路由）：用 /n 表示前缀长度。192.168.4.0/24 表示网络号 24 位，主机 8 位。

例题：192.168.4.66/26：- /26 → 掩码 255.255.255.192；块大小 = 256 - 192 = 64。- 66/64 = 1...2 → 网络地址 = 192.168.4.0 + 64 = 192.168.4.64。- 广播 = 下一个块 - 1 = 192.168.4.127。- 可用主机 = 64 - 2 = 62。

（背三句：块大小=256-掩码字节 → 网络=第一个被包含的块 → 广播=下一块-1 → 可用=块-2。）

4.3 ARP / DHCP / ICMP / 路由（背流程）

- ARP：IP → MAC。同网段内：广播查（“谁有 192.168.4.1？”）→ 目标单播答 → 双方缓存（ARP 缓存）。跨网段则查路由/网关。ARP 只在本广播域有效。
- DHCP（动态主机配置，本项目配网核心）：客户 广播 Discover → 服务器 Offer → 客户 Request → 服务器 Ack（4 次交互），分配 IP/网关/DNS 及租期。本项目手机给设备填 Wi-Fi 密码后，设备就是“STA 客户端”去 DHCP 拿内网 IP。
- ICMP：ping 用 ICMP Echo Request/Reply 测连通性与 RTT；traceroute 用它。
- 路由算法：
- RIP（距离向量，DV）：用“跳数”，每跳 1，最大 15 跳；周期性交换路由表（Bellman-Ford）。慢收敛、有“无穷计数”。
- OSPF（链路状态，LS）：全网同步链路状态，用 Dijkstra 算最短路径（带权），收敛快、分区域。
- BGP（自治系统间）：路径向量，策略路由。

4.4 NAT（网络地址转换）

- 内网私有 IP 通过网关 NAT 映射成公网 IP + 端口（端口 区分不同内网主机）→ 内网可上网同时隐藏拓扑。本项目设备连路由器上网即经 NAT。

N4 练习题

Q1（子网）：200.100.20.130/27 网络地址、广播、可用主机数？解析：/27 → 掩码 255.255.255.224，块 = 256 - 224 = 32。130/32 = 4...2 → 网络 = 0 + 4 × 32 = 200.100.20.128；广播 = 128 + 32 - 1 = 200.100.20.159；可用 = 32 - 2 = 30。Q2（子网数量）：把 192.168.5.0/24 借 3 位子网划分，可得几个子网、每个子网多少主机？解析：借 3 → 子网数 = $2^3 = 8$ ；每个子网主机位数 = 8 - 3 = 5 → 每子网可用主机 = $2^5 - 2 = 30$ 。Q3（ARP 作用域）：跨网段主机 A → B 能直接用 ARP 拿到 B 的 MAC 吗？解析：不行，ARP 只在本网段（广播域）有效；跨网段需

先把包发往**默认网关/路由器**，由网关转发，A 只与"下一跳"通信。→ 节点本地路由表选下一跳，再 ARP 解析下一跳的 MAC。

N5 传输层（★TCP 可靠/流量/拥塞控制是全年大题必守）

5.1 UDP vs TCP 对比（背）

	UDP	TCP
连接	无连接	面向连接
可靠性	不保证（尽力而为）	可靠（确认、重传、序号）
首部	8B	20B+
流量/拥塞	无	有
用途	DNS(53)/SNTP(123)/DHCP/音视频	HTTP、FTP、SMTP
本项目	SNTP 校时、DHCP	配网 HTTP / 下载

5.2 TCP 报文段（背关键字段）

- 源/目的端口、序号 seq、确认号 ack、窗口大小、标志位：SYN/ACK/FIN/RST/PSH/URG。
- 三次握手（建立连接）：客户端 —— SYN(seq=x) ——> 服务器 客户端
← SYN+ACK(seq=y, ack=x+1) —— 服务器 客户端 —— ACK(ack=y+1) ——>
服务器 → 双方都知道"我能收发、对方能收发"。本项目每次 HTTP 连接都经历。
- 四次挥手（释放）：客户端 —— FIN ——> 服务器 客户端 ← ACK —— 服务器 客户端 ← FIN —— 服
务器 客户端 —— ACK(进入 TIME_WAIT 2MSL)——> 服务器 → 半关闭、最后等 2MSL 确保旧包过期。

5.3 可靠传输与流量控制（★计算）

- 滑动窗口 + 累计确认：发送窗口决定"能发多少在途"；收方按序收，整体确认"到序号 n-1"都收到 (ack=n)。
- 确认号语义（必背）：ack=n 表示"期望下一段的起始序号是 n"，即 0~n-1 都已正确收到。
- 流量控制：接收方在 ACK 里写自己的接收窗口 (rwnd)，限制发送方一次最多发多少，防止把自己冲垮。

例题（序号确认）：每段 1000B，发 seq=1000，收 ack=2000 → 实际确认到多少字节？下一段 seq 多少？
解析：ack=2000 表示 1000~1999 这 1000B 已收到并确认，下一个应发 seq=2000。→ "确认号=已收字节+1"。

5.4 ★TCP 拥塞控制（★状态机，背死）

变量：拥塞窗口 cwnd、慢启动阈值 ssthresh。- 慢启动（Slow Start）：cwnd 从 1 个 MSS 开始，每个 RTT 翻倍（指数），直到 cwnd ≥ ssthresh 转拥塞避免。- 拥塞避免：每 RTT +1（线性加），AIMD 的"加性增"。- 遇超时（丢包）：ssthresh=cwnd/2，cwnd=1，回到慢启动。- 快重传：连收 3 个重复 ACK 立即重传（不等超时）。- 快恢复：ssthresh=cwnd/2，cwnd=ssthresh，直接进拥塞避免（不回到 1）。（这是"斩半而非归零"的关键区别。）- 整体策略：慢启动指数冲、拥塞避免线性加、丢包减半（或归零重来）= 加性增、乘性减 AIMD。- 两个窗口别混：流量控制看 rwnd（接收方窗口），拥塞控制看 cwnd（发送方估网络）；实际能发的最小值 = min(cwnd, rwnd)。这是 408 高频辨析。

例题 (cwnd 轨迹, 务必自己画一遍): ssthresh=16, 慢启动 cwnd=1,2,4,8,16 → 到 16 转拥塞避免: 17,18,19,20。假设在 cwnd=20 时**超时** → ssthresh=20/2=10, **cwnd=1 重慢启动**: 1,2,4,8 → 到 10 就转拥塞避免 (因为 ssthresh=10): 10,11,12... (若是**快重传**: ssthresh=20/2=10, cwnd=10, 直接进拥塞避免 11,12...) **要点**: 区分"超时→cwnd=1"和"快恢复→cwnd=ssthresh"是判定题的核心。

进阶例题 (分段轨迹, 408 大题常见): 设 ssthresh 初始 32, 某时刻 cwnd=34 时收到 3 个重复 ACK。- ① 此时进入**快恢复**: ssthresh=cwnd/2=17, cwnd=ssthresh=17, 进入拥塞避免。- ② 之后每个 RTT cwnd 线性 +1: 18,19,20,... - ③ 若到 cwnd=24 时**超时**: ssthresh=cwnd/2=12, cwnd=1 重回慢启动: 1,2,4,8,12(到阈值转避免),13,14... - **命题角度**: 让你填表列每个 RTT 后的 cwnd; 或问 cwnd=34 时为何没有指数增长 (因为已远在 ssthresh 之上走避免)。熟记"哪几个转折点改变模式"就不会错。

拥塞控制与"标注每个区段名称"题: 给一张 cwnd 随时间锯齿状图, 让写出各段对应"慢启动/拥塞避免/快恢复/超时重启"。识别法: - 斜率陡的上升期 (×2 per RTT) = 慢启动; - 平缓上升期 (+1 per RTT) = 拥塞避免; - 突然垂直跌落到一半、又平缓上升 = 快恢复 (或 3 重复 ACK); - 突然跌到 1、再次慢启动 = 超时重启。

N5 练习题

Q1 (拥塞判断): TCP 收到 3 个重复 ACK 后如何调整 cwnd 和 ssthresh? **解析**: 判为**快重传/快恢复**: ssthresh=cwnd/2, cwnd=ssthresh, 进入拥塞避免。**不归 1**。→ 这是 408 必考判别。 **Q2 (握手目的)**: 为何 TCP 要三次握手不是两次? **解析**: 两次做不到"服务器确认客户端也收到了它 SYN+ACK 的确认", 可能导致**已失效的连接请求**再次被当成新连接, 浪费资源。(防"旧 SYN 重入"; 三次才双向确认。) **Q3 (小明配网场景)**: 设备向手机 HTTP 服务器下载录音, 传输层是 TCP。若中间丢一包, TCP 如何保证可靠? **解析**: 发送方收不到 ACK 或收 3 重复 ACK → 重传; 通过**序号 + 确认 + 重传** + 接收方按序缓存, 最终保证字节流完整有序。这就是"可靠传输"。 **Q4 (MSS/RTT 吞吐, 硬算)**: 某 TCP 连接, MSS=1000B, RTT=10ms, 接收窗口足够大, 进入拥塞避免后 cwnd 稳定在 20 个 MSS, 求吞吐量。 **解析**: 每 RTT 发 cwnd=20 MSS = 20000B, RTT=0.01s → 吞吐 = 20000/0.01 = **2 MB/s (2×10⁶ B/s)**。套路: 吞吐 ≈ cwnd×MSS / RTT。 **Q5 (滑动窗口+流量控制综合)**: 接收方通告 rwnd=5000B, 发送方 cwnd 也="足", 每段 1000B, 问最多同时几个段在途? **解析**: 能发最小值 = min(cwnd, rwnd) = 5000B → 5000/1000 = **5 个段在途** (受 rwnd 上限约束)。

N6 应用层 (★HTTP 讲透, 本项目配网/下载全靠它)

6.1 DNS (域名系统)

- 域名是**层次树**: 根 → 顶级 (.com/.cn) → 二级 (baidu.com) → 子域 (www)。
- **解析方式**: 递归查询 (问根一次帮你追到底) vs **迭代查询** (逐级返回下一个要问的服务器自己问)。
- **类型**: A (IPv4)、AAAA (IPv6)、CNAME (别名)、MX (邮件)、NS (域名服务器)。
- 通常用 **UDP:53**, 数据大才用 TCP。缓存减少查询。
- 本项目配网走 IP 直连 192.168.4.1, 不依赖公共 DNS; 联网后 **SNTP** 才需时间服务器域名。

6.2 ★HTTP (本项目配网 + 下载的核心协议)

- 特性: **工作在 TCP 之上**, 默认端口 **80** (本项目 STUDY_WIFI_AP_PORT=80)。
- **请求报文结构**: 请求行(方法 URL 版本) + 首部行 + 空行 + 请求体。

- GET /rec/dl?seq=7 HTTP/1.1 这类就是请求行，?seq=7 是**查询串 (query)**，是"把参数传给服务器"的方式。
- **方法 (首字母必背)**：
 - **GET**：取资源 (本项目浏览器拉取页面、下载录音)。
 - **POST**：提交数据 (本项目配网表单 POST /save 提交 SSID/密码)。
 - PUT/DELETE/HEAD/OPTIONS。
- **状态码**：
 - 1xx 信息；2xx 成功 (**200 OK**)；3xx 重定向；4xx 客户端错 (**404 Not Found**)；5xx 服务端错 (**500**)。
- **常见首部**：
 - Host、Content-Type (如 application/json、audio/wav)、Content-Length、Content-Disposition: attachment; filename=... (**下载时让浏览器存成文件**)、Transfer-Encoding: chunked (**分块流式传输**，本项目下载时逐块发)。
- **连接方式**：
 - **非持久连接**：每个资源一次 TCP (老 HTTP/1.0)。
 - **持久连接 (keep-alive)**：同一 TCP 用多次请求 (HTTP/1.1 默认，本项目配网页/下载都尽量复用连接)。
 - **本项目交互** (把整个 N6.2 落地)：1) 设备无凭证 → 开 **SoftAP 热点** (SSID=自动按 MAC 生成，密码显示在屏)，手机连上，浏览器开 http://192.168.4.1。2) 页面 GET / 返回 HTML 表单；用户提交 POST /save (带 ssid=...&pass=...)；服务器解析 → 存 NVS → 关 AP → 设备去连那个 Wi-Fi。3) 联网后设备是 **HTTP Server**，监听 80 端口；手机 GET /rec/dl?seq=N → 服务器读 FRC → 解码成 WAV → 拼 WAV 头 + **chunked 分块流式响应** → 手机存成 R0000007.wav。

6.3 其他应用层协议 (背)

- **FTP**：控制连接 21 + 数据连接 20 (带外传输)，两条连接。
- **电子邮件**：发送 SMTP (25/465)，接收 POP3 (110) 或 IMAP (143)，MIME 支持附件/编码。
- **SNMP**：网络管理 (161/162)。
- **DHCP 客户端**：见 N4.3 (本项目配网的核心)。

N6 练习题

Q1 (状态码)：GET 一个不存在的 /rec/dl?seq=999，服务器应返回？**解析**：**404 Not Found** (4xx 客户端错，找不到资源)。**Q2** (DNS)：www.baidu.com 的解析，根服务器直接给你最终 IP 吗？**解析**：不一定。**递归**会帮你查到底 (先根、再 .com、再 baidu.com)；**迭代**则只告诉你"下一步问 .com"的服务器，由你继续问。协议用 UDP:53。**Q3** (下载流程真相)：手机下载录音时，浏览器为何能存成 WAV 文件而不是在网页里显示？**解析**：靠响应 Content-Disposition: attachment; filename=R0000007.wav 首部，浏览器收到就触发"另存为/下载"，而不是把音频塞进页面 → 这正好是 N6.2 的工程体现。**Q4** (chunked 意义)：下载大数据为什么用 Transfer-Encoding: chunked 而不是固定长度？**解析**：当服务器**事先不知道总长度** (如一边解码一边发)、或想**边生成边发**时，用 chunked 分块，每块自带长度、最后用空块结束。→ 避免必须先把整个文件算好再发。本项目录音解码是边读边发，正好适合 chunked。

[第四部分小结：计网"一张分层图 + 四类必算"（背）]

分层图：物理→链路→网络→传输→应用；对等层通信、下层封装。**四类必算：**1. 时延：发送=数据/速率，传播=距离/速率，RTT。2. 香农/奈奎斯特： $C=W \log_2(1+S/N)$ 、 $C=2W \log_2 V$ 。3. CSMA/CD 最短帧长：帧长 $\geq$ 速率 $\times 2\tau$ ；复用/CDMA 内积。4. 子网划分：块大小=256-掩码，网络/广播/可用主机；TCP cwnd 走表。

第五大部分 专题回顾 + 综合演练（把四科串起来）

单科学完，用项目里真实发生过的三个场景把跨学科考点串一遍——这是把“一块一块的知识”变成“一张网”的关键一步。

◆ 专题一：按一次按键，发生了什么？（跨计组 + OS + 数据结构）

物理过程与知识点映射：

1. **物理层/周围硬件**：三键共用一路 **ADC（模数转换）** 按分压判键；静电/WiFi 射频会引入电压抖动 → **采样抗噪**（多采样平均、软件消抖）——属于计组“I/O + 校验/判稳”。- 关联：本项目 `bsp_button.c` 的多采样平均 → 本质是**时间/幅值滤波**，对应计网“噪声”、计组“输入去抖”。
2. **中断驱动 I/O**：定时器周期中断读 ADC（ISR 极短）→ 产生按键事件 → 这是计组 C5.4 / C7 的**中断驱动 I/O**（对比“程序查询”和“DMA”）。
3. **临界区与互斥（OS）**：判决线程回调更新 UI，UI 是**共享资源**，要用 `bsp_lvgl_lock(timeout)`（互斥量 + 优先级继承）互斥访问 → OS O2.5 的**临界区/信号量**。
4. **事件/生产者-消费者（DS + OS）**：按键事件压入**环形队列**（DS D3 循环队列判空/判满），消费者（UI 任务）`poll_evt` 出队 → OS 的**生产-消费同步**（有界缓冲 + 信号量思想）。
5. **CPU 与流水（计组）**：每一次回调里的指令在 CPU 上**取指→译码→执行**，遇到分支（`switch/if`）走 `beq`，函数调用走 `jal` + 压栈 → 计组 C4/C5。
6. **幽灵点击（纠错 + 稳定性）**：250ms 窗口内同/异键连按被判为“幽灵”丢弃——这是**软件滤波 / 去抖**，工程层面就是“状态机 + 时间窗”，也是稳定性/健壮性考点（不是单科，但用到了数据结构的时间窗判断）。

一句话总结：一次按键 = ADC(物/I) → 中断(计组) → 入队(DS) → 互斥/出队(OS) → UI 绘制(计组 CPU)。四科在同一件事上咬合。

◆ 专题二：录音 16kHz 崩溃（ESP_RST_PANIC）背后是哪个考点？（跨计组 + OS）

这是一个真实排障场景——它在考场上就是一道最漂亮的综合大题：

- **计组 C1/C3（内存布局）**：代码/常量在 Flash，变量/堆/栈在 SRAM。**任务栈是 RAM 里的一块连续区**。
- **OS O3（内存管理）**：ESP 用**动态堆 + 连续分配**，出现**外部碎片**：空闲堆约 33KB、但**最大连续块只有 16KB**。16KB 大任务栈申请不连续 → 分配失败。
- **计组 C5（异常）**：`malloc/xTaskCreate` 失败后若继续用，会触发**非法内存访问** → **中止（abort）** → IDF 复位重启 → `reset_reason=PANIC`。

- **计组 C3（局部性/缓冲见上）**：Speex WB 编码器要较多堆 → 堆紧张。这解释了为何改成小内存的 8k NB 版本就没事：够放得下。
- **OS（任务栈水位）**：`uxTaskGetStackHighWaterMark()` 看"现场够不够"——栈 HWM=4294967295 (0xFFFFFFFF) 表示该任务还没创建/未启动（无效数据）→ 提醒"任务还没跑起来，别把 HWM 当真实栈余判断"。

答大题模板：① 写出内存去向（Flash 代码 + SRAM 变量/堆/栈）；② 指出外部碎片/连续分配导致大连续块不足；③ 分配失败 + 继续使用 → 非法访问 → abort → 复位；④ 对策：减小任务栈、压缩堆、用碎片整理/位图管理、或回退 8k 版本。

◆ 专题三：配网 + 下载录音 = 整个计网章节的"活体演练"

端到端完整流程（把 N1~N6 全部走一遍）：1. 设备开 **SoftAP**（192.168.4.1，私有地址 → N4.1/NAT）。手机连上这个 WiFi → 走 802.11 **CSMA/CA + ACK**（N3.4）。2. 浏览器 GET `http://192.168.4.1/` → 设备起 **HTTP Server**（TCP:80）。HTTP 走 TCP → **三次握手** → **一次请求/响应** → **（HTTP/1.1 keep-alive 复用连接）**（N5.2、N6.2）。3. 用户填 SSID/密码 **POST /save**（N6.2 方法），传 `ssid=..&pass=..`（查询/表单），服务器 `parse_form` 拆串（DS D4 串处理）→ NVS 持久化（OS O4）。4. 设备关 AP、以 **STA 连路由器** → **DHCP 广播发 Discover/Offer/Request/Ack（4 次）** 拿到内网 IP（N4.3）。5. 手机下载：GET `/rec/dl?seq=N` → 服务器读 FRC → 解码 WAV → **chunked 分块响应** + **Content-Disposition** → 手机存 `R....wav`（N6.2）。6. 全程由 LwIP 提供 **TCP 可靠传输**（N5.3）：丢包靠序号+ACK+重传。

一句话：从配网到下载，整个计网章节（分层、介质、MAC、IP/子网、TCP 握手、HTTP、DNS/DHCP）就是这台设备的日常。把每个"擦边点名"记下来，这一科就通了。

第六大部分 全书复盘 & 考前冲刺清单

6.1 三科主线的"一口闷"记忆卡

- **数据结构主线**：数据结构 = 逻辑结构 × 存储结构 × 运算；选择结构看操作频率；每个结构记"判空/判满/复杂度"。
- **计组主线**：C ← (编译/汇编/链接) → bin ← 装载 → CPU 取指 → 译码 → 执行 ← 存储层次从 Flash 到 Cache 到寄存器。
- **OS主线**：进程 (PCB) → 调度 → 同步/互斥 (PV) = 花最长时间 → 死锁 (银行家) → 内存 (分页/置换) → 文件 (索引/磁盘) → I/O (DMA)。
- **计网主线**：分层 → 每个报文加"首部" → 从手机；IP/TCP/HTTP 三层是考点重地。

6.2 最容易记混 & 高频陷阱 (按科列出, 考前必扫)

数据结构 - 循环 $\times 2 = \log n$ ；双层全 $\times 2 = (\log n)^2$ ，别当 $n \log n$ 。- KMP 的优势是**主串不回溯**；next 是模式串最长相等前后缀。- 完全二叉树编号 $2i/2i+1/\lfloor i/2 \rfloor$ ； $n_0 = n_2 + 1$ 。- 排序稳定=冒插归基；不稳定=快选堆希尔；比较排序下界 $\Omega(n \log n)$ 。

计组 - 主频高 $\neq$ 快，要比 CPI；CPU 时间=指令数 $\times$ CPI $\times$ 时钟周期。- 补码"取反+1"；符号扩展正补 0 负补 1。- 溢出：**异号相加永不溢出**，同号才可能。- IEEE754 阶码偏置 127、隐含 1；NaN/ ∞ 的编码。- Cache：Tag=A-b-s；写回靠脏位；AMAT 公式。- 流水：T=5+(n-1)；转发→数据冒险、预测→控制冒险、分离存储→结构冒险。- 中断/异常：fault 重执行、trap 下一条、abort 系统复位 (PANIC)。- DMA 数据**不经过 CPU**，只在外设↔主存之间。

OS - 状态转换没有"就绪→阻塞"。- SJF 平均等待最短；RR 时间片太大会退化 FCFS。- 死锁四条件；银行家手算安全序列。- 分页=块号拼偏移；LRU vs FIFO；Belady 只在 FIFO。- 混合索引：直接+间接的容量、访存次数。- 磁盘调度 SSTF 会"来回" (饥饿)；SCAN 电梯式。

计网 - 发送时延 = 数据/速率；传播时延 = 距离/速率，二者别混。- 香农看信噪比、奈奎斯特看电平数。- 子网：块大小=256-掩码字节；网络=包含的块；广播=下块-1；可用=块-2。- TCP 确认号=已收字节+1；完成三次握手 (防旧 SYN)。- 拥塞：超时→cwnd=1；快恢复→cwnd=ssthresh。- CSMA/CD 检测、CSMA/CA 避免，别记反。

6.3 考点优先级 (按 408 历年出现频率)

必背必算 (选择题/计算题送分)：- 数据结构：循环队列判空判满、哈希 ASL、排序复杂度与稳定性、树/哈夫曼。- 计组：补码/溢出、Cache 容量位数、IEEE754、流水线周期、指令编码、DMA。- OS：调度平均等待、PV 生产者-消费者、银行家、分页换算、页面置换、inode、磁盘调度。- 计网：时延、香农/奈奎斯特、子网划分、TCP cwnd、对称加密 (HTTP/DNS)。

重点大题：- 计组：Cache 计算、流水冒险、指令编码、中断流程。- OS：PV 大题、银行家、页面置换、混合索引。- 计网：TCP 拥塞、子网、停止等待/窗口、最短帧长。

次重点 (能混眼熟即可)：- 物理层调制细节、VLAN 802.1Q、SNMP 端口、海明码排位 (选做) 等。

6.4 最后：复习节奏建议（回到总纲）

- **第 1 轮**：通读本文档，每个 ★ 例题动手推（不偷懒去看答案）。
- **第 2 轮**：只刷"必算"清单（补码、Cache、子网、cwnd、PV 模板），每天熟一套。
- **第 3 轮**：用第六大部分的"三个专题"自问互串，检验是否"一张网"。
- **错题本**：把每道算错的题，"用了哪个公式 + 为什么错"记一行，考前翻熟。

这台设备不是普通的嵌入式工程——它就是活的《408》。每按一次键、每录一段音、每下载一次，你都在复习计组、OS、数据结构、计网。祝 408 一战上岸！

本手册按《408 考试大纲》与王道/袁春风主线编写，案例数据取自 repo 内真实代码（ESP32-C3 / FreeRTOS / LwIP / SPIFFS / RISC-V）。如有题设与实际出入，以最新代码与官方教材为准。